使用 tikz-cartes 几何计算

2026 年 5 月 1 日

目录

第一章 概述 **1**

1.1 核心特性 1

1.2 平面向量运算命令 1

1.3 空间向量运算命令 2

1.4 直线与圆的相关运算 3

1.5 三角形的相关运算 4

1.6 圆锥曲线相关的运算 5

1.7 `\segment` 和 `\conic` 绘图命令 6

第二章 `tikzmath` 和 `pgfkeys` 简介 **9**

2.1 使用 `tikzmath` 9

2.1.1 基本用法 9

2.1.2 循环语句：`for` 10

2.1.3 条件判断：`if-then-else` 12

2.1.4 定义函数：`function` 12

2.1.5 注意事项与技巧 13

2.1.6 错误排除 15

2.2 `tikzmath` 的计算精度 15

2.3 使用 `pgfkeys` 20

2.3.1 核心概念 20

2.3.2 主要功能特性 20

2.3.3 基本用法示例 22

第三章 矩阵计算 Matrix Computations **25**

3.1 核心注意事项 (Caveats) 25

3.2 主要命令 25

3.2.1 矩阵基础运算 26

3.2.2 高级矩阵运算 26

3.2.3 特征值与特征向量 27
3.2.4 辅助与工具函数 27
3.3 使用示例 28
3.4 开发建议与注意事项 28
3.5 总结 29

第四章 向量运算 Vector Computations 31
4.1 位置向量 31
4.1.1 向量与位置向量 31
4.1.2 向量在 `cartes` 包中的表示 32
4.2 向量加法与减法（Vector Addition and Subtraction） 33
4.2.1 向量的加法与减法 33
4.2.2 语法形式 33
4.2.3 示例 1：平面平行四边形 33
4.2.4 示例 2：空间平行四边形 35
4.3 数乘/标量乘法（Scalar Multiplication） 36
4.3.1 向量的数乘 36
4.3.2 语法形式 36
4.3.3 示例 1：平面线段中点 36
4.3.4 示例 2：空间线段中点 37
4.4 向量的线性组合（Linear Combination） 38
4.4.1 向量的线性组合 38
4.4.2 语法形式 39
4.5 点积/数量积（Dot Product/Scalar Product） 39
4.5.1 向量的点积 39
4.5.2 语法形式 40
4.6 叉积/向量积（Cross Product） 40
4.6.1 向量叉乘 40
4.6.2 语法形式 41
4.7 向量的模（Magnitude/Length） 41
4.7.1 向量的模 41
4.7.2 语法形式 42
4.7.3 示例 1：已知圆心和圆上一点作圆 42
4.7.4 示例 2：在线段上截取指定长度 43
4.8 向量夹角 44
4.8.1 向量的夹角 44
4.8.2 语法形式 45
4.8.3 示例 1：平面向量夹角 45

4.8.4 示例 2：空间向量夹角 46
4.9 向量投影（Vector Projection）、标量投影（Scalar Projection）与向量拒绝（Vector Rejection） 47
4.9.1 向量投影 47
4.9.2 标量投影 48
4.9.3 向量拒绝 48
4.9.4 语法形式 49
4.9.5 示例 1：平面向量投影 49
4.9.6 示例 2：空间向量投影 51
4.10 向量反射（Vector Reflection） 52
4.10.1 向量反射 52
4.10.2 语法形式 53
4.10.3 示例 1：平面向量反射 53
4.10.4 示例 2：空间向量反射 54
4.11 平面向量旋转 55
4.11.1 平面向量的旋转 55
4.11.2 语法形式 56
4.11.3 示例 1：平面向量旋转 56
4.11.4 示例 2：费马点 57
4.12 空间向量旋转 59
4.12.1 罗德里格斯旋转公式 59
4.12.2 语法形式 60
4.12.3 示例：空间向量旋转 60

第五章 直线与圆 63
5.1 直线 63
5.1.1 原点在直线上的投影 63
5.1.2 直线的方向向量 63
5.1.3 语法形式 64
5.1.4 示例 64
5.2 两直线的交点 65
5.2.1 两直线的交点公式 65
5.2.2 语法形式 65
5.2.3 示例：笛卡尔坐标下两直线的交点 65
5.3 直线与圆的交点 67
5.3.1 直线与圆的交点公式 67
5.3.2 语法形式 68
5.3.3 示例 68

5.4 两圆的根轴与交点 69
5.4.1 根轴 69
5.4.2 语法形式 70
5.4.3 示例 1：求两圆的交点 71
5.4.4 示例 2：根心定理 72
5.5 圆外一点作圆的切线 74
5.5.1 圆的切线 74
5.5.2 语法形式 74
5.5.3 示例 74
5.6 两圆的位似中心与公切线 75
5.6.1 位似中心 75
5.6.2 公切线 77
5.6.3 语法形式 77
5.6.4 示例：蒙日定理 77
5.7 反演变换 79
5.7.1 反演点 79
5.7.2 语法形式 80
5.7.3 示例 1：证明托勒密定理 80
5.7.4 示例 2：Pappus Chain 82

第六章 三角形的中心 87
6.1 Barycentric Coordinates 87
6.1.1 重心坐标形式 87
6.1.2 语法形式 89
6.2 三角形的重心 Centroid 89
6.2.1 重心及其性质 89
6.2.2 语法形式 89
6.2.3 示例 89
6.3 三角形的外心 Circumcenter 91
6.3.1 外心及其性质 91
6.3.2 语法形式 92
6.3.3 示例 92
6.4 三角形的垂心 Orthocenter 93
6.4.1 垂心及其性质 93
6.4.2 语法形式 94
6.4.3 示例 94
6.5 三角形的内心 Incenter 95
6.5.1 内心及其性质 95

6.5.2 语法形式 95
6.5.3 示例 95
6.6 三角形的旁心 Excenter 97
6.6.1 旁心及其性质 97
6.6.2 语法形式 97
6.6.3 示例 97
6.7 综合示例：九点圆 99

第七章 圆锥曲线 Conics 103
7.1 齐次坐标与坐标变换 103
7.1.1 齐次坐标 103
7.1.2 向量叉乘 104
7.1.3 坐标变换 Transformations of Coordinates 106
7.1.4 坐标轴变换 Transformations of Axes 107
7.1.5 齐次坐标在 `cartes` 包中的表示 109
7.1.6 语法形式 109
7.1.7 示例：齐次坐标下两直线的交点 110
7.2 圆锥曲线方程的一般形式 111
7.2.1 圆锥曲线的系数矩阵 111
7.2.2 语法形式 112
7.2.3 示例：由方程系数确定的二次曲线 112
7.3 由焦点-准线定义推导圆锥曲线的系数矩阵 113
7.3.1 焦点-准线定义圆锥曲线 113
7.3.2 语法形式 115
7.3.3 示例：椭圆、抛物线和双曲线的焦点-准线定义 115
7.4 由焦点-距离定义推导椭圆/双曲线的系数矩阵 116
7.4.1 焦点-距离定义圆锥曲线 116
7.4.2 语法形式 117
7.4.3 示例 1：椭圆和双曲线的双焦点定义 117
7.4.4 示例 2：由焦点和曲线上一点确定椭圆或双曲线 119
7.5 由圆心-半径或三点确定圆的方程 120
7.5.1 三点确定的圆 120
7.5.2 语法形式 121
7.5.3 示例：圆的定义 121
7.6 化简一般圆锥曲线为标准方程 123
7.6.1 圆锥曲线的旋转变换 123
7.6.2 圆锥曲线的平移变换 125
7.6.3 圆锥曲线的中心 126

7.6.4 化简步骤 126
7.6.5 语法形式 129
7.7 五点确定的圆锥曲线 129
7.7.1 代数解法：齐次线性方程组 129
7.7.2 几何构造法：线性束（Pencil of Conics） 130
7.7.3 语法形式 133
7.7.4 示例 1：五点确定的椭圆 133
7.7.5 示例 2：五点确定的双曲线 134
7.7.6 示例 3：五点确定的抛物线 135
7.8 五条切线确定的圆锥曲线 137
7.8.1 对偶原理：从点到线 137
7.8.2 齐次坐标下的对偶表示 137
7.8.3 语法形式 139
7.8.4 示例 1：五条切线确定的椭圆 139
7.8.5 示例 2：五条切线确定的双曲线 141
7.8.6 示例 3：五条切线确定的抛物线 143
7.9 极点与极线（Pole and Polar Line） 145
7.9.1 极点与极线 145
7.9.2 语法形式 147
7.9.3 示例：极点与极线 147
7.10 直线与圆锥曲线的交点 150
7.10.1 求解交点的参数化方法 150
7.10.2 Richter-Gebert 算法 152
7.10.3 语法形式 152
7.10.4 示例 1：直线（两点）与圆锥曲线的交点 152
7.10.5 示例 2：直线（系数）与圆锥曲线的交点 153
7.11 圆锥曲线的切线 155
7.11.1 利用对偶曲线求切线 155
7.11.2 利用极线求切点 156
7.11.3 语法形式 157
7.11.4 示例：椭圆的切线 157
7.12 分解退化的二次曲线 159
7.12.1 退化的二次曲线 159
7.12.2 语法形式 160
7.13 两圆锥曲线的交点 160
7.13.1 二次曲线束 160
7.13.2 步骤详解 161
7.13.3 语法形式 162

7.13.4 示例：圆锥曲线束 162

参考文献 **165**

第一章　概述

tikz-cartes 是一个功能强大的 TikZ 扩展库，专为平面几何、空间向量以及圆锥曲线的解析几何计算与绘图而设计。通过引入强大的向量运算和矩阵变换功能，彻底改变传统的 TikZ 绘图模式。在传统 TikZ 绘图中，用户往往需要手动计算坐标点，或者依赖有限的 calc 库进行简单的线性插值。本库填补了高级数学计算与图形绘制之间的空白，允许用户在 LaTeX 代码中直接进行向量和矩阵等操作，能够直接在 LaTeX 环境中完成复杂的几何运算（如交点、极点、矩阵变换等），并提供简洁的绘图命令。

1.1　核心特性

- 矩阵计算：支持矩阵的算术运算、线性方程组、特征值和特征向量计算等。
- 向量计算：支持平面/空间向量的点积、叉积、投影、反射及旋转运算等。
- 三角形中心计算：支持三角形的内心、外心、重心、垂心及旁心的计算。
- 圆与直线相关计算：支持根轴、公切线、反演点以及圆与直线的交点计算。
- 圆锥曲线向关计算：引入齐次坐标，支持通过五点、五线或焦点-准线定义圆锥曲线。
- 绘图命令：提供 \segment 和 \conic 命令，自动处理直线和圆锥曲线的参数化绘制。

1.2　平面向量运算命令

表 1.1: 基于原点的向量运算

命令	描述
plane/add/origin={\A,\B,\C}	$\overline{OC}=\overline{OA}+\overline{OB}$
plane/sub/origin={\A,\B,\C}	$\overline{OC}=\overline{OA}-\overline{OB}$
plane/scale/origin={\A,\k,\B}	$\overline{OB}=k\overline{OA}$

命令	描述
`plane/dot/origin={\A,\B,\d}`	$d = \overline{OA} \cdot \overline{OB}$
`plane/length/origin={\A,\a}`	$a = \lvert\overline{OA}\rvert$
`plane/angle/origin={\A,\B,\a}`	$a = \angle AOB$, 有向角: $(-180°, 180°]$
`plane/normalize/origin={\A,\B}`	$OB = \overline{OA}/\lvert\overline{OA}\rvert$
`plane/rotate/origin={\A,\a,\B}`	vector rotation
`plane/project/origin={\A,\B,\C}`	vector projection
`plane/reject/origin={\A,\B,\C}`	vector rejection
`plane/reflect/origin={\A,\B,\C}`	vector reflection

表 1.2: 基于任意点 P 的向量运算

命令	描述
`plane/add ={\P,\A,\B,\C}`	$\overline{OC} = \overline{OP} + \overline{PA} + \overline{PB}$
`plane/sub={\P,\A,\B,\C}`	$\overline{OC} = \overline{OP} + \overline{PA} - \overline{PB}$
`plane/scale={\P,\A,\k,\B}`	$\overline{OB} = \overline{OP} + k\overline{PA}$
`plane/dot={\P,\A,\B,\d}`	$d = \overline{PA} \cdot \overline{PB}$
`plane/length={\P,\A,\a}`	$a = \lvert\overline{PA}\rvert$
`plane/length={\A,\B,\C,\a,\b,\c}`	计算三角形 ABC 的边长 a, b, c
`plane/angle={\P,\A,\B,\a}`	$a = \angle APB$, 有向角: $(-180°, 180°]$
`plane/normalize={\P,\A,\B}`	$OB = \overline{PA}/\lvert\overline{PA}\rvert$
`plane/rotate={\P,\A,\a,\B}`	vector rotation
`plane/project={\P,\A,\B,\C}`	vector projection
`plane/reject={\P,\A,\B,\C}`	vector rejection
`plane/reflect={\P,\A,\B,\C}`	vector reflection

1.3 空间向量运算命令

表 1.3: 基于原点的向量运算

命令	描述
`space/add/origin={\A,\B,\C}`	$\overline{OC} = \overline{OA} + \overline{OB}$
`space/sub/origin={\A,\B,\C}`	$\overline{OC} = \overline{OA} - \overline{OB}$
`space/scale/origin={\A,\k,\B}`	$\overline{OB} = k\overline{OA}$

命令	描述
`space/dot/origin={\A,\B,\d}`	$d = \overline{OA} \cdot \overline{OB}$
`space/cross/origin={\A,\B,\C}`	$\overline{OC} = \overline{OA} \times \overline{OB}$
`space/length/origin={\A,\a}`	$a = \lvert\overline{OA}\rvert$
`space/angle/origin={\A,\B,\a}`	$a = \angle AOB$, 无向角: $[0°, 180°]$
`space/normalize/origin={\A,\B}`	$OB = \overline{OA}/\lvert\overline{OA}\rvert$
`space/rotate/origin={\A,\U,\a,\B}`	vector rotation
`space/project/origin={\A,\B,\C}`	vector projection
`space/reject/origin={\A,\B,\C}`	vector rejection
`space/reflect/origin={\A,\B,\C}`	vector reflection

表 1.4: 基于任意点 P 的向量运算

命令	描述
`space/add ={\P,\A,\B,\C}`	$\overline{OC} = \overline{OP} + \overline{PA} + \overline{PB}$
`space/sub={\P,\A,\B,\C}`	$\overline{OC} = \overline{OP} + \overline{PA} - \overline{PB}$
`space/scale={\P,\A,\k,\B}`	$\overline{OB} = \overline{OP} + k\overline{PA}$
`space/dot={\P,\A,\B,\d}`	$d = \overline{PA} \cdot \overline{PB}$
`space/cross={\P,\A,\B,\C}`	$\overline{OC} = \overline{OP} + \overline{PA} \times \overline{PB}$
`space/length={\P,\A,\a}`	$a = \lvert\overline{PA}\rvert$
`space/angle={\P,\A,\B,\a}`	$a = \angle APB$, 无向角: $[0°, 180°]$
`space/normalize={\P,\A,\B}`	$OB = \overline{PA}/\lvert\overline{PA}\rvert$
`space/rotate={\P,\A,\U,\a,\B}`	vector rotation
`space/project={\P,\A,\B,\C}`	vector projection
`space/reject={\P,\A,\B,\C}`	vector rejection
`space/reflect={\P,\A,\B,\C}`	vector reflection

1.4 直线与圆的相关运算

表 1.5: 直线与圆的相关运算

命令	描述
`plane/anchor={\a,\b,\c,\P}`	计算原点在直线 $ax+by+c=0$ 上的投影 P
`plane/direct={\a,\b,\c,\v}`	计算直线 $ax+by+c=0$ 的方向向量 $\boldsymbol{v}$ (注意单位化)
`plane/intersect ll={\A,\B,\C,\D,\P}`	计算直线 AB 与 CD 的交点 P
`plane/intersect cl={\O,\r,\A,\B,\Pa,\Pb}`	计算圆（圆心为 O，半径为 r）与直线 AB 的交点 P_a 和 P_b，交点的顺序是从 A 至 B
`plane/intersect cn={\O,\r,\a,\b,\c,\Pa,\Pb}`	计算圆（圆心为 O，半径为 r）与直线 $ax+by+c=0$ 的交点 P_a 和 P_b，交点的顺序是直线 $ax+by+c=0$ 的正方向
`plane/radical axis={\Oa,\ra,\Ob,\rb,\H,\U}`	计算根轴的参数：根轴与连心线的交点 H 和根轴的单位方向向量 U
`plane/intersect cc={\Oa,\ra,\Ob,\rb,\Pa,\Pb}`	计算两圆的交点 P_a 和 P_b，其顺序是根轴的正方向
`plane/tangents={\O,\r,\P,\Ta,\Tb}`	计算过 P 作圆 O 的切线与圆的切点 T_a 和 T_b，其顺序是圆 O 与辅助圆的根轴的正方向
`plane/external center={\Oa,\ra,\Ob,\rb,\center}`	两圆的外位似中心
`plane/internal center={\Oa,\ra,\Ob,\rb,\center}`	两圆的内位似中心
`plane/inverse={\O,\r,\P,\Q}`	计算点 P 关于圆 O 的反演点 Q

1.5 三角形的相关运算

表 1.6: 三角形相关的运算

命令	描述
`plane/barycenter={\A,\B,\C,\x,\y,\z,\H}`	计算重心坐标 $(x:y:z)$ 对应的点 `\H`
`plane/incenter={\A,\B,\C,\I}`	三角形 ABC 的内心

命令	描述
plane/excenter={\A,\B,\C,\Ja}	顶点 A 的对面区域（即跨越边 BC 的一侧）的旁心，改变参数位置可以得其它旁心
plane/circumcenter = {\A,\B,\C,\O}	三角形 ABC 的外心
plane/orthocenter={\A,\B,\C,\H}	三角形 ABC 的垂心
plane/centroid ={\A,\B,\C,\G}	三角形 ABC 的重心

1.6 圆锥曲线相关的运算

表 1.7: 圆锥曲线相关的运算

命令	描述
conics/homogenize={\P,\Q}	将二维笛卡尔坐标齐次化，\P 为笛卡尔坐标，\Q 为齐次坐标
conics/dehomogenize={\P,\Q}	将齐次坐标去齐次化，\P 为齐次坐标，\Q 为笛卡尔坐标
conicss/distance={\P,\Q,\d}	求齐次坐标下的两点 P,Q 的距离
conics/cross={\U,\V,\W}	向量叉乘，\U,\V,\W 为点或直线的齐次坐标
conics/x-intercept={\l,\P}	直线 l 在 x 轴上的截距点（齐次坐标）
conics/y-intercept={\l,\P}	直线 l 在 y 轴上的截距点（齐次坐标）
conics/translation matrix={\xshift,\yshift,\T}	构造平移矩阵
conics/rotation matrix={\ang,\R}	构造旋转矩阵，\ang 是旋转角度（deg）
conics/define/quadratic={\coefs,\C}	由二次曲线方程系数计算系数矩阵
conics/define/focus-directrix={\F,\D,\e,\C}	由焦点-准线定义推导圆锥曲线的系数矩阵
conics/define/foci={\Fa,\Fb,\a,\C}	由焦点-准线定义推导圆锥曲线的系数矩阵
conics/define/ellipse/foci-point={\Fa,\Fb,\P,\C}	由焦点和曲线上一点确定一个椭圆

命令	描述
conics/define/hyperbola/foci-point={\Fa,\Fb,\P,\C}	由焦点和曲线上一点确定一个双曲线
conics/define/circle={\A,\B,\C,\Circle}	求过 A, B, C 三点的圆的系数矩阵
conics/define/circle={\O,\r,\Circle}	求以 O 为圆心，r 为半径的圆的系数矩阵
conics/define/five points={\Pa,\Pb,\Pc,\Pd,\Pe,\C}	由五点确定的圆锥曲线的系数矩阵
conics/define/five tangents={\La,\Lb,\Lc,\Ld,\Le,\C}	由五条切线确定的圆锥曲线的系数矩阵
conics/rotate={\CO,\CR,\R,\ang}	将含 xy 交叉项的二次曲线系数矩阵通过旋转变换消除交叉项
conics/translate={\CR,\CT,\T,\shift}	将已消除 xy 交叉项的二次曲线系数矩阵通过平移变换成标准形
conics/polar={\C,\P,\L}	计算极线的齐次坐标
conics/pole={\C,\L,\P}	计算极点的齐次坐标
conics/intersect cl={\C,\U,\V,\P,\Q}	采用参数化方法求圆锥曲线与直线的交点
conics/intersect cn={\C,\l,\P,\Q}	采用 Richter-Gebert 算法求圆锥曲线与直线的交点
conics/decompose={\C,\l,\m}	分解退化二次曲线为两条直线，$C = \boldsymbol{l} \cdot \boldsymbol{m}^T$
conics/intersect cc={\Ca,\Cb,\C,\l,\m}	采用 Richter-Gebert 算法求过两圆锥曲线的交点的退化二次曲线（两条直线）

1.7 \segment 和 \conic 绘图命令

Syntax

- `\segment[options,clip=(xmin:xmax,ymin:ymax)] (\l)`: 用于 `tikzpicture` 环境内绘制直线（线段）
  - `\l`: 直线的齐次坐标，列向量 |
- `\conic[opitons] (\C)`: 用于 `tikzpicture` 环境内绘制圆锥曲线（非退化），对于双曲线和抛物线，可以在 `options` 中设置 `domain`
  - `C`: 圆锥曲线的系数矩阵

在绘制双曲线或抛物线时，`doamin` 默认为 `domain=(-2:2)`，可以在调用时重新设置。

- 在绘制双曲线（$\frac{x^2}{a^2}-\frac{y^2}{b^2}=1$）时，`domain` 是下列双曲线参数方程中 t 的取值范围:

$$\begin{cases} x = \pm a\cosh t \\ y = b\sinh t \end{cases}$$

  t 的几何意义: 射线出原点交单位双曲线 $x^2-y^2=1$ 于 $(x=\cosh t, y=\sinh t)$,
  t 是射线, 双曲线和 x 轴围成的面积的二倍. 对于双曲线上位于 x 轴下方的点, 这个面积被认为是负值.
- 在绘制抛物线（$y^2=2px$）时，`domain` 是下列抛物线参数方程中 t 的取值范围:

$$\begin{cases} x = 2pt^2 \\ y = 2pt \end{cases}$$

  p 表示焦准距（焦点到准线的距离，$p>0$）。

第二章 tikzmath 和 pgfkeys 简介

在 tikzlibraryeuclidea.code.tex、tikzlibrarymc.code.tex 和 tikzlibraryvc.code.tex 使用了 TikZ 的 math 库和 pgfkeys 处理机制，实现计算过程的编程和可复用。以下是简要介绍。

2.1 使用 tikzmath

TikZ 是 LaTeX 中一个功能极其强大的绘图宏包，而 tikzmath 是 TikZ 自 3.0 版本起内置的一个专门用于执行数学计算的库，允许你在绘图过程中定义变量、执行算术运算、编写简单的循环和条件判断，从而实现参数化绘图和复杂的几何计算。

通过熟练掌握 tikzmath，你可以将 TikZ 从一个静态绘图工具转变为一个能够执行复杂逻辑和计算的动态绘图引擎。以下是 tikzmath 的详细使用教程。

要使用 tikzmath，首先需要在 LaTeX 文档的导言区加载 TikZ 宏包，并调用 math 库。

```
\usepackage{tikz}
\usetikzlibrary{math}
```

随后，你可以在 tikzpicture 环境内部或外部使用 tikzmath 环境：

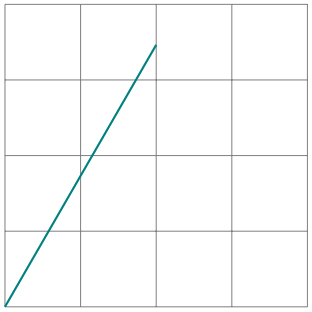
```
\tikzmath{
  % 在这里写代码
}
```

2.1.1 基本用法

计算 TikZ 坐标是 tikzmath 最常用的功能之一，如下所示：

```
\begin{tikzpicture}
```

```
  \tikzmath{
    \radius = 4;
    \x = \radius * cos(60);
    \y = \radius * sin(60);
  }

  \draw[help lines] (0,0) grid (4,4);
  \draw[thick, teal] (0,0) -- (\x, \y);
\end{tikzpicture}
```



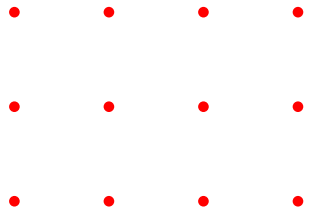
你也可以直接在 \coordinate 命令中嵌入计算，但 tikzmath 适合处理多步骤的复杂逻辑。

2.1.2 循环语句：for

tikzmath 提供了 for 循环功能，这对于绘制重复图形（如网格、多边形、分形）非常有用。

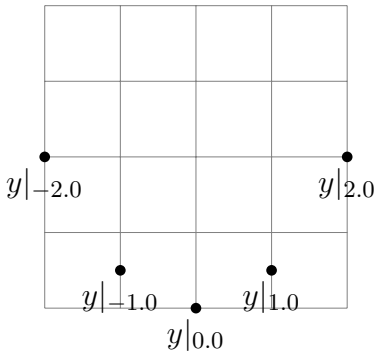
绘制一个简单的点阵：

```
\begin{tikzpicture}
  \tikzmath{
    for \i in {0,1,2,3} {
      for \j in {0,1,2} {
        % 计算 x 和 y 坐标
        \x = \i * 1.25;
        \y = \j * 1.25;
        % 注意：绘图命令需要用花括号包围
        { \fill[red] (\x, \y) circle (2pt); };
      };
    };
  }
\end{tikzpicture}
```



绘制函数上的点:

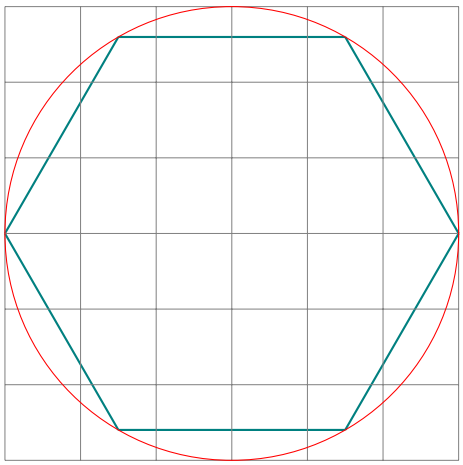
```
\begin{tikzpicture}
  \draw[help lines] (-2,0) grid (2,4);

  \tikzmath{
    \a = 0.5;
    for \x in {-2, -1, ..., 2}{
      \y = \a * (\x)^2;  % 注意: 不要写成 \y = \a * \x^2;
      {\fill (\x, \y) circle (2pt) node[below] {$y|_{\x}$};};
    };
  }
\end{tikzpicture}
```



综合运用变量定义、循环和三角函数计算，绘制正多边形:

```
\begin{tikzpicture}

  \tikzmath{
    \n = 6;
    \r = 3;
    % 计算并存储顶点坐标
    int \i;
    for \i in {1,...,\n} {
      \ang = (360/\n) * (\i - 1);
      \x{\i} = \r * cos(\ang);
      \y{\i} = \r * sin(\ang);
    };
    for \i in {1,...,\n}{
      % 计算下一个点的索引
      int \j;
```

```
      if \i == \n then { \j = 1; } else { \j = \i + 1; };
      {\draw[thick, teal] (\x{\i}, \y{\i}) -- (\x{\j}, \y{\j});};
    };
  }

  \draw[help lines] (-3,-3) grid (3,3);
  \draw[red] (0,0) circle(\r);

\end{tikzpicture}
```



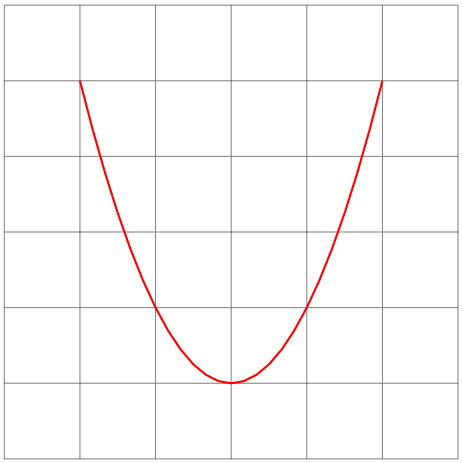
2.1.3 条件判断：if-then-else

tikzmath 支持基本的逻辑判断。

2.1.4 定义函数：function

你可以使用 function 关键字定义自己的数学函数。

```
\begin{tikzpicture}
  \tikzmath{
    function f(\x) { return \x*\x; };
  }
  \draw[help lines] (-3,-1) grid (3,5);
  \draw[thick, red] plot [domain=-2:2] (\x, {f(\x)});
\end{tikzpicture}
```



2.1.5 注意事项与技巧

使用 `tikzmath` 库与普通 TikZ 代码或 LaTeX 数学模式有显著不同。

以下是使用 `tikzmath` 时最需要注意的关键事项:

1. **语法格式严格：必须以分号 ; 结尾**

`tikzmath` 环境内的每一条语句（包括变量定义、函数定义、循环、条件判断等）**必须以分号 ;** 结束。

2. **`\tikzmath{}` 内部严禁出现空行**

这是最容易遇到的错误（报错信息通常为 `Paragraph ended before \tikz@math@parse@...`）。

- `tikzmath` 环境内部**不能包含空行**。空行会被 LaTeX 解释为段落结束，从而终止 `tikzmath` 的解析，导致后续代码出错。
- 即使是在注释之后，也不要留空行。

3. **变量作用域与类型**

- **默认实数**：`tikzmath` 中的变量默认是实数（floating point），即使你赋值为整数。
- 索引必须声明为 int，否则导致索引错误。
- **局部变量**：在 `tikzmath` 块中定义的变量通常只在该块内部有效（除非使用了全局定义技巧，但一般不建议）。
- **无需反斜杠定义变量**：在 `tikzmath` 块**内部**定义变量时，**不需要**加反斜杠 `\`。
  - 内部：`x = 5;` (正确)
  - 外部引用：`\x` (需要在 TikZ 坐标或节点中使用该变量时，必须加反斜杠，如 `(\x, 0)`)。

  - **特例**：如果在 `tikzmath` 内部引用已经存在的 TeX 宏或计数器，可能需要特殊处理，但通常定义新变量直接写名字即可。
- 变量名不要与其它包的命令冲突，如不要命名 `\angle`
- 运算时添加 `()`，防止负号运算错误，如当 `\x=-3;` 时，`\x^2` 被解析成 `-3^2`。

4. **循环与条件语句的括号**

`for` 循环和 `if` 语句的语法比较特殊，大括号 `{}` 的使用非常关键，并注意在末尾加上`;`。

- **语法限制**：`tikzmath` 的 `for` 语句解析时，范围部分（如 `{0,...,\n}`）必须是硬编码的整数或列表，变量（如 `\n`）不会被动态解析为范围边界。
- **设计目的**：`tikzmath` 旨在提供轻量级的数学计算支持，而非完整的编程控制流，因此它不支持动态范围。

5. **单位处理**

`tikzmath` 计算时默认不带单位（视为纯数字，通常对应 pt 或 cm，取决于上下文），但在赋值给坐标或长度时需要注意。

- 在 `tikzmath` 内部计算时，最好使用纯数字。
- 如果需要带单位的长度，通常在输出到 TikZ 命令时添加单位，或者在 `tikzmath` 中使用 `pt`, `cm` 等单位后缀（支持有限，建议计算纯数值，使用时加单位）。
  - 例如：`\x = 5;` 然后在绘图中使用 `(\x cm, 0)`。

6. **函数定义参数不能超过 9 个**

见 https://tikz.dev/math-algorithms

7. **与普通 TikZ 或 TEX 命令的混合**

`tikzmath` 块主要用于**计算和赋值**。它本身不直接绘制图形。你需要将计算结果传递给普通的 TikZ 命令。如前所述，在 `tikzmath` 代码块内部调用 `\draw`, `\node` 等命令时，必须用 `{ }` 包裹，以将其与数学计算代码区分开。使用 `\pgferror` 也是类似的。

8. **角度与弧度**

默认情况下，`sin`, `cos` 等函数使用**角度制**。如果你需要使用弧度，可以使用 `r` 后缀或转换函数。

9. **坐标数组**

上述多边形示例中使用了 `\x{\i}` 这种类似数组的语法来存储多个坐标，这是 TikZ 中处理循环坐标的常用技巧。`\a{\i,\j}` 可以模拟二维数组或矩阵。

10. 函数

- **嵌套调用**：`tikzmath` 函数完全支持嵌套调用，可以像 `f(g(x))` 这样使用。
- **嵌套定义**：不支持在一个函数体内定义另一个函数，但可以通过平级定义或临时变量实现类似逻辑。

2.1.6 错误排除

- **`Missing character ... ignored`** 或 **`Paragraph ended before ...`**：90% 是因为**空行**或**缺少分号**。
- **`Undefined control sequence`**：可能在 `tikzmath` 内部错误地使用了 `\` 开头定义变量（如 `\x = 1;` 应该是 `x = 1;`），或者在外部使用时忘了加 `\`。
- **浮点数精度问题**：`tikzmath` 使用 TeX 的定点数运算或 FPU（浮点单元），对于极高精度的科学计算可能不如 Python 精确，但在绘图范围内足够。如果涉及极大/极小值，可能需要加载 `fpu` 库并在 math 环境中启用。

2.2 `tikzmath` 的计算精度

`tikzmath` 库的计算精度主要受限于 TEX 本身的定点数运算机制。默认情况下，TEX 的计算精度有限，通常只能保证小数点后 4-5 位的准确性，并且数值范围也有限制。

要提高 `tikzmath` 的计算精度，主要有以下几种方法：

1. 使用 `fpu` 库（推荐）

`TikZ` 提供了一个 `fpu`（浮点单元）库，可以提供更高的精度和更大的数值范围。这是最常用的方法。

使用方法：

在 `tikzmath` 环境中或之前加载 `fpu` 库，并使用 `/pgf/fpu` 选项。

```
\documentclass{article}
\usepackage{tikz}
\usetikzlibrary{math, fpu}

\begin{document}

\begin{tikzpicture}
  % 启用 fpu 库
  \tikzmath{
```

```
    \x = 0.1;
    \y = 0.2;
    \z = \x + \y;
    % 默认情况下，fpu 会改变计算方式
  }
  \node at (0,0) {\z}; % 输出 0.3
\end{tikzpicture}

\end{document}
```

更精确的控制：

```
\documentclass{article}
\usepackage{tikz}
\usetikzlibrary{math, fpu}

\begin{document}

\begin{tikzpicture}
  % 使用 fpu 库进行高精度计算
  \pgfkeys{/pgf/fpu=true, /pgf/fpu/output format=fixed}
  \tikzmath{
    \a = 1.0 / 3.0;
    \b = \a * 3.0;
  }
  \node at (0,0) {\b}; % 输出 1.00000（高精度）

  % 恢复默认设置
  \pgfkeys{/pgf/fpu=false}
\end{tikzpicture}

\end{document}
```

2. 调整 `fpu` 的精度设置

`fpu` 库允许你设置计算的精度：

```
\pgfkeys{
  /pgf/fpu=true,
  /pgf/fpu/precision=15, % 设置精度为15位小数
  /pgf/fpu/output format=fixed
}
```

3. 使用 `xfp` 包（LaTeX3 浮点运算）

对于需要极高精度的计算，可以使用 `xfp` 包，它提供了 LaTeX3 的浮点运算引擎。

```
\documentclass{article}
```

```latex
\usepackage{tikz}
\usepackage{xfp}
\usetikzlibrary{math}

\begin{document}

\begin{tikzpicture}
  % 使用 xfp 进行高精度计算
  \tikzmath{
    \result = \fpeval{1.0/3.0 * 3.0};
  }
  \node at (0,0) {\result}; % 输出 1.0
\end{tikzpicture}

\end{document}
```

4. 注意事项

- **性能影响**：使用 `fpu` 或 `xfp` 会增加编译时间。
- **兼容性**：某些 `TikZ` 命令可能与 `fpu` 不兼容，需要小心使用。
- **输出格式**：`fpu` 默认输出科学计数法，使用 `/pgf/fpu/output format=fixed` 可以改为固定小数点格式。

> 在使用 LaTeX 的 `xfp` 宏包（或者直接用 `\fpeval`）进行数学运算时，三角函数默认确实是采用弧度制（Radians）。
>
> - `\fpeval`：默认使用弧度制（除非用 `sind`）。
> - `tikzmath`：默认使用角度制（Degrees）。

示例：高精度计算圆周率

```latex
\documentclass{article}
\usepackage{tikz}
\usetikzlibrary{math, fpu}

\begin{document}

\begin{tikzpicture}
  % 使用 fpu 计算圆周率
  \pgfkeys{/pgf/fpu=true, /pgf/fpu/output format=fixed, /pgf/fpu/
   precision=10}
  \tikzmath{
    \pi = 3.14159265358979;
    \radius = 2.0;
```

```
    \area = \pi * \radius * \radius;
  }
  \node at (0,0) {面积: \area}; % 输出高精度结果

  \pgfkeys{/pgf/fpu=false}
\end{tikzpicture}

\end{document}
```

总结：对于大多数需要提高精度的场景，使用 `fpu` 库是最简单有效的方法。如果需要极高的精度或复杂的数学运算，可以考虑使用 `xfp` 包。

> 浮点数比较的陷阱：TikZ 使用的是底层的 pgfmath 引擎（通常基于固定点算术或特定的浮点库）。残差比较不要低于 `\eps = 0.00001`。

在使用 TikZ 和 PGF 进行数学运算时，你遇到的“判断错误”很大程度上源于 PGFMath 的底层设计。PGFMath 默认并不是基于现代计算机通用的 IEEE 754 双精度浮点数，而是有着自己的一套逻辑。

以下是关于 **PGFMath 运算精度**的深度解析及避坑指南：

1. 精度限制：固定点算术 (Fixed Point) 在默认情况下，PGFMath 使用的是 **16 位固定点算术**。

- **数值范围**：它能处理的最大数值大约是 ± 16383.99999。
- **精度瓶颈**：它的最小有效位约为 0.000015 (1/65536)。
- **后果**：你的 `\eps = 0.000001`（六位小数）已经**超出了默认引擎的辨别能力**。在 PGFMath 眼里，比 0.00001 更小的数极易被舍入为 0，或者因为累积误差而变成一个随机的微小值。

1. 累积误差 (Cumulative Error) 当你进行矩阵运算（如坐标变换中的 $C_{1,1}$ 计算）时，每一步加减乘除都会产生舍入。

- 如果你的 $C_{1,1}$ 是通过多次乘法算出来的，它的理论值可能是 0，但 PGFMath 计算出的结果可能是 0.00024。
- 由于 $0.00024 > 0.000001$，`if` 判断会直接跳入 `else` 分支。

1. 解决方案

方案 A：调大容差（最简单）在绘图领域，通常不需要物理级别的精度。将 `\eps` 设为一个肉眼不可见但在 PGFMath 处理范围内的值：

```
\tikzmath{
    \eps = 0.001; % 或者是 0.01，取决于你的坐标缩放
}
```

方案 B：使用 `fpu` 库（推荐）如果你需要处理很大或很小的数，可以开启 PGF 的**浮点运算单元 (FPU)**。它将数值范围扩大到 $\pm 10^{300}$。

```
\usepackage{pgfplots}
\usepgflibrary{fpu}

\begin{tikzpicture}
    \pgfset{/pgf/fpu=true} % 开启 FPU
    \tikzmath{
        % 这里的运算将使用更高级的浮点逻辑
        \a = abs(\C{1,1});
        ...
    }
    \pgfset{/pgf/fpu=false} % 绘图前记得关闭，因为 \draw 不支持 FPU 格式
\end{tikzpicture}
```

方案 C：调用 LuaTeX（终极方案）如果你使用的是 `lualatex` 编译，你可以利用 Lua 的双精度浮点数处理逻辑，这比任何 PGF 内部库都要快且准。

```
\directlua{
    local eps = 1e-7
    if math.abs(\Coneone) < eps then
        -- 执行逻辑
    end
}
```

4. 调试小技巧为了看清到底发生了什么，可以在 `\tikzmath` 中利用 `\typeout` 打印出当前的真实数值：

```
\tikzmath{
    \a = abs(\C{1,1});
    print {\typeout{DEBUG: a is \a, eps is \eps}};
}
```

如果在控制台（Log 文件）里看到 `DEBUG: a is 0.00005, eps is 0.00000`，你就立刻明白为什么判断会失败了。

2.3 使用 pgfkeys

pgfkeys 是 LaTeX 中 pgf (Portable Graphics Format) 和 TikZ 宏包提供的一个极其强大的键值对（key-value）处理系统。它由 Till Tantau 开发，主要用于配置复杂的选项，是 TikZ 绘图功能的核心基础，但也可以独立用于任何需要灵活选项管理的 LaTeX 宏包或文档类。

> \tikzset{} 内部不要出现空行。

以下是对 pgfkeys 的详细介绍：

2.3.1 核心概念

pgfkeys 允许用户定义“键”（keys），并为这些键分配“值”（values）或特定的“代码”（code）。当用户设置一个键时，系统会执行与该键关联的操作。

它的语法结构通常如下：

```
\pgfkeys{
  /家族/子家族/键名 = 值,
  /家族/子家族/另一个键 = 另一段代码
}
```

2.3.2 主要功能特性

2.3.2.1 命名空间（Families/Path）

为了避免键名冲突，pgfkeys 使用类似文件系统的路径结构。

- 例如：/tikz/fill=red 和 /pgf/number format/fixed 属于不同的命名空间。
- 用户可以自定义自己的路径，如 /myapp/config/color。

2.3.2.2 处理器（Handlers）

这是 pgfkeys 最强大的地方。处理器是附加在键名后面的特殊标记（以 . 开头），用于改变键的行为。常见的处理器包括：

- **.store in**: 将值存储到一个 TeX 宏中。

```
\pgfkeys{/mymacro/name/.store in=\myname}
\pgfkeys{/mymacro/name=Bob}
% 现在 \myname 的内容是 "Bob"
```

- **.code**: 当键被设置时，直接执行一段 TeX 代码。

```
\pgfkeys{/action/print/.code={\typeout{Hello World!}}}
\pgfkeys{/action/print} % 输出 Hello World!
```

- **.initial**: 设置键的默认值。
- **.default**: 如果用户调用键时不提供值（即只写 key 而不写 =value），则使用此默认值。
- **.style**: 将一个键定义为一系列其他键的缩写（类似于 CSS 类）。

```
\pgfkeys{
  /tikz/myline/.style={draw=blue, thick, ->}
}
% 使用 /tikz/myline 等同于同时设置 draw=blue, thick, ->
```

- **.is choice**: 用于创建枚举选项。

```
\pgfkeys{
  /shape/type/.is choice,
  /shape/type/circle/.code={\def\shapetype{circle}},
  /shape/type/rect/.code={\def\shapetype{rectangle}},
  /shape/type=circle % 执行 circle 对应的代码
}
```

- **.forward to**: 将设置转发给另一个键（常用于兼容性或简化接口）。

2.3.2.3 值的解析

pgfkeys 能够智能地处理逗号分隔的列表。如果在值内部需要包含逗号，必须用花括号 {} 将值包裹起来。

```
\pgfkeys{key={value, with, commas}}
```

2.3.3 基本用法示例

下面是一个完整的、独立的 LaTeX 示例，展示了如何定义和使用 pgfkeys：

```latex
\documentclass{article}
\usepackage{pgfkeys}

% 1. 定义键及其行为
\pgfkeys{
  /student/.cd, % 进入 /student 命名空间
  name/.store in=\stuName,      % 将名字存入 \stuName
  age/.store in=\stuAge,        % 将年龄存入 \stuAge
  grade/.initial=A,             % 设置默认等级为 A
  info/.code={                  % 定义一个动作键
    学生姓名: \stuName, 年龄: \stuAge, 等级: \pgfkeysvalueof{/student/grade}
  },
  % 定义一个样式，一次性设置多个键
  set default student/.style={
    name=Unknown,
    age=0,
    grade=C
  }
}

% 方便调用的命令
\newcommand{\setStudent}[1]{\pgfkeys{/student/#1}}
\newcommand{\printStudent}{\pgfkeys{/student/info}}

\begin{document}

% 使用默认样式
\setStudent{set default student}
\printStudent

\vspace{1cm}

% 自定义设置
\setStudent{
  name=Alice,
  age=20,
  grade=A+
}
\printStudent

\vspace{1cm}

% 只修改部分属性（grade 保持上次的 A+，或者如果想重置需显式声明）
```

```
\setStudent{name=Bob, age=22}
\printStudent

\end{document}
```

第三章　矩阵计算 Matrix Computations

`tikzlibrarymc.code.tex` 是用于在 LaTeX 中通过 `tikzmath` 引擎实现矩阵数值计算和线性代数运算。该库利用 TikZ 的 `tikzmath` 引擎和 `xfp` 高精度计算包，在 LaTeX 内部实现了完整的线性代数运算体系，可以帮助你在绘图时动态计算节点坐标，或在文档中直接展示数学运算过程。

3.1　核心注意事项 (Caveats)

在使用该库之前，必须了解 `tikzmath` 的局限性，否则容易导致编译错误或逻辑错误：

1. **变量作用域**：`tikzmath` 中的变量默认是**全局**的。为了避免冲突，库内部使用了 `mc@` 前缀。
2. **语法限制**：
   - 代码块内不能有空行。
   - 语句必须以分号 `;` 结尾。
   - `for` 循环没有 `break` 或 `continue`，且第一次迭代总是无条件执行。
   - 函数参数不能直接使用数组，且最大参数个数为 9。
3. **数据类型**：整数下标必须显式定义为 `int`，否则会被视为浮点数导致宏无法找到。
4. **调试困难**：该库极难调试，建议在调用前确保参数格式正确。

3.2　主要命令

该库通过 `\tikzset` 定义了一系列 `.code args` 接口。以下是主要功能的分类说明：

3.2.1 矩阵基础运算

用于处理矩阵的生成、变形和四则运算。

表 3.1: 矩阵基础运算

命令 (Key)	参数格式	功能描述
mat/show	{\A,\m,\n}	**打印显示**：将矩阵 A 输出到文档中，m 为行数，n 为列数。
mat/stdout	{\A,\m,\n}	**控制台输出**：将矩阵 A 打印到编译日志 (Log) 中。
mat/assign	{\A,\m,\n,\B}	**赋值**：将源矩阵复制到目标矩阵 $B = A$。
mat/identity	{\A,\n}	**单位阵**：生成 $n \times n$ 的单位矩阵 A。
mat/transpose	{\A,\m,\n,\B}	**转置**：计算矩阵 A 的转置并存入 B，$B = A^T$。
mat/scale	{\A,\m,\n,\k,\B}	**数乘**：计算 $B = k \cdot A$。
mat/add	{\A,\B,\m,\n,\C}	**加法**：计算 $C = A + B$。
mat/sub	{\A,\B,\m,\n,\C}	**减法**：计算 $C = A - B$。
mat/combine	{\A,\B,\m,\n,\a,\b,\C}	**线性组合**：计算 $C = a \cdot A + b \cdot B$。

3.2.2 高级矩阵运算

用于处理矩阵乘法、行列式、逆矩阵及秩相关计算。

表 3.2: 高级矩阵运算

命令 (Key)	参数格式	功能描述
mat/product	{\A,\B,\m,\n,\k,\C}	**乘法**：计算 $C = A \cdot B$。A 为 $m \times n$ 矩阵，B 为 $n \times k$ 矩阵。
mat/det	{\A,\n,\d}	**行列式**：计算方阵 A 的行列式，结果存入 d，$d = \det(A)$。
mat/minors	{\A,\n,\M}	**余子式**：计算 A 的所有余子式矩阵 M。
mat/cofactors	{\A,\n,\C}	**代数余子式**：计算 A 的代数余子式矩阵 C。
mat/inverse	{\A,\n,\B}	**逆矩阵**：利用高斯-约旦消元法计算 A 的逆矩阵 B，$B = A^{-1}$。

命令 (Key)	参数格式	功能描述
mat/solve	{\A,\B,\m,\n,\X}	**解方程**：求解线性方程组 $AX = B$，解存入 X，A 为 $m \times m$ 矩阵，B 为 $m \times n$ 矩阵。

3.2.3 特征值与特征向量

提供了两种不同的算法来处理特征值问题。

- **Jacobi 方法 (mat/jacobi)**：
  - **适用**：实对称矩阵。
  - **参数**：{A, n, D, V}。
  - **功能**：计算特征值和特征向量。D 为对角化的特征值矩阵，V 为特征向量矩阵。
  - **注意**：内部使用全局变量 \status 标记是否成功（1 为成功，0 为迭代超限）。
- **幂方法 (mat/power method)**：
  - **适用**：求解主特征值（模最大的特征值）及其对应的特征向量。
  - **参数**：{A, n, lambda, v}。
  - **功能**：结果存入 lambda (特征值) 和 v (特征向量)。

3.2.4 辅助与工具函数

- **mat/maxabs**：求矩阵元素绝对值的最大值。
- **mat/normalize**：根据最大绝对值对矩阵进行归一化。
- **mat/pivot**：在矩阵中寻找主元（绝对值最大的元素）及其位置。
- **mat/row / mat/column**：提取矩阵的指定行或列作为列向量。
- **mat/congruence**：计算合同变换 $P^T AP$。
- **mat/bilinear**：计算双线性形式 $x^T Ay$。
- **mat/decompose rank-1**：将秩 1 矩阵 $A_{m\times n}$ 分解成两个列向量相乘：$A = \max A_{i,j} \cdot \boldsymbol{u} \cdot \boldsymbol{v}^T$

3.3 使用示例

要在 LaTeX 文档中使用此库，请确保已加载相关依赖（xfp, tikz, pgfplots），然后调用 `\tikzset`。

示例 1：矩阵乘法

```
% 定义两个 2x2 矩阵 A 和 B
\tikzmath{
    \A{1,1} = 2; \A{1,2} = 1;
    \A{2,1} = 1; \A{2,2} = 3;
    \B{1,1} = 2; \B{1,2} = 1;
    \B{2,1} = 1; \B{2,2} = 3;
}
\tikzset{
  mat/product={\A,\B,2,2,2,\C} % C = A * B
}
% 显示结果
\tikzset{mat/show={\C,2,2}}
```

示例 2：求解线性方程组 $Ax = b$

```
% 假设 \A 是系数矩阵 (2x2), \b 是常数向量 (2x1)
\tikzset{
  mat/solve={\A,\b,2,1,\x} % 求解 x
}
% 检查状态变量 \status (需在 tikzmath 外部或通过其他方式捕获)
% 显示解向量
\tikzset{mat/show={\x,2,1}}
```

示例 3：计算特征值 (Jacobi)

```
% 假设 \SymMat 是一个 3x3 的对称矩阵
\tikzset{
  mat/jacobi={\SymMat,3,\EigenValues,\EigenVectors}
}
% \status 变量将指示计算是否收敛
```

3.4 开发建议与注意事项

- 全局变量风险：`tikzmath` 内部定义的变量是全局的。为避免冲突，本库内部使用了 `\mc@` 前缀。用户自定义变量时请尽量避开。

- 循环特性：`tikzmath` 的 `for` 循环第一次迭代总是执行的，即使范围不符合。库内已针对此特性做了 `if` 限制。
- 调试建议：`tikzmath` 报错信息通常模糊。如遇到问题，建议先使用 `mat/stdout` 将中间过程输出到终端（Console）查看数值。
- 精度控制：可以通过修改 `\EPSILON` 和 `\JacobiEpsilon` 来调整计算停止的阈值（默认为 0.00005）。
- 语法细节：在 `.code args` 中传递算式作为参数时，务必加括号，例如 `mat/scale={\A, 2, 2, (1+1), \B}`。
- 分号陷阱：在 `\tikzmath` 内部，每一行指令（包括变量赋值）必须以分号 `;` 结尾，且块内严禁出现空行。
- 索引类型：当使用 `\A{\i,\j}` 访问元素时，如果 `\i` 是通过计算得到的，务必将其转换为整数（如 `int(\i)`），否则库可能因无法匹配浮点数下标而崩溃。
- 内存限制：尽管 `xfp` 提高了精度，但 LaTeX 的内存池（TeXMemory Stack）依然有限。对于 10×10 以上的矩阵，编译速度会显著变慢。
- 循环变量泄露：库中使用的 `\mc@i` 等变量在 `\tikzmath`中是全局的。如果你在外部也定义了同名变量，可能会被库函数篡改。建议始终使用 `\temp` 或自定义前缀。
- 关于 `.code args`：在使用 `mat/scale` 等命令时，如果缩放因子是一个复杂的数学表达式（如 `2+sin(30)`），必须用括号包裹，否则 `pgfkeys` 可能会解析参数错误。

3.5 总结

该库是一个功能强大的 LaTeX 线性代数工具箱，虽然受限于 TikZ 的编程模型（如全局变量、无 break 语句等），但它成功封装了高斯消元、Jacobi 迭代等复杂算法。用户在使用时应特别注意参数传递的格式（尤其是行列数的指定）以及变量名的冲突风险。

第四章　向量运算 Vector Computations

`tikzlibraryvc.code.tex` 是 LaTeX 中 TikZ 绘图包的一个扩展库，定义了一系列基于向量代数的坐标运算命令和语法糖，使得用户能够直接在 `\tikzmath` 中执行向量加减、数乘、点积、投影、旋转以及线性组合等复杂操作，从而无需手动计算中间坐标值即可精确构建几何图形，极大地简化了涉及几何作图流程。

4.1　位置向量

4.1.1　向量与位置向量

在数学和物理学中，**向量（Vector）**与**位置向量（Position Vector）**的关系可以类比为 “位移” 与 “坐标” 的关系。简单来说，位置向量是向量的一个特殊子集。

以下是它们的主要区别与联系：

向量（自由向量） 是指具有**大小（模）**和**方向**的量。在经典几何中，向量通常被视为 “自由” 的，这意味着只要大小和方向不变，向量可以在空间中任意平移。

位置向量（限定向量） 当一个向量的**起点被固定在坐标原点** O 时，这个向量就称为该点的 “位置向量”。它唯一地确定了空间中某一点 P 的位置，点 P 与它的位置向量 $\overline{OP}$ 是一一对应的。

表 4.1: 关键差异对比

特性	向量 (Vector)	位置向量 (Position Vector)
起点	任意（可在空间平移）	**固定**（必须从原点 O 出发）
终点	任意	指向空间中的特定点 P
描述对象	描述运动、力、位移等量	描述空间中某个位置的坐标

特性	向量 (Vector)	位置向量 (Position Vector)
自由度	自由向量	绑定在特定点上的向量

在几何作图中，你的目标是找由点 P 移动向量 $\overline{PA}$ 和 $\overline{PB}$ 叠加后到达的位置 C。此时，计算两向量之和 $\overline{PC}$：

$$\overline{PC} = \overline{PA} + \overline{PB}$$

而点 C 的位置向量（Position Vector，起点固定为原点）OC 为：

$$\overline{OC} = \overline{OP} + \overline{PA} + \overline{PB}$$

在 `tikizlibraryvc.code.tex` 中，基于原点出发的向量运算，命令添加 `origin`，形如：`plane/add/origin`，`space/add/origin`，等等；基于任意点出发的向量运算的命令形如：`plane/add`，`space/add`，等等。

4.1.2 向量在 cartes 包中的表示

在 `cartes` 包中，使用带序号的宏来表示向量，如平面向量表示为：

```
\tikzmath{
  \P{1} = 1.5; \P{2} = 5.0;
  \Q{1} = 0.5; \Q{2} = 3.0;
}
```

空间向量表示为：

```
\tikzmath{
  \P{1} = 1.5; \P{2} = 5.0; \P{3} = -5.0;
  \Q{1} = 0.5; \Q{2} = 3.0; \Q{3} = 2.0;
}
```

`cartes` 中的向量可以视作**数组**，不能直接与矩阵（见矩阵计算 Matrix Computations[p. 25]）进行运算。

4.2 向量加法与减法（Vector Addition and Subtraction）

4.2.1 向量的加法与减法

两个同维度的向量相加，结果是一个新的向量，其每个分量是原向量对应分量的和。

几何意义：遵循平行四边形法则或三角形法则。将第二个向量的起点移到第一个向量的终点，从第一个向量的起点指向第二个向量终点的向量即为和向量。

向量 $\boldsymbol{a}$ 减去 $\boldsymbol{b}$ 等于 $\boldsymbol{a}$ 加上 $\boldsymbol{b}$ 的负向量（$-\boldsymbol{b}$）。

几何意义：从向量 $\boldsymbol{b}$ 的终点指向向量 $\boldsymbol{a}$ 的终点的向量（当两向量起点重合时）。

4.2.2 语法形式

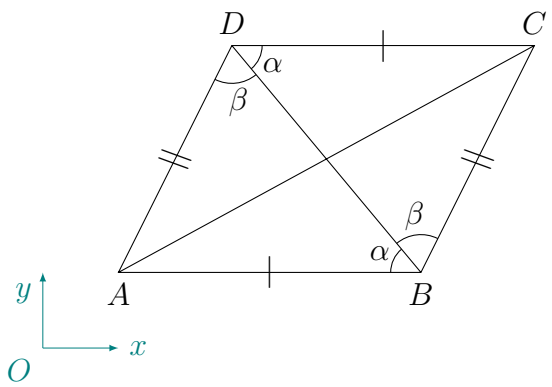
Syntax

- `plane/add/origin={\A,\B,\C}`: 平面向量加法，$\overline{OC} = \overline{OA} + \overline{OB}$
- `plane/add={\P,\A,\B,\C}`: 平面向量加法，$\overline{OC} = \overline{OP} + \overline{PA} + \overline{PB}$
- `space/add/origin={\A,\B,\C}`: 空间向量加法，$\overline{OC} = \overline{OA} + \overline{OB}$
- `space/add={\P,\A,\B,\C}`: 空间向量加法，$\overline{OC} = \overline{OP} + \overline{PA} + \overline{PB}$
- `plane/sub/origin={\A,\B,\C}`: 平面向量减法，$\overline{OC} = \overline{OA} - \overline{OB}$
- `plane/sub={\P,\A,\B,\C}`: 平面向量减法，$\overline{OC} = \overline{OP} + \overline{PA} - \overline{PB}$
- `space/sub/origin={\A,\B,\C}`: 空间向量减法，$\overline{OC} = \overline{OA} - \overline{OB}$
- `space/sub={\P,\A,\B,\C}`: 空间向量减法，$\overline{OC} = \overline{OP} + \overline{PA} - \overline{PB}$

4.2.3 示例 1：平面平行四边形

```
\begin{tikzpicture}
  % tikzmath 和 tikzset 也可置于 tikzpicture 外部
  \tikzmath{% 定义点 A, B, C 的坐标
    \A{1} = 1; \A{2} = 1;
    \B{1} = 5; \B{2} = 1;
    \C{1} = 6.5; \C{2} = 4;
  }

  \tikzset{% 调用向量运算库命令
    plane/add={\B,\A,\C,\D},
```

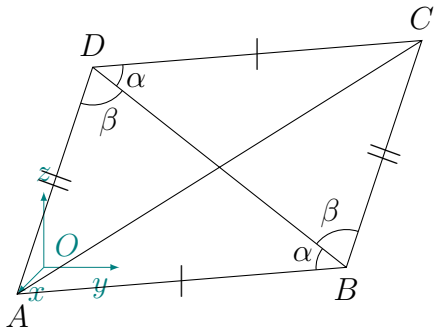
```
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0) -- (1,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0) -- (0,1) node[below left]{$y
  $};
  \draw[teal] (0,0) node[below left]{$O$};

  % 定义 tikz 坐标
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (D) at (\D{1},\D{2});

  % 绘制平行四边形
  \draw (A) node[below]{$A$} -- (B) node[below]{$B$}
    -- (C) node[above]{$C$} -- (D) node[above]{$D$} -- cycle
    (A) -- (C)  (B) -- (D);

  % 标准边
  \draw
    ($(A)!.5!(B)$) node {$|$}
    ($(C)!.5!(D)$) node {$|$}
    ($(A)!.5!(D)$) node[rotate=70] {$||$}
    ($(B)!.5!(C)$) node[rotate=70] {$||$};

  % 标注角
  \pic [draw,pic text=$\alpha$,angle radius=4mm,
      angle eccentricity=1.5] {angle=D--B--A};
  \pic [draw,pic text=$\alpha$,angle radius=4mm,
      angle eccentricity=1.5] {angle = B--D--C};
  \pic [draw,pic text=$\beta$,angle radius=5mm,
      angle eccentricity=1.5] {angle = A--D--B};
  \pic [draw,pic text=$\beta$,angle radius=5mm,
      angle eccentricity=1.5] {angle = C--B--D};
\end{tikzpicture}
```



4.2.4 示例 2：空间平行四边形

```latex
\begin{tikzpicture}[
  x={({0.5*cos(-135)},{0.5*sin(-135)})}, % default: cm
  y={(1cm,0cm)},
  z={(0cm,1cm)}
]
  % tikzmath 和 tikzset 也可置于 tikzpicture 外部
  \tikzmath{% 定义点 A, B, C 的坐标
    \A{1} = 1; \A{2} = 0; \A{3} = 0;
    \B{1} = 0; \B{2} = 4; \B{3} = 0;
    \C{1} = 0; \C{2} = 5; \C{3} = 3;
  }

  \tikzset{% 调用向量运算库命令
    space/sub={\A,\C,\B,\D},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0,0) -- (1,0,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,1,0) node[below left]{$y$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,0,1) node[above]{$z$};
  \draw[teal] (0,0) node[above right]{$O$};

  % 定义 tikz 坐标
  \coordinate (A) at (\A{1},\A{2},\A{3});
  \coordinate (B) at (\B{1},\B{2},\B{3});
  \coordinate (C) at (\C{1},\C{2},\C{3});
  \coordinate (D) at (\D{1},\D{2},\D{3});

  % 绘制平行四边形
  \draw (A) node[below]{$A$} -- (B) node[below]{$B$}
    -- (C) node[above]{$C$} -- (D) node[above]{$D$} -- cycle
    (A) -- (C)  (B) -- (D);

  % 标准边
  \draw
    ($(A)!.5!(B)$) node {$|$}
    ($(C)!.5!(D)$) node {$|$}
    ($(A)!.5!(D)$) node[rotate=70] {$||$}
    ($(B)!.5!(C)$) node[rotate=70] {$||$};

  % 标注角
  \pic [draw,pic text=$\alpha$,angle radius=4mm,
      angle eccentricity=1.5] {angle=D--B--A};
  \pic [draw,pic text=$\alpha$,angle radius=4mm,
      angle eccentricity=1.5] {angle = B--D--C};
```

```
  \pic [draw,pic text=$\beta$,angle radius=5mm,
      angle eccentricity=1.5] {angle = A--D--B};
  \pic [draw,pic text=$\beta$,angle radius=5mm,
      angle eccentricity=1.5] {angle = C--B--D};
\end{tikzpicture}
```



4.3 数乘/标量乘法（Scalar Multiplication）

4.3.1 向量的数乘

一个标量（实数 k）与一个向量相乘，结果是一个新向量，其方向与原向量相同（$k > 0$）或相反（$k < 0$），长度变为原长度的 $|k|$ 倍。

4.3.2 语法形式

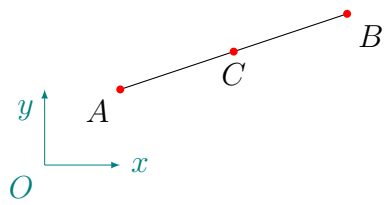
Syntax

- `plane/scale/origin={\A,\k,\B}`: 平面向量数乘，$\overline{OB} = k\overline{OA}$
- `plane/scale={\P,\A,\k,\B}`: 平面向量数乘，$\overline{OB} = \overline{OP} + k\overline{PA}$
- `space/scale/origin={\A,\k,\B}`: 空间向量数乘，$\overline{OB} = k\overline{OA}$
- `space/scale={\P,\A,\k,\B}`: 空间向量数乘，$\overline{OB} = \overline{OP} + k\overline{PA}$

4.3.3 示例 1：平面线段中点

```
\begin{tikzpicture}
  % tikzmath 和 tikzset 也可置于 tikzpicture 外部
  \tikzmath{% 定义点 A, B 的坐标
    \A{1} = 1; \A{2} = 1;
```

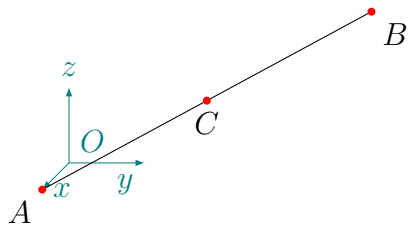
```
    \B{1} = 4; \B{2} = 2;
  }

  \tikzset{% 调用向量运算库命令
    plane/scale={\A,\B,1/2,\C},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0) -- (1,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0) -- (0,1) node[below left]{$y
  $};
  \draw[teal] (0,0) node[below left]{$O$};

  % 定义 tikz 坐标
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});

  % 绘制线段
  \draw (A) -- (B);

  % 标注点
  \foreach \p/\placement in {A/below left, B/below right, C/below}{
    \fill[red] (\p) circle (1.5pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



4.3.4 示例 2：空间线段中点

```
\begin{tikzpicture}[
  x={({0.5*cos(-135)},{0.5*sin(-135)})}, % default: cm
  y={(1cm,0cm)},
  z={(0cm,1cm)}
]
  % tikzmath 和 tikzset 也可置于 tikzpicture 外部
  \tikzmath{% 定义点 A, B 的坐标
    \A{1} = 1; \A{2} = 0; \A{3} = 0;
    \B{1} = 0; \B{2} = 4; \B{3} = 2;
  }
```

```latex

  \tikzset{% 调用向量运算库命令
    space/scale={\A,\B,1/2,\C},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0,0) -- (1,0,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,1,0) node[below left
   ]{$y$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,0,1) node[above]{$z$};
  \draw[teal] (0,0) node[above right]{$O$};

  % 定义 tikz 坐标
  \coordinate (A) at (\A{1},\A{2},\A{3});
  \coordinate (B) at (\B{1},\B{2},\B{3});
  \coordinate (C) at (\C{1},\C{2},\C{3});

  % 绘制线段
  \draw (A) -- (B);

  % 标注点
  \foreach \p/\placement in {A/below left, B/below right, C/below}{
    \fill[red] (\p) circle (1.5pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



4.4 向量的线性组合（Linear Combination）

4.4.1 向量的线性组合

简单来说，给定一组向量，将它们分别乘以各自的标量（常数）后再相加，所得到的结果就是一个线性组合。

4.4.2 语法形式

Syntax

- `plane/combine/origin={\A,\B,\a,\b,\C}`: 平面向量线性组合，$\overline{OC} = a \cdot \overline{OA} + b \cdot \overline{OB}$
- `plane/combine={\P,\A,\B,\a,\b,\C}`: 平面向量线性组合，$\overline{OC} = \overline{OP} + a \cdot \overline{PA} + b \cdot \overline{PB}$
- `space/combine/origin={\A,\B,\a,\b,\C}`: 空间向量线性组合，$overlineOC = a \cdot \overline{OA} + b \cdot \overline{OB}$
- `space/combine={\P,\A,\B,\a,\b,\C}`: 空间向量线性组合，$\overline{OC} = \overline{OP} + a \cdot \overline{PA} + b \cdot \overline{PB}$

4.5 点积/数量积（Dot Product/Scalar Product）

4.5.1 向量的点积

向量的点积一个**标量**（数值），而不是向量。两个同维度向量的点积结果是，对应分量相乘后求和。

$$\boldsymbol{a} \cdot \boldsymbol{b} = \sum_{i=1}^{n} a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

几何公式：

$$\boldsymbol{a} \cdot \boldsymbol{b} = |\boldsymbol{a}||\boldsymbol{b}| \cos\theta$$

其中 θ 是两向量之间的夹角，$|\boldsymbol{a}|$ 表示向量的模（长度）。

重要性质与应用：

- 若 $\boldsymbol{a} \cdot \boldsymbol{b} = 0$ 且两向量非零，则它们**垂直**（正交）。
- 可用于计算向量在另一个向量上的**投影**。

4.5.2 语法形式

Syntax

- `plane/dot/origin={\A,\B,\d}`: 平面向量点乘，$d = \overline{OA} \cdot \overline{OB}$
- `plane/dot={\P,\A,\B,\d}`: 平面向量点乘，$d = \overline{PA} \cdot \overline{PB}$
- `space/dot/origin={\A,\B,\d}`: 空间向量点乘，$d = \overline{OA} \cdot \overline{OB}$
- `space/dot={\P,\A,\B,\d}`: 空间向量点乘，$d = \overline{PA} \cdot \overline{PB}$

4.6 叉积/向量积（Cross Product）

4.6.1 向量叉乘

向量叉乘（Vector Cross Product），也称为向量积或外积，是三维空间中两个向量之间的一种运算。与点乘（内积）产生标量不同，叉乘的结果是一个**向量**。

几何定义

若有向量 $\boldsymbol{a}$ 和 $\boldsymbol{b}$，它们的叉乘表示为 $\boldsymbol{a} \times \boldsymbol{b}$。结果向量 $\boldsymbol{c}$ 具有以下特征：

- **方向**：$\boldsymbol{c}$ 同时垂直于 $\boldsymbol{a}$ 和 $\boldsymbol{b}$。具体方向遵循**右手定则**：四指从 $\boldsymbol{a}$ 转向 $\boldsymbol{b}$，大拇指所指的方向即为 $\boldsymbol{a} \times \boldsymbol{b}$ 的方向。
- **模长（大小）**：其模长等于以这两个向量为邻边构成的**平行四边形的面积**。

$$|\boldsymbol{a} \times \boldsymbol{b}| = |\boldsymbol{a}||\boldsymbol{b}| \sin\theta$$

（其中 θ 是 $\boldsymbol{a}$ 与 $\boldsymbol{b}$ 之间的夹角，$0 \leq \theta \leq \pi$）

代数坐标运算

在三维笛卡尔坐标系中，设 $\boldsymbol{a} = (a_1, a_2, a_3)$，$\boldsymbol{b} = (b_1, b_2, b_3)$，则叉乘可以通过行列式形式方便地计算：

$$\boldsymbol{a} \times \boldsymbol{b} = \begin{vmatrix} \boldsymbol{i} & \boldsymbol{j} & \boldsymbol{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}$$

展开后的坐标为：

$$\boldsymbol{a} \times \boldsymbol{b} = (a_2b_3 - a_3b_2, a_3b_1 - a_1b_3, a_1b_2 - a_2b_1)$$

重要性质

- **反交换律**：$\boldsymbol{a} \times \boldsymbol{b} = -(\boldsymbol{b} \times \boldsymbol{a})$（方向相反）。
- **分配律**：$\boldsymbol{a} \times (\boldsymbol{b} + \boldsymbol{c}) = \boldsymbol{a} \times \boldsymbol{b} + \boldsymbol{a} \times \boldsymbol{c}$。
- **与标量结合**：$k(\boldsymbol{a} \times \boldsymbol{b}) = (k\boldsymbol{a}) \times \boldsymbol{b} = \boldsymbol{a} \times (k\boldsymbol{b})$。
- **平行判定**：若 $\boldsymbol{a} \times \boldsymbol{b} = \boldsymbol{0}$，且 $\boldsymbol{a}, \boldsymbol{b}$ 为非零向量，则说明 $\boldsymbol{a}$ 与 $\boldsymbol{b}$ 平行（夹角为 0 或 $180°$）。

4.6.2 语法形式

Syntax

- `space/cross/origin={\A,\B,\C}`: 空间向量叉乘，$\overline{OC} = \overline{OA} \times \overline{OB}$
- `space/cross={\P,\A,\B,\C}`: 空间向量叉乘，$\overline{OC} = \overline{OP} + \overline{PA} \times \overline{PB}$

4.7 向量的模（Magnitude/Length）

4.7.1 向量的模

向量的长度。

公式（欧几里得范数 L_2）：

$$|\boldsymbol{a}| = \sqrt{a_1^2 + a_2^2 + \cdots + a_n^2} = \sqrt{\boldsymbol{a} \cdot \boldsymbol{a}}$$

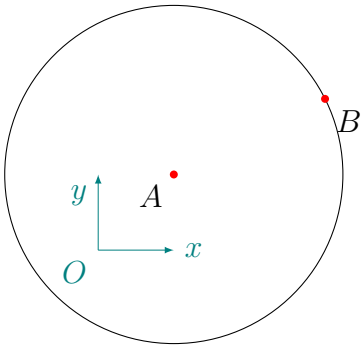
4.7.2 语法形式

Syntax

- `plane/length/origin={\A,\a}`: 平面向量的长度，$a = |\overline{OA}|$
- `plane/length={\P,\A,\a}`: 平面向量的长度，$a = |\overline{PA}|$
- `plane/length={\A,\B,\C,\a,\b,\c}`: 计算三角形 ABC 的边长 a, b, c
- `space/length/origin={\A,\a}`: 空间向量的长度，$a = |\overline{OA}|$
- `space/length={\P,\A,\a}`: 空间向量的长度，$a = |\overline{PA}|$

4.7.3 示例 1：已知圆心和圆上一点作圆

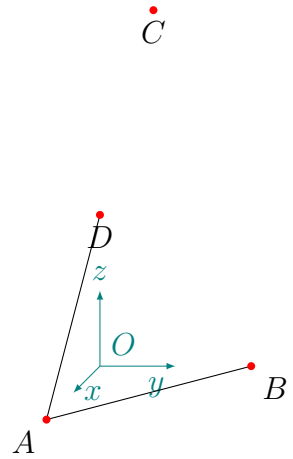
```latex
\begin{tikzpicture}
  % tikzmath 和 tikzset 也可置于 tikzpicture 外部
  \tikzmath{% 定义点 A, B 的坐标
    \A{1} = 1; \A{2} = 1;
    \B{1} = 3; \B{2} = 2;
  }

  \tikzset{% 调用向量运算库命令
    plane/length={\A,\B,\radius},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0) -- (1,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0) -- (0,1) node[below left]{$y
  $};
  \draw[teal] (0,0) node[below left]{$O$};

  % 定义 tikz 坐标
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});

  % 绘制圆
  \draw (A) circle (\radius);

  % 标注点
  \foreach \p/\placement in {A/below left, B/below right}{
    \fill[red] (\p) circle (1.5pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



4.7.4 示例 2：在线段上截取指定长度

```
\begin{tikzpicture}[
  x={({0.5*cos(-135)},{0.5*sin(-135)})}, % default: cm
  y={(1cm,0cm)},
  z={(0cm,1cm)}
]
  % tikzmath 和 tikzset 也可置于 tikzpicture 外部
  \tikzmath{% 定义点 A, B, C 的坐标
    \A{1} = 2; \A{2} = 0; \A{3} = 0;
    \B{1} = 0; \B{2} = 2; \B{3} = 0;
    \C{1} = -2; \C{2} = 0; \C{3} = 4;
  }

  \tikzset{% 在 AC 上截取 AD, 使得 AD 与 AB 的长度相等
    space/length={\A,\B,\lena},
    space/length={\A,\C,\lenb},
    space/scale={\A,\C,(\lena/\lenb),\D},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0,0) -- (1,0,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,1,0) node[below left
   ]{$y$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,0,1) node[above]{$z$};
  \draw[teal] (0,0) node[above right]{$O$};

  % 定义 tikz 坐标
  \coordinate (A) at (\A{1},\A{2},\A{3});
  \coordinate (B) at (\B{1},\B{2},\B{3});
  \coordinate (C) at (\C{1},\C{2},\C{3});
  \coordinate (D) at (\D{1},\D{2},\D{3});

  % 绘制线段
  \draw (A) -- (B) (A) -- (D);

```

```
  % 标注点
  \foreach \p/\placement in {A/below left, B/below right, C/below, D/below}{
    \fill[red] (\p) circle (1.5pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



4.8 向量夹角

4.8.1 向量的夹角

向量夹角的标准数学定义是没有正负的，但为简单起见，`tikzlibraryvc.code.tex` 在计算二维向量夹角是采用了**有向角**，而三维向量夹角时采用了**无向角**。

二维向量的夹角可以定义正负，根据应用场景的不同，有不同的定义习惯。在二维空间中，由于存在明确的 “顺时针” 和 “逆时针” 概念，定义正负夹角是非常自然且强大的工具。

- 如果只关心大小，就用**无向角**（非负，Undirected/Unoriented Angle），取值范围是 $[0, 180^\circ]$ (或 $[0, \pi]$)。
- 如果关心方向（顺时针还是逆时针），就用**有向角**（带正负，Directed/Oriented Angle），取值范围是 $(-180^\circ, 180^\circ]$（或 $(-\pi, \pi]$）。

在标准的数学定义（特别是欧几里得几何和线性代数）中，三维空间中两个向量之间的夹角通常定义为非负值，没有正负之分。（如果要计算有向旋转角，需要指定了旋转轴或参考法向量。）

4.8.2 语法形式

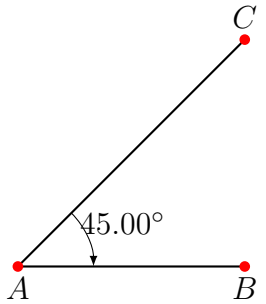
Syntax

- `plane/angle/origin={\A,\B,\a}`: 平面向量夹角，$a = \angle AOB$，有向角: $(-180°, 180°]$
- `plane/angle={\P,\A,\B,\a}`: 平面向量夹角，$a = \angle APB$，有向角: $(-180°, 180°]$
- `space/angle/origin={\A,\a}`: 空间向量夹角，$a = \angle AOB$，无向角: $[0°, 180°]$
- `space/angle={\P,\A,\B,\a}`: 空间向量夹角，$a = \angle APB$，无向角: $[0°, 180°]$

4.8.3 示例 1：平面向量夹角

```
\begin{tikzpicture}
  % 设置 pgf 数字打印格式
  \pgfkeys{/pgf/number format/.cd,
    fixed,          % 固定小数点表示 (非科学计数法)
    precision=2,    % 保留位小数
  }

  \tikzmath{
    \A{1} = -3; \A{2} = 0;
    \B{1} = 0; \B{2} = 0;
    \C{1} = 0; \C{2} = 3;
  }

  \tikzset{
    plane/angle={\A,\B,\C,\a},
  }

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});

  \draw[thick] (B) -- (A) -- (C);
  \pic [draw, latex-, "$\pgfmathprintnumber{\a}^\circ$", angle
   radius=1cm, angle eccentricity=1.5] {angle = B--A--C};

  \foreach \p/\placement in {A/below,B/below,C/above} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
```

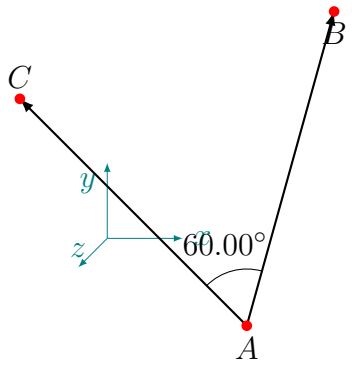
```
  }
\end{tikzpicture}
```



4.8.4 示例 2：空间向量夹角

```
\begin{tikzpicture}
  % 设置 pgf 数字打印格式
  \pgfkeys{/pgf/number format/.cd,
    fixed,          % 固定小数点表示（非科学计数法）
    precision=2,    % 保留位小数
  }

  \tikzmath{
    \A{1} = 3; \A{2} = 0; \A{3} = 3;
    \B{1} = 3; \B{2} = 3; \B{3} = 0;
    \C{1} = 0; \C{2} = 3; \C{3} = 3;
  }

  \tikzset{
    space/angle={\A,\B,\C,\a},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0,0) -- (1,0,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,1,0) node[below left
   ]{$y$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,0,1) node[above]{$z$};

  \coordinate (A) at (\A{1},\A{2},\A{3});
  \coordinate (B) at (\B{1},\B{2},\B{3});
  \coordinate (C) at (\C{1},\C{2},\C{3});

  \draw[thick,-latex] (A) -- (B);
  \draw[thick,-latex] (A) -- (C);

  \pic[draw, "$\pgfmathprintnumber{\a}^\circ$", angle radius=.75cm,
```

```
    angle eccentricity=1.5] {angle = B--A--C};

  \foreach \p/\placement in {A/below,B/below,C/above} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



4.9 向量投影（Vector Projection）、标量投影（Scalar Projection）与向量拒绝（Vector Rejection）

4.9.1 向量投影

向量投影（Vector Projection）的定义是向量 $\boldsymbol{a}$ 在向量 $\boldsymbol{b}$ 上的**正交投影**（Orthogonal Projection），是指将 $\boldsymbol{a}$ **垂直**投射到 $\boldsymbol{b}$ 所在的直线上所得到的向量。它代表了 $\boldsymbol{a}$ 在 $\boldsymbol{b}$ 方向上的分量。

几何意义：想象有一束光垂直于向量 $\boldsymbol{b}$ 照射，向量 $\boldsymbol{a}$ 在 $\boldsymbol{b}$ 上留下的 “影子” 就是投影向量。这个影子与 $\boldsymbol{b}$ 平行（同向或反向）。

设 $\boldsymbol{a}$ 和 $\boldsymbol{b}$ 为非零向量，$\boldsymbol{a}$ 在 $\boldsymbol{b}$ 上的投影记为 $\mathrm{proj}_{\boldsymbol{b}}\boldsymbol{a}$：

$$\mathrm{proj}_{\boldsymbol{b}}\boldsymbol{a} = \left(\frac{\boldsymbol{a}\cdot\boldsymbol{b}}{|\boldsymbol{b}|^2}\right)\boldsymbol{b}$$

或者使用单位向量 $\hat{\boldsymbol{b}} = \dfrac{\boldsymbol{b}}{|\boldsymbol{b}|}$ 表示：

$$\mathrm{proj}_{\boldsymbol{b}}\boldsymbol{a} = (\boldsymbol{a}\cdot\hat{\boldsymbol{b}})\hat{\boldsymbol{b}}$$

- $\boldsymbol{a} \cdot \boldsymbol{b}$ 是点积（标量）。
- $|\boldsymbol{b}|^2 = \boldsymbol{b} \cdot \boldsymbol{b}$ 是 $\boldsymbol{b}$ 长度的平方。
- 括号内的部分 $\dfrac{\boldsymbol{a} \cdot \boldsymbol{b}}{|\boldsymbol{b}|^2}$ 是一个标量系数，决定了投影的长度和方向。

4.9.2 标量投影

标量投影（Scalar Projection）是向量投影的长度（带符号），也称为分量：

$$\mathrm{comp}_{\boldsymbol{b}}\boldsymbol{a} = \frac{\boldsymbol{a} \cdot \boldsymbol{b}}{|\boldsymbol{b}|} = |\boldsymbol{a}| \cos\theta$$

4.9.3 向量拒绝

向量拒绝（Vector Rejection）的定义是向量 $\boldsymbol{a}$ 关于向量 $\boldsymbol{b}$ 的**拒绝**（Rejection，也称为正交分量），是指 $\boldsymbol{a}$ 中**垂直**于 $\boldsymbol{b}$ 的那部分分量。

几何意义：如果把 $\boldsymbol{a}$ 分解为**平行**于 $\boldsymbol{b}$ 和**垂直**于 $\boldsymbol{b}$ 两个部分，那么**拒绝**就是那个垂直的部分。它是从 $\boldsymbol{a}$ 的尖端到其在 $\boldsymbol{b}$ 上投影点的向量。

根据向量加法，$\boldsymbol{a}$ 等于其投影加上其拒绝：

$$\boldsymbol{a} = \mathrm{proj}_{\boldsymbol{b}}\boldsymbol{a} + \mathrm{rej}_{\boldsymbol{b}}\boldsymbol{a}$$

因此，拒绝可以通过从原向量中减去投影得到：

$$\mathrm{rej}_{\boldsymbol{b}}\boldsymbol{a} = \boldsymbol{a} - \mathrm{proj}_{\boldsymbol{b}}\boldsymbol{a}$$

代入投影公式：

$$\mathrm{rej}_{\boldsymbol{b}}\boldsymbol{a} = \boldsymbol{a} - \left(\frac{\boldsymbol{a} \cdot \boldsymbol{b}}{|\boldsymbol{b}|^2}\right)\boldsymbol{b}$$

性质：

- $\mathrm{rej}_{\boldsymbol{b}}\boldsymbol{a}$ 与 $\boldsymbol{b}$ 正交（垂直），即 $(\mathrm{rej}_{\boldsymbol{b}}\boldsymbol{a}) \cdot \boldsymbol{b} = 0$。
- 在三维空间中，如果 $\boldsymbol{b}$ 是平面的法向量，那么 $\mathrm{rej}_{\boldsymbol{b}}\boldsymbol{a}$ 就是 $\boldsymbol{a}$ 在该平面上的投影向量。

4.9.4 语法形式

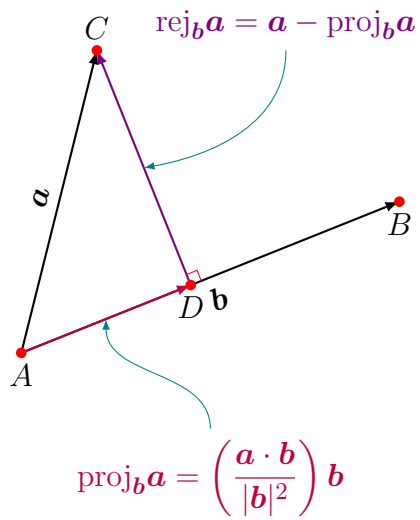
Syntax

- `plane/project/origin={\A,\B,\C}`: 平面向量 $\overline{OA}$ 在 $\overline{OB}$ 上的向量投影（$\overline{OC}$）
- `plane/project={\P,\A,\B,\C}`: 平面向量 $\overline{PA}$ 在 $\overline{PB}$ 上的向量投影（$\overline{PC}$），$\overline{OC}=\overline{OP}+\overline{OC}$
- `plane/component/origin={\A,\B,\comp}`: 平面向量 $\overline{OA}$ 在 $\overline{OB}$ 上的标量投影（`\comp`）
- `plane/component={\P,\A,\B,\comp}`: 平面向量 $\overline{PA}$ 在 $\overline{PB}$ 上的标量投影（`\comp`）
- `plane/reject/origin={\A,\B,\C}`: 平面向量 $\overline{OA}$ 在 $\overline{OB}$ 上的向量拒绝（$\overline{OC}$）
- `plane/reject={\P,\A,\B,\C}`: 平面向量 $\overline{PA}$ 在 $\overline{PB}$ 上的向量拒绝（$\overline{PC}$），$\overline{OC}=\overline{OP}+\overline{OC}$
- `space/project/origin={\A,\B,\C}`: 空间向量 $\overline{OA}$ 在 $\overline{OB}$ 上的向量投影（$\overline{OC}$）
- `space/project={\P,\A,\B,\C}`: 空间向量 $\overline{PA}$ 在 $\overline{PB}$ 上的向量投影（$\overline{PC}$），$\overline{OC}=\overline{OP}+\overline{OC}$
- `space/component/origin={\A,\B,\comp}`: 空间向量 $\overline{OA}$ 在 $\overline{OB}$ 上的标量投影（`\comp`）
- `space/component={\P,\A,\B,\comp}`: 空间向量 $\overline{PA}$ 在 $\overline{PB}$ 上的标量投影（`\comp`）
- `space/reject/origin={\A,\B,\C}`: 空间向量 $\overline{OA}$ 在 $\overline{OB}$ 上的向量拒绝（$\overline{OC}$）
- `space/reject={\P,\A,\B,\C}`: 空间向量 $\overline{PA}$ 在 $\overline{PB}$ 上的向量拒绝（$\overline{PC}$），$\overline{OC}=\overline{OP}+\overline{OC}$

4.9.5 示例 1：平面向量投影

```
\begin{tikzpicture}
  \tikzmath{
    \A{1} = -3; \A{2} = -2;
    \B{1} = 2; \B{2} = 0;
    \C{1} = -2; \C{2} = 2;
  }

  \tikzset{
```

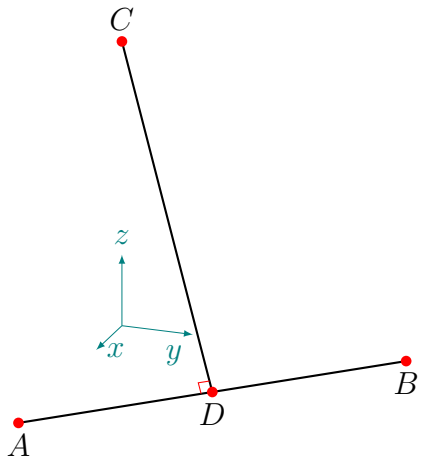
```
    plane/project={\A,\C,\B,\D},
  }

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (D) at (\D{1},\D{2});

  \draw[-latex,thick] (A) -- (B) node[midway, below, sloped] {$\bm{b}$};
  \draw[-latex,thick] (A) -- (C) node[midway, above, sloped] {$\bm{a}$};
  \draw[-latex,thick,purple] (A) -- (D);
  \draw[-latex,thick,violet] (D) -- (C);

  \draw[purple] (-.5,-3) node[below] {$\text{proj}_{\bm{b}}\bm{a} = \left( \dfrac{\bm{a} \cdot \bm{b}}{|\bm{b}|^2} \right) \bm{b}$};
  \draw[-latex,teal] (-.5,-3) to [out=90,in=-90] ($(A)!.5!(D)$);

  \draw[violet] (.5,2) node[above] {$\text{rej}_{\bm{b}}\bm{a} = \bm{a} - \text{proj}_{\bm{b}}\bm{a}$};
  \draw[-latex,teal] (.5,2) to [out=-90,in=10] ($(C)!.5!(D)$);

  \pic [draw,red,thin,angle radius=1.5mm]
    {right angle = B--D--C};

  \foreach \p/\placement in {A/below,B/below,C/above,D/below} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



4.9.6 示例 2：空间向量投影

```
% 设置主视角：仰角 70 度，方位角 110 度
\tdplotsetmaincoords{70}{110}
\begin{tikzpicture}[tdplot_main_coords]
  \tikzmath{
    % 直线上 A 和 B
    \A{1}=4; \A{2}=0; \A{3}=0;
    \B{1}=0; \B{2}=4; \B{3}=0;
    % 目标点 C
    \C{1}=0; \C{2}=0; \C{3}=4;
  }

  \tikzset{
    space/project={\A,\C,\B,\D},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0,0) -- (1,0,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,1,0) node[below left
   ]{$y$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,0,1) node[above]{$z$};

  \coordinate (A) at (\A{1},\A{2},\A{3});
  \coordinate (B) at (\B{1},\B{2},\B{3});
  \coordinate (C) at (\C{1},\C{2},\C{3});
  \coordinate (D) at (\D{1},\D{2},\D{3});

  \draw[thick] (A) -- (B);
  \draw[thick] (C) -- (D);

  \pic [draw,red,thin,angle radius=1.5mm]
    {right angle = A--D--C};

  \foreach \p/\placement in {A/below,B/below,C/above,D/below} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```

% 设置主视角：仰角 70 度，方位角 110 度



4.10 向量反射（Vector Reflection）

4.10.1 向量反射

向量反射（Vector Reflection）定义是向量 $\boldsymbol{a}$ 关于向量 $\boldsymbol{b}$（通常作为法向量）的**反射**，是指 $\boldsymbol{a}$ 像光线照在镜子上一样被“弹回”后形成的向量。这里的“镜子”是垂直于 $\boldsymbol{b}$ 的平面（或在 2D 中是垂直于 $\boldsymbol{b}$ 的直线）。

几何意义：常用于计算机图形学（如光线追踪计算反光）、物理（碰撞反弹）。入射向量是 $\boldsymbol{a}$，法线是 $\boldsymbol{b}$（通常需归一化），反射向量 $\boldsymbol{r}$ 与 $\boldsymbol{a}$ 关于法线对称。

反射向量 $\boldsymbol{r}$ 可以通过保留平行分量并反转垂直分量，或者更直观地通过投影推导出来。我们要减去两倍的“垂直分量”（即两倍的拒绝）

$$\mathrm{ref}_{\boldsymbol{b}}\boldsymbol{a} = \boldsymbol{a} - 2\mathrm{rej}_{\boldsymbol{b}}\boldsymbol{a}$$

代入拒绝公式：

$$\mathrm{ref}_{\boldsymbol{b}}\boldsymbol{a} = 2\left(\frac{\boldsymbol{a}\cdot\boldsymbol{b}}{|\boldsymbol{b}|^2}\right)\boldsymbol{b} - \boldsymbol{a}$$

4.10.2 语法形式

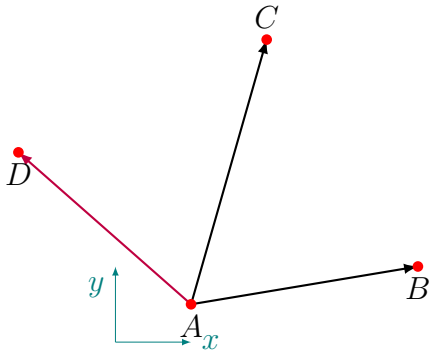
Syntax

- `plane/reflect/origin={\A,\B,\C}`: 平面向量 $\overline{OA}$ 关于 $\overline{OB}$ 的对称向量 $\overline{OC}$
- `plane/reflect={\P,\A,\B,\C}`: 平面向量 $\overline{PA}$ 关于 $\overline{PB}$ 的对称向量 $\overline{PC}$，$\overline{OC}=\overline{OP}+\overline{PC}$
- `space/reflect/origin={\A,\B,\C}`: 空间向量 $\overline{OA}$ 关于 $\overline{OB}$ 的对称向量 $\overline{OC}$
- `space/reflect={\P,\A,\B,\C}`: 空间向量 $\overline{PA}$ 关于 $\overline{PB}$ 的对称向量 $\overline{PC}$，$\overline{OC}=\overline{OP}+\overline{PC}$

4.10.3 示例 1：平面向量反射

```
\begin{tikzpicture}
  \tikzmath{
    \A{1} = 1; \A{2} = 0.5;
    \B{1} = 4; \B{2} = 1;
    \C{1} = 2; \C{2} = 4;
  }

  \tikzset{
    plane/reflect={\A,\B,\C,\D},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0) -- (1,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0) -- (0,1) node[below left]{$y
   $};

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (D) at (\D{1},\D{2});

  \draw [thick,-latex] (A) -- (B);
  \draw [thick,-latex] (A) -- (C);
  \draw [thick,-latex,purple] (A) -- (D);

  \foreach \p/\placement in {A/below,B/below,C/above,D/below} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
```

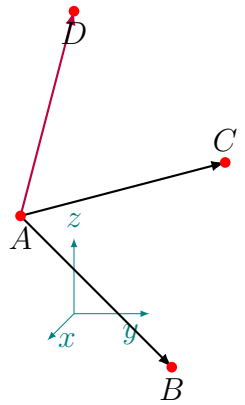
```
  }
\end{tikzpicture}
```



4.10.4 示例 2：空间向量反射

```
\begin{tikzpicture}[
  x={({0.5*cos(-135)},{0.5*sin(-135)})}, % default: cm
  y={(1cm,0cm)},
  z={(0cm,1cm)}
]
  \tikzmath{
    \A{1} = 2; \A{2} = 0; \A{3} = 2;
    \B{1} = 2; \B{2} = 2; \B{3} = 0;
    \C{1} = 0; \C{2} = 2; \C{3} = 2;
  }

  \tikzset{
    space/reflect={\A,\B,\C,\D},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0,0) -- (1,0,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,1,0) node[below left
   ]{$y$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,0,1) node[above]{$z$};

  \coordinate (A) at (\A{1},\A{2},\A{3});
  \coordinate (B) at (\B{1},\B{2},\B{3});
  \coordinate (C) at (\C{1},\C{2},\C{3});
  \coordinate (D) at (\D{1},\D{2},\D{3});

  \draw [thick,-latex] (A) -- (B);
  \draw [thick,-latex] (A) -- (C);
  \draw [thick,-latex,purple] (A) -- (D);

```

```
  \foreach \p/\placement in {A/below,B/below,C/above,D/below} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



4.11 平面向量旋转

4.11.1 平面向量的旋转

我们讨论的是将一个平面向量围绕**原点**逆时针旋转一个角度 θ。

假设有一个初始向量 $\boldsymbol{a} = (x, y)$，将其逆时针旋转角度 θ 得到新向量 $\boldsymbol{a'} = (x', y')$，其坐标计算公式为：

$$\begin{cases} x' = x\cos\theta - y\sin\theta \\ y' = x\sin\theta + y\cos\theta \end{cases}$$

> 如果是顺时针旋转，只需将 θ 替换为 $-\theta$。

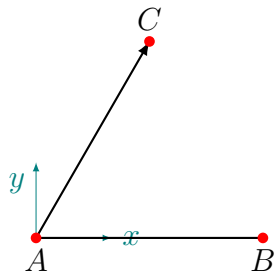
4.11.2 语法形式

Syntax

- `plane/rotate/origin={\A,\ang,\B}`: 平面向量 $\overline{OA}$ 绕原点旋转 `\ang` 角度到达 $\overline{OB}$
- `plane/project={\P,\A,\ang,\B}`: 平面向量 $\overline{PA}$ 绕点 P 旋转 `\ang` 角度到达 $\overline{PB}$，$\overline{OB} = \overline{OP} + \overline{PB}$

4.11.3 示例 1：平面向量旋转

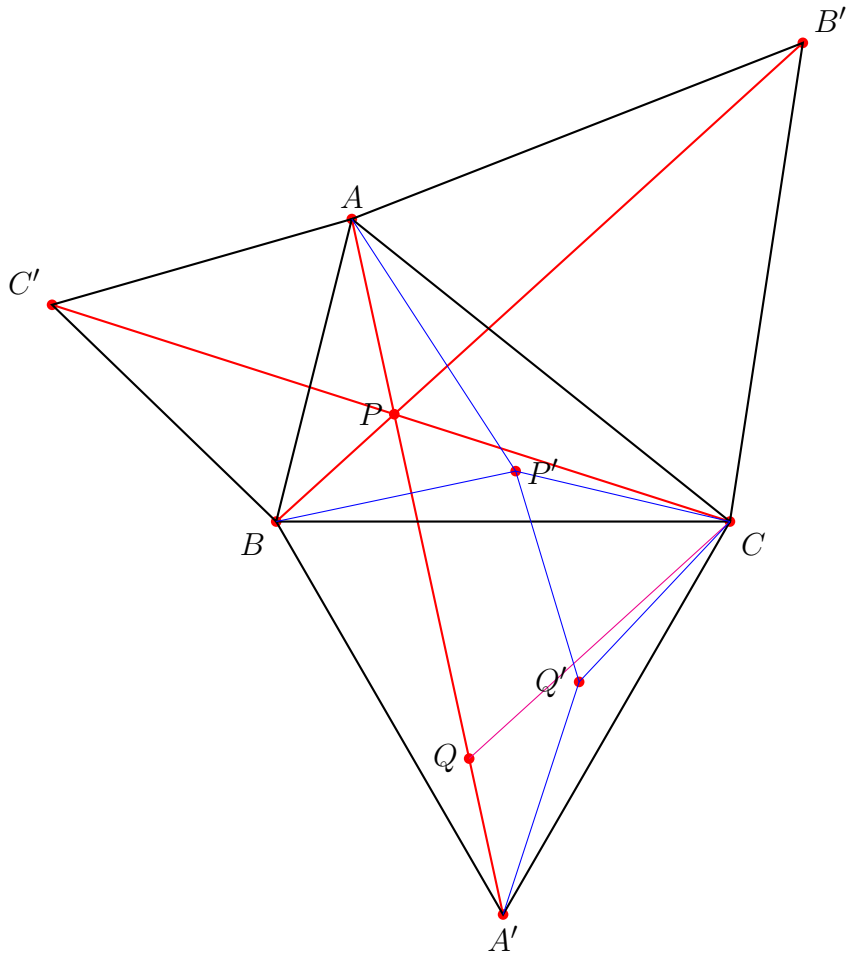
```
\begin{tikzpicture}
  \tikzmath{
    \A{1} = 0; \A{2} = 0;
    \B{1} = 3; \B{2} = 0;
  }

  \tikzset{
    plane/rotate={\A,\B,60,\C},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0) -- (1,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0) -- (0,1) node[below left]{$y
   $};

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});

  \draw[thick,-latex] (A) -- (B) (A) -- (C);

  \foreach \p/\placement in {A/below,B/below,C/above} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



4.11.4 示例 2：费马点

```
\begin{tikzpicture}
  \tikzmath{
    % 定义三角形 A, B, C
    \A{1} = -1; \A{2} = 4;
    \B{1} = -2; \B{2} = 0;
    \C{1} = 4; \C{2} = 0;
    % 定义三角形内部任意一点的重心坐标
    \x = 1; \y = 2; \z = 3;
  }
  \tikzset{
    plane/barycenter={\A,\B,\C,\x,\y,\z,\PP},
    plane/rotate={\B,\A,60,\CC},% A 绕 B 旋转 60 度
    plane/rotate={\A,\C,60,\BB},
    plane/rotate={\C,\B,60,\AA},
    plane/intersect ll={\A,\AA,\B,\BB,\P},
    plane/rotate={\C,\P,60,\Q},
    plane/rotate={\C,\PP,60,\QQ},
  }

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (P) at (\P{1},\P{2});
  \coordinate (Q) at (\Q{1},\Q{2});
  \coordinate (A') at (\AA{1},\AA{2});
  \coordinate (B') at (\BB{1},\BB{2});
  \coordinate (C') at (\CC{1},\CC{2});
  \coordinate (P') at (\PP{1},\PP{2});
  \coordinate (Q') at (\QQ{1},\QQ{2});

  \foreach \point in {A,B,C,A',B',C',P,Q,P',Q'}
    \fill (\point) [red] circle (2pt);
  \draw[thick,red] (A) -- (A') (B) -- (B') (C) -- (C');
  \draw[magenta] (Q) -- (C);
  \draw[thick] (A) -- (B) -- (C') -- cycle;
```

```
  \draw[thick] (B) -- (C) -- (A') -- cycle;
  \draw[thick] (C) -- (A) -- (B') -- cycle;
  \draw[blue] (A) -- (P') -- (Q') -- (A')
    (P') -- (C) (Q') -- (C) (P') -- (B);
  \draw (A) node[above] {$A$};
  \draw (B) node[below left] {$B$};
  \draw (C) node[below right] {$C$};
  \draw (A') node[below] {$A'$};
  \draw (B') node[above right] {$B'$};
  \draw (C') node[above left] {$C'$};
  \draw (P) node[left] {$P$};
  \draw (Q) node[left] {$Q$};
  \draw (P') node[right] {$P'$};
  \draw (Q') node[left] {$Q'$};
\end{tikzpicture}
```



4.12 空间向量旋转

4.12.1 罗德里格斯旋转公式

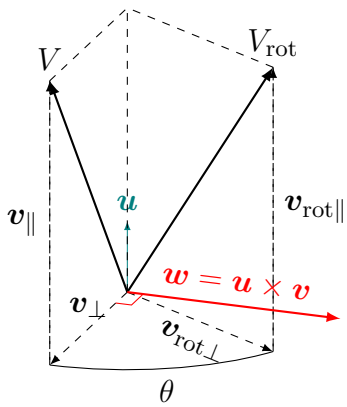
罗德里格斯旋转公式（Rodrigues' rotation formula）是三维空间中绕任意轴旋转一个向量的数学公式。该公式提供了一种直接的方法，将一个向量 $\boldsymbol{v}$ 绕单位向量 $\boldsymbol{u}$（旋转轴）旋转给定角度 θ。

设：

- $\boldsymbol{u}$ 是单位向量，表示旋转轴（即 $\|\boldsymbol{u}\| = 1$）。
- $\boldsymbol{v}$ 是要旋转的三维向量。
- θ 是旋转角度（按右手定则确定正方向）。

则旋转后的向量 $\boldsymbol{v}_{\text{rot}}$ 由下式给出：

$$\boldsymbol{v}_{\text{rot}} = \cos\theta\boldsymbol{v} + (\boldsymbol{u}\cdot\boldsymbol{v})(1-\cos\theta)\boldsymbol{u} + \sin\theta(\boldsymbol{u}\times\boldsymbol{v})$$



推导过程如下：

将原向量 $\boldsymbol{v}$ 投影到向量 $\boldsymbol{u}$ 上，可以分解为两个分量：

$$\boldsymbol{v} = \boldsymbol{v}_{\parallel} + \boldsymbol{v}_{\perp}$$

$$\boldsymbol{v}_{\parallel} = (\boldsymbol{u}\cdot\boldsymbol{v})\boldsymbol{u}$$

$$\boldsymbol{v}_{\perp} = \boldsymbol{v} - \boldsymbol{v}_{\parallel} = \boldsymbol{v} - \ \boldsymbol{u}\cdot\boldsymbol{v}\ \cdot\boldsymbol{u}$$

其中：

- 平行于旋转轴 $\boldsymbol{u}$ 的分量 $\boldsymbol{v}_{\parallel}$ 在旋转之后不变；
- 垂直于旋转轴 $\boldsymbol{u}$ 的分量 $\boldsymbol{v}_{\perp}$ 在垂直于 $\boldsymbol{u}$ 的平面内旋转 θ 角。

利用外积可以计算与 $\boldsymbol{u}$ 和 $\boldsymbol{v}$ 都垂直，且长度等于 $\boldsymbol{v}_{\perp}$ 的向量 $\boldsymbol{w}$：

$$\boldsymbol{w} = \boldsymbol{u} \times \boldsymbol{v}$$

注意：$\boldsymbol{v}_{\perp}$ 和 $\boldsymbol{w}$ 构成直角坐标系，旋转后的垂直分量 $\boldsymbol{v}_{\mathrm{rot}\perp}$ 的幅角为 θ，所以：

$$\boldsymbol{v}_{\mathrm{rot}\perp} = \cos\theta \boldsymbol{v}_{\perp} + \sin\theta \boldsymbol{w}$$

因此，旋转后的向量为：

$$\boldsymbol{v}_{\mathrm{rot}} = \boldsymbol{v}_{\parallel} + \boldsymbol{v}_{\mathrm{rot}\perp}$$

代入各项公式，得到：

$$\boldsymbol{v}_{\mathrm{rot}} = \cos\theta \boldsymbol{v} + (\boldsymbol{u} \cdot \boldsymbol{v})(1 - \cos\theta)\boldsymbol{u} + \sin\theta(\boldsymbol{u} \times \boldsymbol{v})$$

4.12.2 语法形式

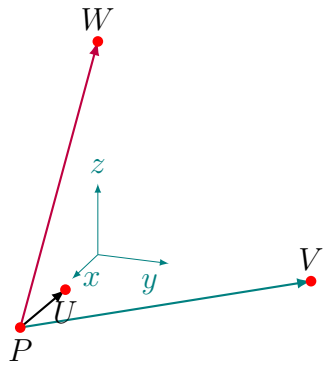
Syntax

- `space/rotate/origin={\U,\V,\ang,\W}`: 空间向量 $\overline{OV}$ 绕 $\overline{OU}$ 轴旋转 `\ang` 角度到达 $\overline{OW}$
- `space/rotate={\P,\U,\V,\ang,\W}`: 空间向量 $\overline{PV}$ 绕 $\overline{PU}$ 轴旋转 `\ang` 角度到达 $\overline{PW}$，$\overline{OW} = \overline{OP} + \overline{PW}$

> 在向量计算库内部会将旋转轴向量单位化。

4.12.3 示例：空间向量旋转

```latex
\tdplotsetmaincoords{70}{110}

\begin{tikzpicture}[tdplot_main_coords]
  % 设置主视角：仰角 70 度，方位角 110 度
  \tikzmath{
    % 旋转轴和待旋转向量起点
    \P{1} = 3; \P{2} = 0; \P{3} = 0;
    % 旋转轴终点
    \U{1} = 4; \U{2} = 1; \U{3} = 1;
    % 待旋转向量终点
    \V{1} = 0; \V{2} = 3; \V{3} = 0;
  }

  \tikzset{
    space/rotate={\P,\U,\V,60,\W},
  }

  % 绘制坐标轴
  \draw[help lines,teal,-latex] (0,0,0) -- (1,0,0) node[right]{$x$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,1,0) node[below left]{$y$};
  \draw[help lines,teal,-latex] (0,0,0) -- (0,0,1) node[above]{$z$};

  \coordinate (P) at (\P{1},\P{2},\P{3});
  \coordinate (U) at (\U{1},\U{2},\U{3});
  \coordinate (V) at (\V{1},\V{2},\V{3});
  \coordinate (W) at (\W{1},\W{2},\W{3});

  \draw[thick,-latex] (P) -- (U);
  \draw[thick,-latex,teal] (P) -- (V);
  \draw[thick,-latex,purple] (P) -- (W);

  \foreach \p/\placement in {P/below,U/below,V/above,W/above} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```


W
z
x
y
V
U
P

第五章　直线与圆

5.1　直线

5.1.1　原点在直线上的投影

将原点 $(0,0)$ 投影到直线 $ax+by+c=0$ 上，实质上是求从原点向直线作垂线，垂足 $P(x_0,y_0)$ 的坐标。

这个投影点具有两个特性：

1. **在直线上**：满足 $ax_0+by_0+c=0$。
2. **方向垂直**：向量 $\vec{OP}$ 与直线的法向量 (a,b) 平行。

对于直线 $ax+by+c=0$，原点的投影点 P 的坐标公式为：

$$\left(\frac{-ac}{a^2+b^2},\frac{-bc}{a^2+b^2}\right)$$

5.1.2　直线的方向向量

普通的直线 $ax+by+c=0$ 是没有方向的，但当我们把它变为**有向直线**时，通常遵循标准参数化定义的正方向。当我们把直线方程写成**参数方程**时，正方向由参数 t 增加的方向决定。规定直线与 x 轴正方向所成的角 α 为倾斜角（$\alpha\in[0,\pi)$），直线的参数方程为：

$$\begin{cases}x=x_0+t\cos\alpha\\ y=y_0+t\sin\alpha\end{cases}$$

定义直线的方向向量为:

$$\boldsymbol{v} = \begin{cases} (-b, a) & a > 0 \\ (|b|, 0) & a = 0 \\ (b, -a) & a < 0 \end{cases}$$

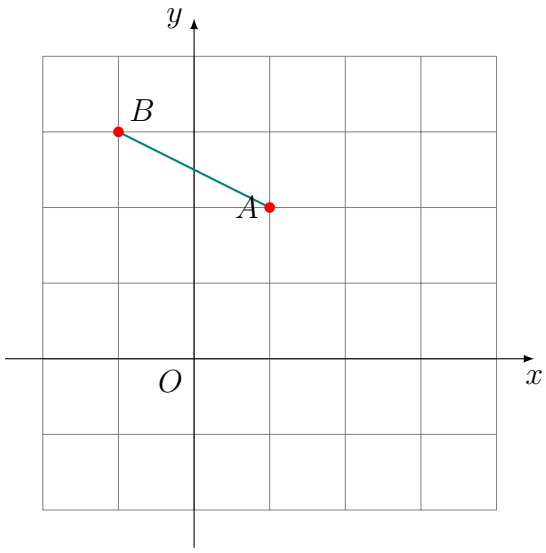
5.1.3 语法形式

Syntax

- `plane/anchor={\a,\b,\c,\P}`: 计算原点在直线 $ax + by + c = 0$ 上的投影 P
- `plane/direct={\a,\b,\c,\v}`: 计算直线 $ax + by + c = 0$ 的方向向量 $\boldsymbol{v}$ (注意单位化)

5.1.4 示例

```
\begin{tikzpicture}
  \tikzmath{
    % 直线方程的系数
    \a = 1; \b = 2; \c = -5;
  }
  \tikzset{
    plane/anchor={\a,\b,\c,\A},
    plane/direct={\a,\b,\c,\V},
    plane/add/origin={\A,\V,\B},
  }

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});

  \axes[grid] (-2:4,-2:4);
  \draw[teal,thick] (A) -- (B);

  \foreach \p/\placement in {A/left,B/above right}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



5.2 两直线的交点

5.2.1 两直线的交点公式

设平面上四点坐标分别为：$A(x_1, y_1)$、$B(x_2, y_2)$、$C(x_3, y_3)$ 和 $D(x_4, y_4)$，则直线 AB 和 CD 的一般式方程分别为：

$$\begin{cases} (y_1 - y_2)x + (x_2 - x_1)y & = x_2y_1 - x_1y_2 \\ (y_3 - y_4)x + (x_4 - x_3)y & = x_4y_3 - x_3y_4 \end{cases}$$

可以通过高斯消去法解该方程组求得交点坐标。

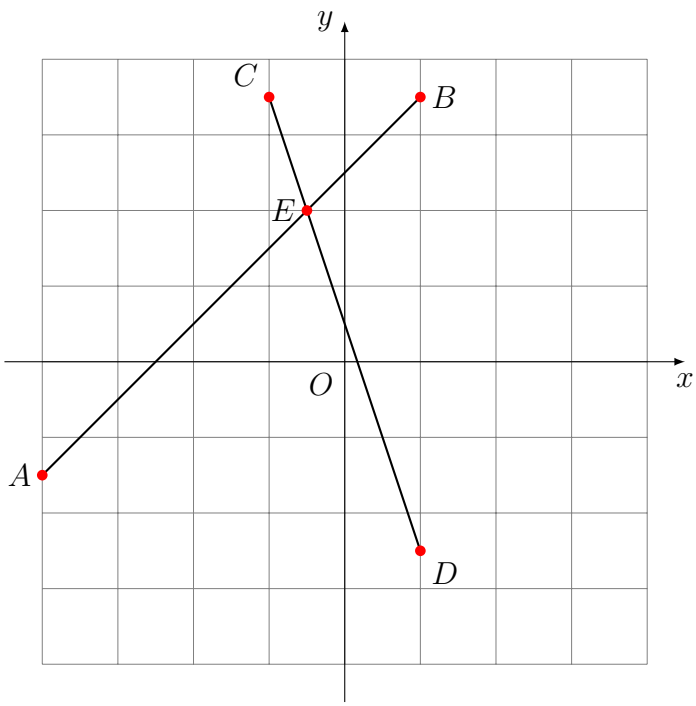
5.2.2 语法形式

Syntax

- `plane/intersect ll={\A,\B,\C,\D,\P}`: 计算直线 AB 与 CD 的交点 P

5.2.3 示例：笛卡尔坐标下两直线的交点

```cpp
\begin{tikzpicture}
  \tikzmath{
    \A{1} = -4; \A{2} = -1.5;
    \B{1} = 1; \B{2} = 3.5;
    \C{1} = -1; \C{2} = 3.5;
    \D{1} = 1; \D{2} = -2.5;
  }
  \tikzset{
    plane/intersect ll={\A,\B,\C,\D,\E},
  }
  \axes[grid] (-4:4,-4:4);
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (D) at (\D{1},\D{2});
  \coordinate (E) at (\E{1},\E{2});
  \draw[thick] (A) -- (B) (C) -- (D);
  \foreach \p/\placement in {A/left,B/right,
  C/above left,D/below right,E/left}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



5.3 直线与圆的交点

5.3.1 直线与圆的交点公式

利用向量投影的方法求解圆与直线的交点, 先找到圆心在直线上的投影点 (即垂足), 然后利用勾股定理计算从垂足到交点的距离向量。利用向量投影（bmtor Projection）求解圆与直线的交点是一种非常直观且高效的几何方法。它避开了繁琐的代数方程组（联立二次方程），通过**几何分量**的拆解来直接计算坐标。

假设圆的中心为 $O(x_0, y_0)$，半径为 r；直线由两点 $A(x_1, y_1)$ 和 $B(x_2, y_2)$ 定义。

我们将整个过程拆解为寻找三个关键向量：直线方向向量、圆心到直线的垂直投影点、以及投影点到交点的偏移量。

第一步：确定直线向量与单位向量

首先，定义直线的方向向量 $\boldsymbol{v}$:

$$\boldsymbol{v} = \overline{AB}$$

为了方便投影计算，将其单位化为 $\boldsymbol{u}$:

$$\boldsymbol{u} = \frac{\boldsymbol{v}}{|\boldsymbol{v}|}$$

第二步：计算圆心在直线上的投影点 M

先计算向量 $\overline{AO}$ 在向量 $\overline{AB}$ 上的投影 $\overline{AM}$，然后计算 $M = A + \overline{AM}$，见 *[向量投影 (Vector Projection)]*。

第三步：判定相交关系

计算圆心 O 到直线（即到投影点 M）的垂直距离 d:

$$d = |\overline{OM}|$$

- 如果 $d > r$：直线与圆不相交。
- 如果 $d = r$：直线与圆相切，交点即为 M。
- 如果 $d < r$：直线与圆有两个交点。

第四步：求解交点坐标

在直角三角形中（由圆心 O、投影点 M、交点 P 组成），利用勾股定理求出 M 到交点 P 的半弦长 h：

$$h = \sqrt{r^2 - d^2}$$

由于交点都在直线上，它们相对于 M 的偏移方向正是直线的单位向量 $\boldsymbol{u}$：

$$P = M \pm h \cdot \boldsymbol{u}$$

5.3.2 语法形式

Syntax

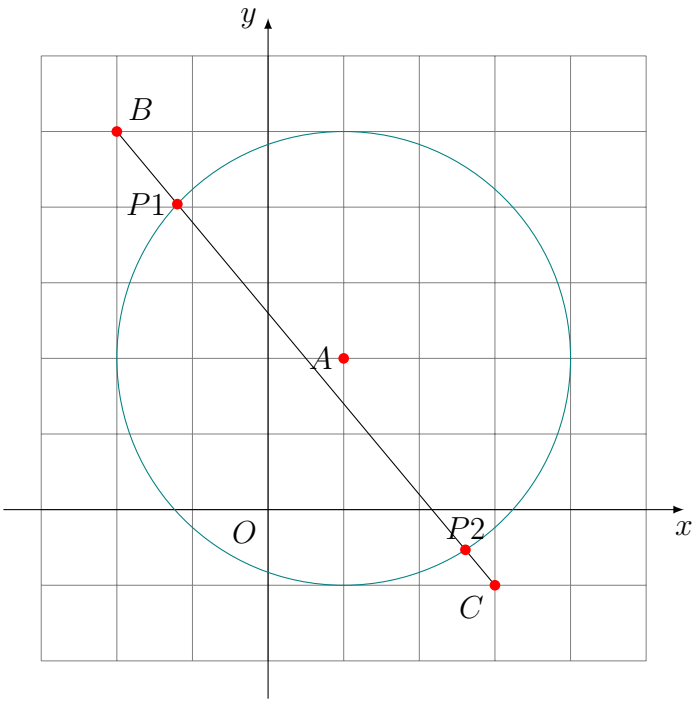
- `plane/intersect cl={\O,\r,\A,\B,\Pa,\Pb}`: 计算圆（圆心为 O，半径为 r）与直线 AB 的交点 P_a 和 P_b，交点的顺序是从 A 至 B
- `plane/intersect cn={\O,\r,\a,\b,\c,\Pa,\Pb}`: 计算圆（圆心为 O，半径为 r）与直线 $ax + by + c = 0$ 的交点 P_a 和 P_b，交点的顺序是直线 $ax + by + c = 0$ 的正方向

或。

5.3.3 示例

```
\begin{tikzpicture}
  \tikzmath{
    \radius = 3;
    \A{1} = 1; \A{2} = 2;
    \B{1} = -2; \B{2} = 5;
    \C{1} = 3; \C{2} = -1;
  }
  \tikzset{
    plane/intersect cl={\A,\radius,\B,\C,\Pa,\Pb},
  }
  \axes[grid] (-3:5,-2:6);
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (P1) at (\Pa{1},\Pa{2});
```

```
  \coordinate (P2) at (\Pb{1},\Pb{2});
  \draw[teal] (A) circle (\radius);
  \draw (B) -- (C);
  \foreach \p/\placement in {A/left,B/above right,
  C/below left,P1/left,P2/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



5.4 两圆的根轴与交点

5.4.1 根轴

根轴（Radical Axis）是指平面上对两个不同心圆的圆幂（Power of a Point）相等的点的轨迹。这条轨迹是一条直线，且该直线垂直于两圆圆心的连线（连心线 O_1O_2）。

设平面内有两个不同心的圆：

- 圆 C_1：圆心为 O_1，半径为 r_1。
- 圆 C_2：圆心为 O_2，半径为 r_2。

如果你需要在具体问题中求出根轴方程，可以遵循以下步骤：

1. **建立坐标系或选取基点**：通常以 O_1 为原点，或者直接利用已知坐标。计算连心线向量 $\overline{d} = \overline{O_1O_2} = O_2 - O_1$。计算距离平方 $D^2 = |\overline{d}|^2$。
2. **计算常数项**：计算 $C = \frac{1}{2}(D^2 + r_1^2 - r_2^2)$。
3. **写出向量方程**：根轴上任意点 P 满足：

$$\overline{O_1P} \cdot \overline{d} = C$$

即：

$$(P - O_1) \cdot (O_2 - O_1) = \frac{1}{2}(|O_2 - O_1|^2 + r_1^2 - r_2^2)$$

特殊情况

- **两圆相交**：根轴即为过两交点的**公共弦**所在的直线。
- **两圆相切**：根轴即为过切点的**公切线**。
- **两圆相离**：根轴位于两圆之间，垂直于连心线，且靠近半径较小的圆（如果 $r_1 \neq r_2$）。
- **两圆半径相等** ($r_1 = r_2$)：此时常数项 $C = \frac{1}{2}|\overline{O_1O_2}|^2$。方程变为 $\overline{O_1P} \cdot \overline{O_1O_2} = \frac{1}{2}|\overline{O_1O_2}|^2$。这意味着 $\overline{O_1P}$ 在 $\overline{O_1O_2}$ 上的投影长度是 O_1O_2 长度的一半。此时根轴是线段 O_1O_2 的**垂直平分线**。

使用向量 $\overline{O_1O_2}$ 求解根轴的核心公式为：

$$\overline{O_1P} \cdot \overline{O_1O_2} = \frac{|\overline{O_1O_2}|^2 + r_1^2 - r_2^2}{2}$$

该式清晰地表明根轴是一条**法向量为** $\overline{O_1O_2}$ 的直线。

求解两圆的交点的问题可以转换为求解根轴与其中一圆的交点。

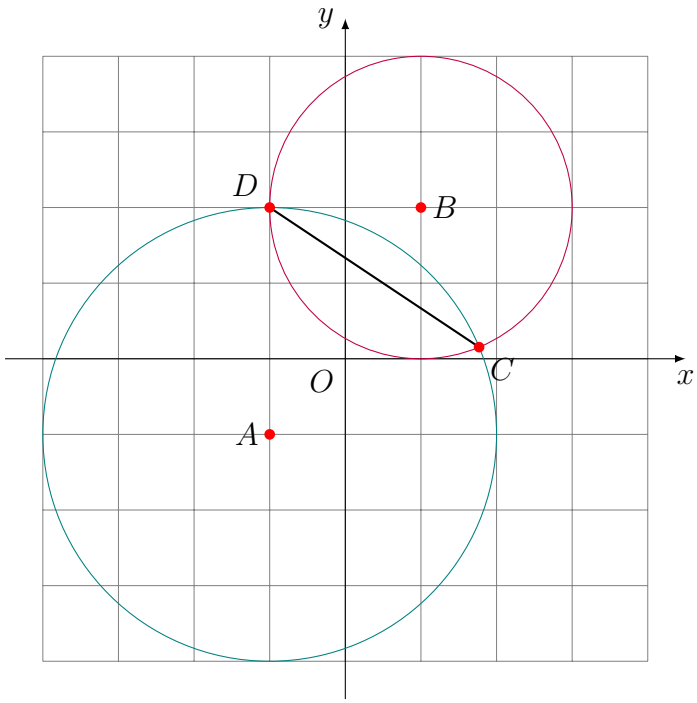
5.4.2 语法形式

Syntax

- `plane/radical axis={\Oa,\ra,\Ob,\rb,\H,\U}`：计算根轴的参数：根轴与连心线的交点 H 和根轴的单位方向向量 U
- `plane/intersect cc={\Oa,\ra,\Ob,\rb,\Pa,\Pb}`：计算两圆的交点 P_a 和 P_b，其顺序是根轴的正方向

5.4.3 示例 1：求两圆的交点

```
\begin{tikzpicture}
  \tikzmath{
    \ra = 3;
    \rb = 2;
    \A{1} = -1; \A{2} = -1;
    \B{1} = 1; \B{2} = 2;
  }
  \tikzset{
    plane/intersect cc={\A,\ra,\B,\rb,\C,\D},
  }
  \axes[grid] (-4:4,-4:4);
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (D) at (\D{1},\D{2});

  \draw[teal] (A) circle (\ra);
  \draw[purple] (B) circle (\rb);

  \draw[thick] (C) -- (D);

  \foreach \p/\placement in {A/left,B/right,C/below right,D/above left}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```

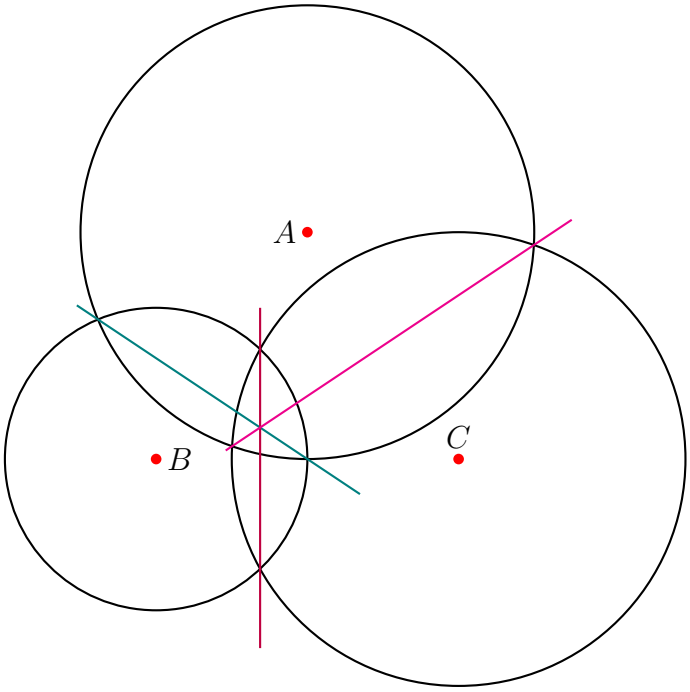


5.4.4 示例 2：根心定理

根心定理（Radical Center Theorem），也称为三圆根轴定理，是平面几何中关于圆的幂（Power of a Point）的一个重要结论。当三个圆的圆心不在同一直线上时，两两圆之间的三条根轴（Radical Axes）必交于一点，这个点被称为这三个圆的**根心（Radical Center）**。

```
\begin{tikzpicture}
  \tikzmath{
    \A{1} = 3; \A{2} = 3;
    \ra = 3;
    \B{1} = 1; \B{2} = 0;
    \rb = 2;
    \C{1} = 5; \C{2} = 0;
    \rc = 3;
  }

  \tikzset{
    plane/radical axis={\A,\ra,\B,\rb,\Pa,\Ua},
    plane/radical axis={\B,\rb,\C,\rc,\Pb,\Ub},
    plane/radical axis={\C,\rc,\A,\ra,\Pc,\Uc},
    plane/add/origin={\Pa,\Ua,\Qa},
    plane/add/origin={\Pb,\Ub,\Qb},
    plane/add/origin={\Pc,\Uc,\Qc},
  }
```

```
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});

  \coordinate (Pa) at (\Pa{1},\Pa{2});
  \coordinate (Pb) at (\Pb{1},\Pb{2});
  \coordinate (Pc) at (\Pc{1},\Pc{2});

  \coordinate (Qa) at (\Qa{1},\Qa{2});
  \coordinate (Qb) at (\Qb{1},\Qb{2});
  \coordinate (Qc) at (\Qc{1},\Qc{2});

  \draw[thick] (A) circle (\ra);
  \draw[thick] (B) circle (\rb);
  \draw[thick] (C) circle (\rc);

  \draw[thick,teal]  ($(Pa)!-2.5!(Qa)$) -- ($(Pa)!2!(Qa)$);
  \draw[thick,purple] ($(Pb)!-2.5!(Qb)$) -- ($(Pb)!2!(Qb)$);
  \draw[thick,magenta] ($(Pc)!-2.5!(Qc)$) -- ($(Pc)!3!(Qc)$);

  \foreach \p/\placement in {A/left,B/right,C/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



5.5 圆外一点作圆的切线

5.5.1 圆的切线

利用“直径所对的圆周角是直角”这一性质从圆外一点 P 作圆 O 的切线，或者利用“极点-极线”的性质来作圆的切线。此处采用第 1 种方法，即以 OP 为直径作辅助圆，该圆与圆 O 的交点即切点。

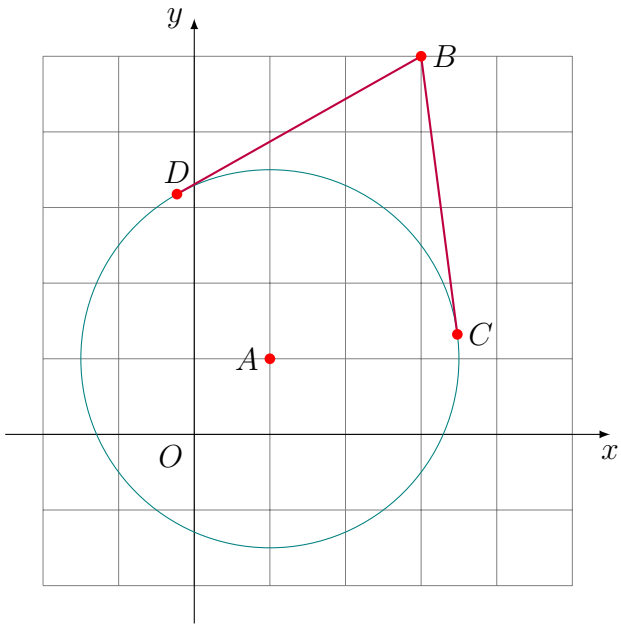
5.5.2 语法形式

Syntax

- `plane/tangents={\O,\r,\P,\Ta,\Tb}`: 计算过 P 作圆 O 的切线与圆的切点 T_a 和 T_b，其顺序是圆 O 与辅助圆的根轴的正方向

5.5.3 示例

```
\begin{tikzpicture}
  \tikzmath{
    \ra = 2.5;
    \A{1} = 1; \A{2} = 1;
    \B{1} = 3; \B{2} = 5;
  }
  \tikzset{
    plane/tangents={\A,\ra,\B,\C,\D},
  }
  \axes[grid] (-2:5,-2:5);
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (D) at (\D{1},\D{2});

  \draw[teal] (A) circle (\ra);

  \draw[thick,purple] (B) -- (C) (B) -- (D);

  \foreach \p/\placement in {A/left,B/right,C/right,D/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



5.6 两圆的位似中心与公切线

5.6.1 位似中心

在几何学中，两个圆总是位似（homothetic）的，即总能找到一个点（位似中心）位似中心（Homothetic Center），使得其中一个圆可以通过以该点为中心进行缩放（放缩）和位移，从而与另一个圆完全重合。对于任意两个圆，通常存在两个位似中心：外位似中心与内位似中心。

如何找到它们？一个简单的方法是画出两组相互平行的半径：

1. 画出圆 O_1 的半径 R_1 和圆 O_2 的同向平行半径 R_2，连接半径末端的直线与连心线的交点即为**外位似中心**。
2. 画出圆 O_1 的半径 R_1 和圆 O_2 的反向平行半径 R_2，连接半径末端的直线与连心线的交点即为**内位似中心**。

如果使用向量 $\overline{O_1O_2}$ 作为基准，位似中心 P 的位置可以直接表示为从 O_1 出发的偏移量。这种表达方式在解析几何和参数方程中非常实用。

设定：$\overline{O_1}$ 为起点，$\overline{O_1O_2} = \overline{O_2} - \overline{O_1}$ 为基准向量。

1. **外位似中心（External Center of Similitude）**

外位似中心位于两圆的连心线之上，且在两圆的同一侧。

- **几何意义**：它是两圆的**外切线**的交点（如果两圆半径不等）。
- **特性**：如果你从这个点观察两个圆，你会看到一个小圆套在大圆里面，或者两个圆在视线上完美重叠。
- **坐标规律**：它将连心线段按半径之比 $r_1 : r_2$ 进行**外分**。

外位似中心位于 O_1O_2 的延长线上（假设 $r_2 > r_1$ 时在 O_1 左侧，$r_1 > r_2$ 时在 O_2 右侧）。其通项公式为：

$$\overline{P_{ex}} = \overline{O_1} + \frac{r_1}{r_1 - r_2}\overline{O_1O_2}$$

符号分析：

- 如果 $r_1 > r_2$，系数 $\dfrac{r_1}{r_1 - r_2} > 1$，说明 P_{ex} 在 O_2 的右侧。
- 如果 $r_1 < r_2$，系数 $\dfrac{r_1}{r_1 - r_2} < 0$，说明 P_{ex} 在 O_1 的左侧（反方向）。
- 如果 $r_1 = r_2$，分母为 0，向量无解，验证了平移圆无有限位似中心。

2. **内位似中心（Internal Center of Similitude）**

内位似中心同样位于两圆的连心线上，但位于两圆之间。

- **几何意义**：它是两圆的**内公切线**的交点（如果内公切线存在）。
- **特性**：经过这个点的光线，在穿过一个圆后，其对应的影像会在另一个圆上呈现倒立的状态。
- **坐标规律**：它将连心线段按半径之比 $r_1 : r_2$ 进行**内分**。

根据分点公式的向量变形：

$$\overline{P_{in}} = \overline{O_1} + \frac{r_1}{r_1 + r_2}\overline{O_1O_2}$$

表 5.1: 特殊情况说明

情况	位似中心的状态
等圆 (半径相等)	外位似中心在无穷远处（外切线平行）；内位似中心是连心线的中点。
同心圆	内外位似中心重合，都在圆心处。
内含/内切	内位似中心在圆内；外位似中心在圆外或切点处。
相交/外离	两个中心均存在。

5.6.2 公切线

两圆的**公切线**（Common Tangent）是指同时与两个圆相切的直线。根据两圆的相对位置，公切线的数量和种类会有所不同。

公切线的分类

- **外公切线**：两圆位于切线的同一侧。
- **内公切线**：两圆位于切线的两侧（切线穿过两圆之间）。

两圆公切线的数量直接取决于**圆心距** d 与两圆半径 r_1, r_2（假设 $r_1 \geq r_2$）的关系：

表 5.2: 两圆位置关系与切线数量

相对位置	判定条件	外公切线	内公切线	总数
外离	$d > r_1 + r_2$	2	2	4
外切	$d = r_1 + r_2$	2	1	3
相交	$r_1 - r_2 < d < r_1 + r_2$	2	0	2
内切	$d = r_1 - r_2$	1	0	1
内含	$d < r_1 - r_2$	0	0	0

5.6.3 语法形式

Syntax

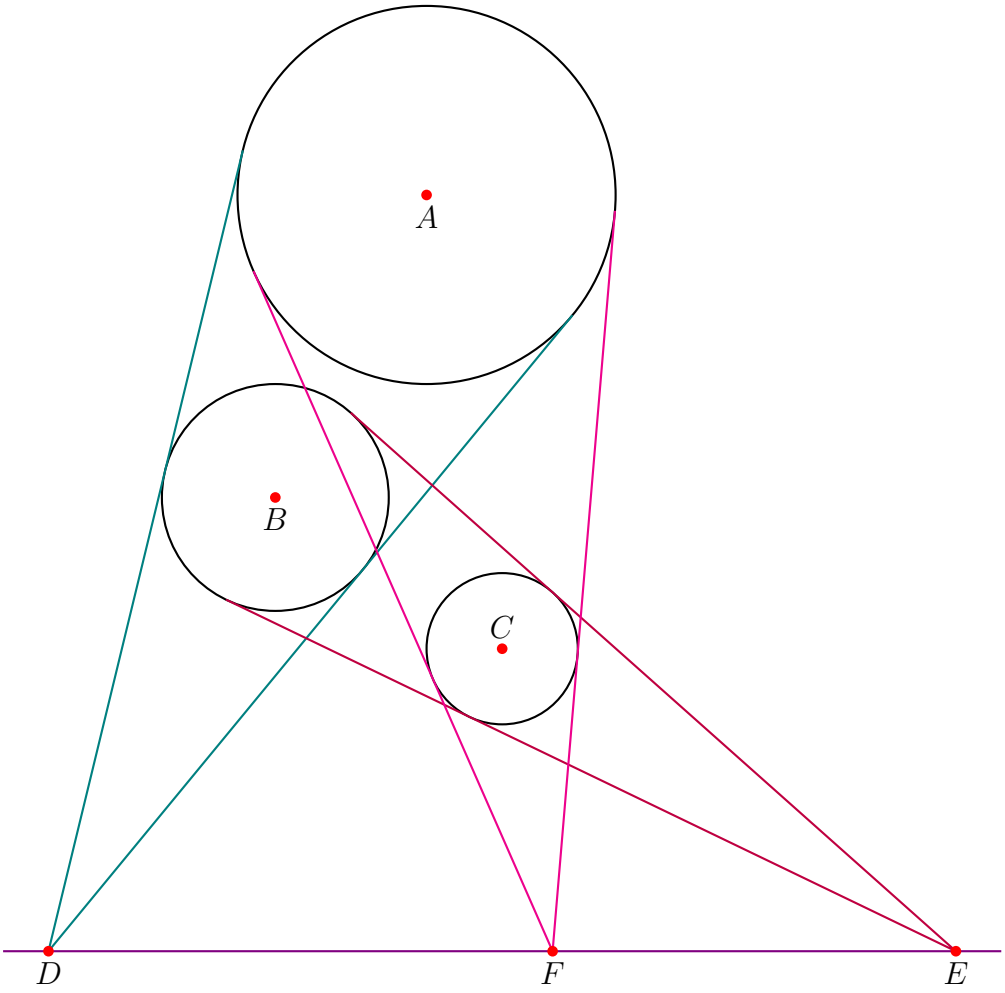
- `plane/external center={\Oa,\ra,\Ob,\rb,\P}`: 两圆的外位似中心
- `plane/internal center={\Oa,\ra,\Ob,\rb,\P}`: 两圆的内位似中心

5.6.4 示例：蒙日定理

蒙日定理（Monge's Theorem）指出，对于平面上的三个圆（半径互不相等，且没有一个圆包含在另一个圆内），每两个圆的两条外公切线的交点（即外位似中心）共有三个，这三个点共线。

```
\begin{tikzpicture}
  \tikzmath{
    \A{1} = 3; \A{2} = 7;
```

```
    \ra = 2.5;
    \B{1} = 1; \B{2} = 3;
    \rb = 1.5;
    \C{1} = 4; \C{2} = 1;
    \rc = 1;
  }

  \tikzset{
    plane/external center={\A,\ra,\B,\rb,\D},
    plane/external center={\B,\rb,\C,\rc,\E},
    plane/external center={\C,\rc,\A,\ra,\F},
    plane/tangents={\A,\ra,\D,\Ua,\Ub},
    plane/tangents={\B,\rb,\E,\Va,\Vb},
    plane/tangents={\A,\ra,\F,\Wa,\Wb},
  }

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (D) at (\D{1},\D{2});
  \coordinate (E) at (\E{1},\E{2});
  \coordinate (F) at (\F{1},\F{2});

  \draw[thick] (A) circle (\ra);
  \draw[thick] (B) circle (\rb);
  \draw[thick] (C) circle (\rc);

  \draw[thick,teal] (D) -- (\Ua{1},\Ua{2}) (D) -- (\Ub{1},\Ub{2});
  \draw[thick,purple] (E) -- (\Va{1},\Va{2}) (E) -- (\Vb{1},\Vb{2});
  \draw[thick,magenta] (F) -- (\Wa{1},\Wa{2}) (F) -- (\Wb{1},\Wb{2})
   ;
  \draw[thick,violet] ($(D)!-.05!(E)$) -- ($(D)!1.05!(E)$);

  \foreach \p/\placement in {A/below,B/below,C/above,D/below,E/below
   ,F/below} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



5.7 反演变换

5.7.1 反演点

反演点 (Inverse Points) 是几何变换中 “圆反演” (Circle Inversion) 的核心概念。简单来说，它是一种将圆内点映射到圆外点（反之亦然）的对称变换。

假设有一个以点 O 为圆心、半径为 R 的圆，这个圆被称为**反演圆**。

对于平面上任意一点 P（除圆心 O 外），如果在射线 OP 上有一点 P'，满足以下数量关系：

$$|OP| \cdot |OP'| = R^2$$

那么，P 与 P' 就互为关于该圆的**反演点**。

- 若 P 在圆内，则 P' 在圆外；
- 若 P 在圆外，则 P' 在圆内；
- 若 P 在圆周上，则 P 与 P' 重合（即 $P = P'$）；
- 若 P 趋近于圆心 O，则 P' 趋近于无穷远处。

圆反演（Circle Inversion）是几何学中一个非常强大且优雅的工具。简单来说，它是一种将平面上的点“翻转”到圆内或圆外的变换。圆反演最神奇的地方在于它能将复杂的几何关系转化为简单的关系（例如将相切的圆转化为平行的直线）。

- **圆/线转换**：
  - 过反演中心的直线，反演后仍为**直线**（自身）。
  - 不过反演中心的直线，反演后变为**过反演中心的圆**。
  - 不过反演中心的圆，反演后仍为**圆**。
- **保角性**：反演变换不改变曲线之间的夹角（角度大小不变，但方向相反）。
- **距离关系**：设 A, B 的反演点为 A', B'，则它们之间的距离公式为：

$$|A'B'| = \frac{R^2 \cdot |AB|}{|OA| \cdot |OB|}$$

5.7.2 语法形式

Syntax

- `plane/inverse={\O,\r,\P,\Q}`: 计算点 P 关于圆 O 的反演点 Q

5.7.3 示例 1：证明托勒密定理

圆反演的一个基本性质：**过反演中心的圆，反演后会变成一条直线**。在证明托勒密定理时，我们巧妙地利用把“圆内接四边形”变成了“直线上的三个点”，从“乘积和”变成“线段相加”。

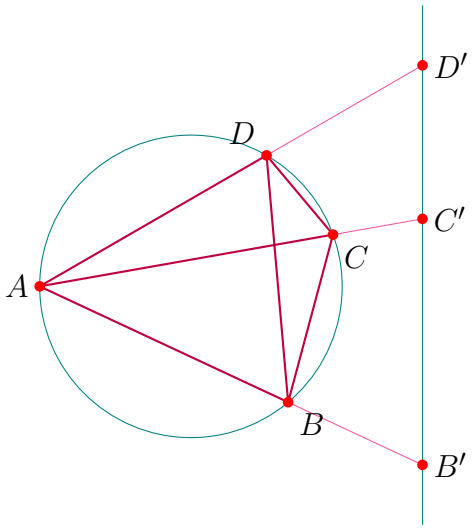
一旦 B', C', D' 变成了直线上的三个点，且 C' 在中间，那么它们之间的距离关系就变得极其简单：

$$B'D' = B'C' + C'D'$$

这就是托勒密定理证明中最关键的一步。接下来，只需要利用反演的距离公式

（$X'Y' = \dfrac{r^2 \cdot XY}{AX \cdot AY}$）把 $B'D'$、$B'C'$、$C'D'$ 还原回原来的弦长，代入上面的简单加法等式，整理后就能直接得到托勒密定理的公式。

```
\begin{tikzpicture}
  % 使用反演变换证明托勒密定理
  \tikzmath{
    % 定义圆(O,radius)上四点 A, B, C, D
    \radius = 2;
    \anga = 180;
    \angb = -50;
    \angc = 20;
    \angd = 60;
    \A{1} = \radius * cos(\anga); \A{2} = \radius * sin(\anga);
    \B{1} = \radius * cos(\angb); \B{2} = \radius * sin(\angb);
    \C{1} = \radius * cos(\angc); \C{2} = \radius * sin(\angc);
    \D{1} = \radius * cos(\angd); \D{2} = \radius * sin(\angd);
    % 以 A 为反演中心，半径为 Radius，作 B, C, D 的反演点
    \Radius = 4.5;
  }
  \tikzset{
    plane/inverse={\A,\Radius,\B,\Bi},
    plane/inverse={\A,\Radius,\C,\Ci},
    plane/inverse={\A,\Radius,\D,\Di},
  }

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (D) at (\D{1},\D{2});
  \coordinate (B') at (\Bi{1},\Bi{2});
  \coordinate (C') at (\Ci{1},\Ci{2});
  \coordinate (D') at (\Di{1},\Di{2});

  \draw[teal] (0,0) circle (\radius);
  \draw[purple,thick] (A) -- (B) -- (C) -- (D) -- cycle (A) -- (C) (B) -- (D);
  \draw[magenta!80] (B) -- (B') (C) -- (C') (D) -- (D');
  \draw[teal] ($(B')!-0.15!(D')$) -- ($(B')!1.15!(D')$);

  \foreach \p/\placement in {A/left,B/below right,C/below right,D/above left,
  B'/right,C'/right,D'/right}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



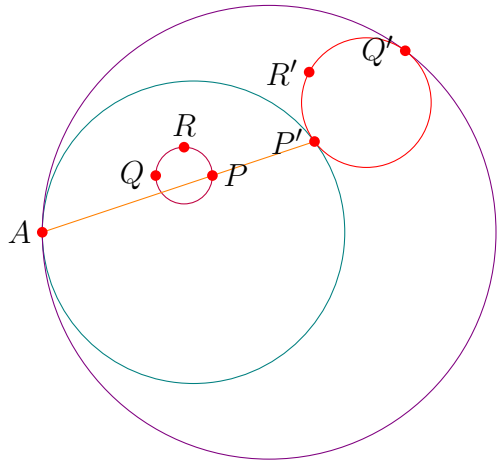
5.7.4 示例 2：Pappus Chain

帕普斯链的定义是：在由三个半圆围成的鞋匠刀形区域内，构造一系列圆使得每个圆都与另外两个半圆以及前一个圆相切。

> 在反演变换之后，切点依然是切点。这是反演变换的一个核心性质，称为保角性 (Conformality)。千万不要直接把圆心反演过去当作新圆心，必须通过计算新圆的半径和位置关系（如利用切线距离）来确定新圆心。

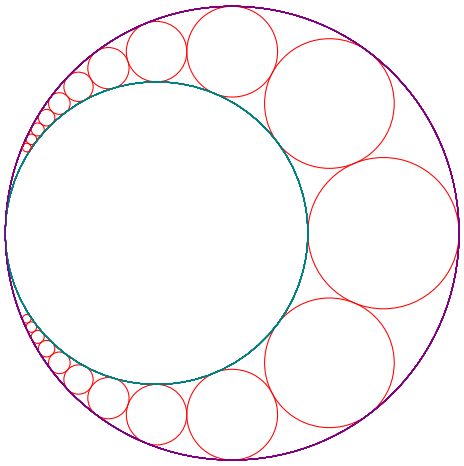
```
\begin{tikzpicture}
  \tikzmath{
    % 定义两圆心，半径及切点
    \Oa{1} = 2; \Oa{2} = 0; \ra = 2;
    \Ob{1} = 3; \Ob{2} = 0; \rb = 3;
    \A{1} = 0; \A{2} = 0;
    % 以 A 为反演中心 2 r 为半径作反演变换
    \r = 3;
    % 圆 Oa 变换为直线 x = 2 r^2 / ra
    % 圆 Ob 变换为直线 x = 2 r^2 / rb
    % 作与两直线相切的圆
    \i = 1; % 任意值
    \d = (\r)^2 / (2*\ra) - (\r)^2 / (2*\rb);
    \P{1} = (\r)^2 / (2*\ra); \P{2} = \i*\d; % 直径端点
    \Q{1} = (\r)^2 / (2*\rb); \Q{2} = \P{2}; % 直径端点
    \R{1} = (\P{1}+\Q{1})/2;
    \R{2} = (\P{1}-\Q{1})/2+\P{2};
  }
```

```
  \tikzset{
    plane/inverse={\A,(\rb),\P,\Pa},
    plane/inverse={\A,(\rb),\Q,\Qa},
    plane/inverse={\A,(\rb),\R,\Ra},
    plane/circumcenter={\Pa,\Qa,\Ra,\Ca},
    plane/length={\Ca,\Pa,\rc},
    plane/circumcenter={\P,\Q,\R,\C},
    plane/length={\C,\P,\rd},
  }

  \coordinate (O1) at (\Oa{1},\Oa{2});
  \coordinate (O2) at (\Ob{1},\Ob{2});
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (P) at (\P{1},\P{2});
  \coordinate (Q) at (\Q{1},\Q{2});
  \coordinate (R) at (\R{1},\R{2});
  \coordinate (C') at (\Ca{1},\Ca{2});
  \coordinate (P') at (\Pa{1},\Pa{2});
  \coordinate (Q') at (\Qa{1},\Qa{2});
  \coordinate (R') at (\Ra{1},\Ra{2});

  \draw[teal] (O1) circle (\ra);
  \draw[violet] (O2) circle (\rb);
  \draw[red] (C') circle (\rc);
  \draw[purple] (C) circle (\rd);

  \draw[orange] (A) -- (P');

  \foreach \p/\placement in {A/left,P/right,Q/left,R/above,
  P'/left,Q'/left,R'/left}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```




```tikzpicture
\begin{tikzpicture}
  \foreach \i in {-10,...,10}{
    \tikzmath{
      % 定义两圆心，半径及切点
      \Oa{1} = 2; \Oa{2} = 0; \ra = 2;
      \Ob{1} = 3; \Ob{2} = 0; \rb = 3;
      \A{1} = 0; \A{2} = 0;
      % 以 A 为反演中心 2 r 为半径作反演变换
      \r = 3;
      % 圆 Oa 变换为直线 x = 2 r^2 / ra
      % 圆 Ob 变换为直线 x = 2 r^2 / rb
      % 作与两直线相切的圆
      \d = (\r)^2 / (2*\ra) - (\r)^2 / (2*\rb);
      \P{1} = (\r)^2 / (2*\ra); \P{2} = \i*\d; % 直径端点
      \Q{1} = (\r)^2 / (2*\rb); \Q{2} = \P{2}; % 直径端点
      \R{1} = (\P{1}+\Q{1})/2;
      \R{2} = (\P{1}-\Q{1})/2+\P{2};
    }
    \tikzset{
      plane/inverse={\A,(\rb),\P,\Pa},
      plane/inverse={\A,(\rb),\Q,\Qa},
      plane/inverse={\A,(\rb),\R,\Ra},
      plane/circumcenter={\Pa,\Qa,\Ra,\Ca},
      plane/length={\Ca,\Pa,\rc},
    }

    \coordinate (O1) at (\Oa{1},\Oa{2});
    \coordinate (O2) at (\Ob{1},\Ob{2});
    \coordinate (A) at (\A{1},\A{2});
    \coordinate (P) at (\P{1},\P{2});
    \coordinate (Q) at (\Q{1},\Q{2});
    \coordinate (R) at (\R{1},\R{2});
    \coordinate (C') at (\Ca{1},\Ca{2});
    \coordinate (P') at (\Pa{1},\Pa{2});
    \coordinate (Q') at (\Qa{1},\Qa{2});
```

```
    \coordinate (R') at (\Ra{1},\Ra{2});

    \draw[teal] (O1) circle (\ra);
    \draw[violet] (O2) circle (\rb);
    \draw[red] (C') circle (\rc);
  }
\end{tikzpicture}
```



第六章　三角形的中心

6.1　Barycentric Coordinates

在三角形的几何研究中，三角形内一点（通常称为 P 点）与三个顶点（A、B、C）之间的向量关系，最核心的就是**重心坐标（Barycentric Coordinates）**表示法。

6.1.1　重心坐标形式

设三角形为 ABC，其顶点对应的坐标分别为 A, B, C。若点 P 在三角形 ABC 内部（或边界上），则存在实数 x, y, z 使得：

$$P = xA + yB + zC$$

其中满足条件：

$$x + y + z = 1$$

且若 P 严格在三角形内部，则：

$$x > 0, \quad y > 0, \quad z > 0$$

这里的 x, y, z 称为点 P 关于 $\triangle ABC$ 的**重心坐标（Barycentric Coordinates）**。

这些系数的具体物理意义是：$x = \dfrac{S_{\triangle PBC}}{S_{\triangle ABC}}$，$y = \dfrac{S_{\triangle PAC}}{S_{\triangle ABC}}$，$z = \dfrac{S_{\triangle PAB}}{S_{\triangle ABC}}$。即系数代表了对侧小三角形面积与总面积的比值。

主要包含两个部分：

1. **存在性与唯一性**：证明平面内任意一点 P 都可以唯一地表示为 $P = xA+yB+zC$ 且 $x+y+z=1$。
2. **内部点的充要条件**：证明点 P 位于三角形内部（不含边界）当且仅当系数 x, y, z 均大于 0。

对于三角形的重心, 外心, 垂心内心外心可以用统一的公式形式 [1] [2] [3].

1. 重心 (Centroid) G

$$\overline{GA} + \overline{GB} + \overline{GC} = \mathbf{0}$$

2. 外心 (Circumcenter) O

$$\sin 2A \cdot \overline{OA} + \sin 2B \cdot \overline{OB} + \sin 2C \cdot \overline{OC} = \mathbf{0}$$

3. 垂心 (Orthocenter) H

$$\tan A \cdot \overline{HA} + \tan B \cdot \overline{HB} + \tan C \cdot \overline{HC} = \mathbf{0}$$

   由于直角的正切不存在，此时需要使用 $\tan\theta = \sin\theta/\cos\theta$ 进行转化。

4. 内心 (Incenter) I

$$a \cdot \overline{IA} + b \cdot \overline{IB} + c \cdot \overline{IC} = \mathbf{0}$$

   a, b, c 是对应边长

5. 旁心 (Excenter) J

$$-a \cdot \overline{J_A A} + b \cdot \overline{J_A B} + c \cdot \overline{J_A C} = \mathbf{0}$$

   上面是顶点 A 对应的外心，其它两个系数分别是 $(a, -b, c)$ 和 $(a, b, -c)$。

[1] https://en.wikipedia.org/wiki/Triangle_center
[2] https://mathworld.wolfram.com/TriangleCenter.html
[3] https://mathworld.wolfram.com/BarycentricCoordinates.html

6.1.2 语法形式

Syntax

- `plane/barycenter={\A,\B,\C,\a,\b,\c,\H}`: 计算重心坐标 $(x:y:z)$ 对应的点 `\H`

6.2 三角形的重心 Centroid

6.2.1 重心及其性质

连接三角形的一个顶点与它对边中点的线段叫做**中线（Median）**。一个三角形有三条中线，它们总是交于一点，这个点就是**重心（Centroid）**。

重心的性质

1. 重心到顶点的距离与重心到对边中点的距离之比为 $2:1$。
2. 重心和三角形任意两个顶点组成的 3 个三角形面积相等。即重心到三条边的距离与三条边的长成反比。
3. 重心到三角形 3 个顶点距离的平方和最小。
4. 在平面直角坐标系中，重心的坐标是顶点坐标的算术平均数，即其重心坐标为

$$(1,1,1)$$

5. 以重心为起点，以三角形三顶点为终点的三条向量之和等于零向量。

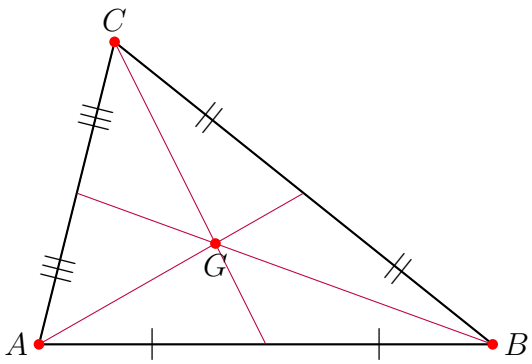
6.2.2 语法形式

Syntax

- `plane/centroid={\A,\B,\C,\G}`: 三角形 ABC 的重心

6.2.3 示例

```tikzpicture
\begin{tikzpicture}
  \tikzmath{
    \A{1} = -2; \A{2} = 0;
    \B{1} = 4; \B{2} = 0;
    \C{1} = -1; \C{2} = 4;
  }

  \tikzset{
    plane/centroid={\A,\B,\C,\G},
  }

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (G) at (\G{1},\G{2});
  \coordinate (D) at ($(B)!0.5!(C)$);
  \coordinate (E) at ($(C)!0.5!(A)$);
  \coordinate (F) at ($(A)!0.5!(B)$);

  \draw[thick] (A) -- (B) -- (C) -- cycle;
  \draw[purple] (A) -- (D) (B) -- (E) (C) -- (F);

  \draw (A) -- (B) node[near start,sloped] {$|$} node[near end,
   sloped] {$|$};
  \draw (B) -- (C) node[near start,sloped] {$||$} node[near end,
   sloped] {$||$};
  \draw (C) -- (A) node[near start,sloped] {$|||$} node[near end,
   sloped] {$|||$};

  \foreach \p/\placement in {A/left,B/right,C/above,G/below} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



6.3 三角形的外心 Circumcenter

6.3.1 外心及其性质

三角形三条边的垂直平分线相交于一点，这个点到三角形三个顶点的距离相等。由于外心到三个顶点的距离相等，因此以该点为圆心、以该距离为半径可以画出一个经过三角形所有顶点的圆，称为外接圆。

外心的坐标由下面的公式计算:

$$\sin 2A \cdot \overline{OA} + \sin 2B \cdot \overline{OB} + \sin 2C \cdot \overline{OC} = \mathbf{0}$$

由正弦定理可知:

$$\sin A : \sin B : \sin C = a : b : c$$

由余弦定理可知:

$$\cos A : \cos B : \cos C = \frac{b^2 + c^2 - a^2}{2bc} : \frac{c^2 + a^2 - b^2}{2ca} : \frac{a^2 + b^2 - c^2}{2ab}$$

由这些式子可以计算:

$$\sin 2A : \sin 2B : \sin 2C = a^2(b^2 + c^2 - a^2) : b^2(c^2 + a^2 - b^2) : c^2(a^2 + b^2 - c^2)$$

外心的性质

1. 当三角形为锐角三角形时，外心在三角形内部。
2. 当三角形为钝角三角形时，外心在三角形外部。
3. 当三角形为直角三角形时，外心在斜边上，与斜边的中点重合。
4. 外心到三顶点的距离相等。

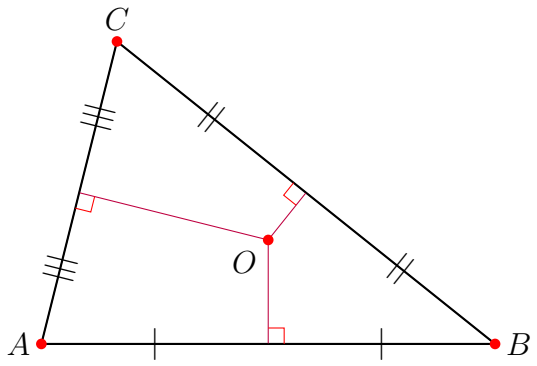
6.3.2 语法形式

Syntax

- `plane/circumcenter={\A,\B,\C,\O}`: 三角形 ABC 的外心

6.3.3 示例

```
\begin{tikzpicture}
  \tikzmath{
    \A{1} = -2; \A{2} = 0;
    \B{1} = 4; \B{2} = 0;
    \C{1} = -1; \C{2} = 4;
  }

  \tikzset{
    plane/circumcenter={\A,\B,\C,\O},
  }

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (O) at (\O{1},\O{2});
  \coordinate (D) at ($(B)!(O)!(C)$);
  \coordinate (E) at ($(C)!(O)!(A)$);
  \coordinate (F) at ($(A)!(O)!(B)$);

  \draw[thick] (A) -- (B) -- (C) -- cycle;
  \draw[purple] (O) -- (D) (O) -- (E) (O) -- (F);

  \draw (A) -- (B) node[near start,sloped] {$|$} node[near end,
   sloped] {$|$};
  \draw (B) -- (C) node[near start,sloped] {$||$} node[near end,
   sloped] {$||$};
  \draw (C) -- (A) node[near start,sloped] {$|||$} node[near end,
   sloped] {$|||$};
  \pic [draw,red,angle radius=6pt] {right angle=O--D--C};
  \pic [draw,red,angle radius=6pt] {right angle=O--E--A};
  \pic [draw,red,angle radius=6pt] {right angle=O--F--B};

  \foreach \p/\placement in {A/left,B/right,C/above,O/below left} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



6.4 三角形的垂心 Orthocenter

6.4.1 垂心及其性质

从三角形的每个顶点向其对边（或对边的延长线）引垂线，这三条垂线一定会交于一点，这个点就是垂心。

垂心的坐标由下面的公式计算：

$$\tan A \cdot \overline{HA} + \tan B \cdot \overline{HB} + \tan C \cdot \overline{HC} = \mathbf{0}$$

仿照三角形外心的推导过程（见三角形的外心 *Circumcenter*[p. 91]），可以由正弦定理和余弦定理得到坐标公式为：

$$\big((a^2+b^2-c^2)(c^2+a^2-b^2), (b^2+c^2-a^2)(a^2+b^2-c^2), (c^2+a^2-b^2)(b^2+c^2-a^2)\big)$$

垂心的性质

1. 三角形三个顶点，三个垂足，垂心这 7 个点可以得到 6 个四点圆。
2. 三角形外心 O，重心 G 和垂心 H 三点共线（**欧拉线**），且 $OG : GH = 1 : 2$。
3. 垂心到三角形一顶点距离为此三角形外心到此顶点对边距离的 2 倍。
4. 垂心分每条高线的两部分乘积相等。

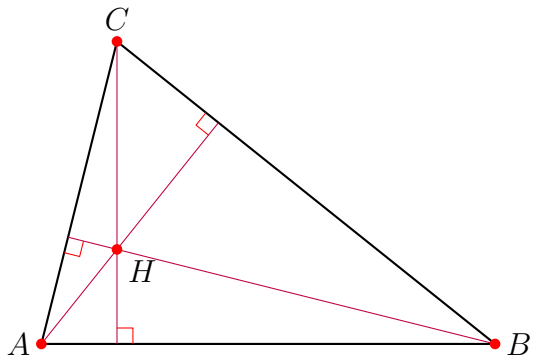
6.4.2 语法形式

Syntax

- `plane/orthocenter={\A,\B,\C,\H}`: 三角形 ABC 的垂心

6.4.3 示例

```
\begin{tikzpicture}
  \tikzmath{
    \A{1} = -2; \A{2} = 0;
    \B{1} = 4; \B{2} = 0;
    \C{1} = -1; \C{2} = 4;
  }

  \tikzset{
    plane/orthocenter={\A,\B,\C,\H},
  }

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (H) at (\H{1},\H{2});
  \coordinate (D) at ($(B)!(A)!(C)$);
  \coordinate (E) at ($(A)!(B)!(C)$);
  \coordinate (F) at ($(B)!(C)!(A)$);

  \draw[thick] (A) -- (B) -- (C) -- cycle;
  \draw[purple] (A) -- (D) (B) -- (E) (C) -- (F);

  \pic [draw,red,angle radius=6pt] {right angle=H--D--C};
  \pic [draw,red,angle radius=6pt] {right angle=H--E--A};
  \pic [draw,red,angle radius=6pt] {right angle=H--F--B};

  \foreach \p/\placement in {A/left,B/right,C/above,H/below right} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



6.5 三角形的内心 Incenter

6.5.1 内心及其性质

三角形三个内角的平分线相交于一点，这个点到三角形三条边的垂直距离相等。由于内心到三边的距离相等，因此以该点为圆心、以该距离为半径可以画出一个与三角形三边都相切的圆，称为内切圆（Incircle）。

内心的性质

1. 直角三角形的内心到边的距离等于两直角边的和与斜边的差的二分之一。
2. 欧拉定理 $\triangle ABC$ 中，R 和 r 分别为外接圆为和内切圆的半径，O 和 I 分别为其外心和内心，则 $OI^2 = R^2 - 2Rr$。
3. 内心到三角形三边距离相等。

6.5.2 语法形式

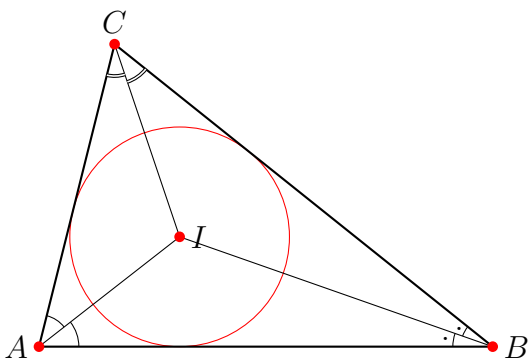
Syntax

- `plane/incenter={\A,\B,\C,\I}`: 三角形 ABC 的内心

6.5.3 示例

```
\begin{tikzpicture}
  \tikzmath{
    \A{1} = -2; \A{2} = 0;
```

```
    \B{1} = 4; \B{2} = 0;
    \C{1} = -1; \C{2} = 4;
  }

  \tikzset{
    plane/incenter={\A,\B,\C,\I},
    plane/excenter={\A,\B,\C,\Ja},
    plane/excenter={\B,\C,\A,\Jb},
    plane/excenter={\C,\A,\B,\Jc},
  }

  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (I) at (\I{1},\I{2});

  \node[draw,red] at (I) [circle through=($(B)!(I)!(C)$)]{};
  \draw[thick] (A) -- (B) -- (C) -- cycle;
  \draw (A) -- (I) (B) -- (I) (C) -- (I);

  \pic [draw,angle radius=12pt] {angle=I--A--C};
  \pic [draw,angle radius=15pt] {angle=B--A--I};
  \pic [draw,double,angle radius=12pt] {angle=A--C--I};
  \pic [draw,double,angle radius=15pt] {angle=I--C--B};
  \pic [draw,pic text=.,angle radius=12pt,
    angle eccentricity=1.2] {angle=C--B--I};
  \pic [draw,pic text=.,angle radius=15pt,
    angle eccentricity=1.2] {angle=I--B--A};

  \foreach \p/\placement in {A/left,B/right,C/above,I/right} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



6.6 三角形的旁心 Excenter

6.6.1 旁心及其性质

三角形的一条内角平分线与其他两个顶角的外角平分线交于一点，这个点到三角形的一条边及其余两边的延长线的距离相等。

旁切圆：以旁心为圆心、以该距离为半径，可以画出一个与三角形的一条边以及另外两条边的延长线相切的圆，称为旁切圆（Excircle）。

每个三角形都有三个旁心，分别位于三个顶点的对面（通常记作 J_a, J_b, J_c）。

旁心的性质

1. 旁心到三角形三条边（或其延长线）的距离相等
2. 原三角形的内心 I 是旁心三角形 $J_AJ_BJ_C$ 的垂心
3. 原三角形的外接圆是旁心三角形的九点圆。

6.6.2 语法形式

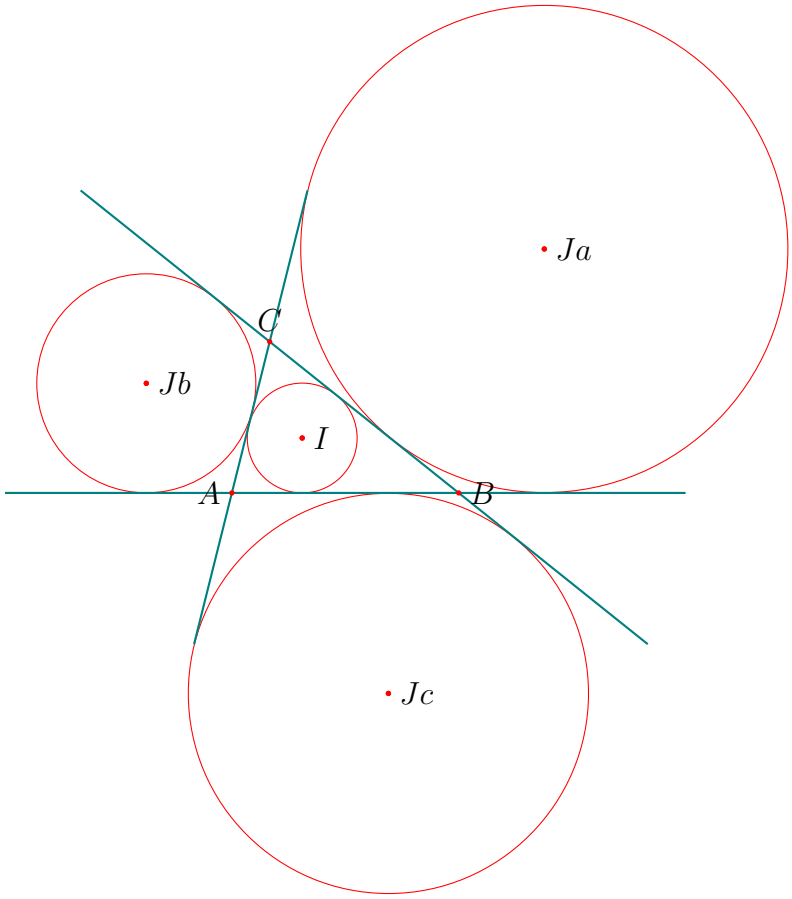
Syntax

- `plane/excenter={\A,\B,\C,\J}`: 顶点 A 的对面区域（即跨越边 BC 的一侧）的旁心，改变参数位置可以得其它旁心

6.6.3 示例

```
\begin{tikzpicture}[scale=.5]
  \tikzmath{
    \A{1} = -2; \A{2} = 0;
    \B{1} = 4; \B{2} = 0;
    \C{1} = -1; \C{2} = 4;
  }

  \tikzset{
    plane/incenter={\A,\B,\C,\I},
    plane/excenter={\A,\B,\C,\Ja},
    plane/excenter={\B,\C,\A,\Jb},
    plane/excenter={\C,\A,\B,\Jc},
  }

```

```
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
  \coordinate (I) at (\I{1},\I{2});
  \coordinate (Ja) at (\Ja{1},\Ja{2});
  \coordinate (Jb) at (\Jb{1},\Jb{2});
  \coordinate (Jc) at (\Jc{1},\Jc{2});

  \foreach \point in {I,Ja,Jb,Jc}
    \fill (\point) [red] circle (2pt);

  \node[draw,red] at (I) [circle through=($(B)!(I)!(C)$)]{};
  \node[draw,red] at (Ja) [circle through=($(B)!(Ja)!(C)$)]{};
  \node[draw,red] at (Jb) [circle through=($(B)!(Jb)!(C)$)]{};
  \node[draw,red] at (Jc) [circle through=($(B)!(Jc)!(C)$)]{};

  \draw[teal,thick] ($(A)!-1!(B)$) -- ($(A)!2!(B)$);
  \draw[teal,thick] ($(B)!-1!(C)$) -- ($(B)!2!(C)$);
  \draw[teal,thick] ($(C)!-1!(A)$) -- ($(C)!2!(A)$);

  \foreach \p/\placement in {A/left,B/right,C/above,I/right,Ja/right
   ,Jb/right,Jc/right} {
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



6.7 综合示例：九点圆

九点圆（Nine-Point Circle）是平面几何中一个优美且重要的定理，也被称为**欧拉圆**或**费尔巴哈圆**。

在任意一个三角形中，以下九个特殊的点必定在同一个圆上（这九个点可以分为三组）：

1. **三边的中点**：三角形三条边的中点。
2. **三条高的垂足**：从三角形三个顶点向对边作垂线，垂线与对边的交点。
3. **垂心与顶点连线的中点**：连接三角形的垂心（三条高的交点）与三个顶点，这三条线段各自的中点。

主要性质

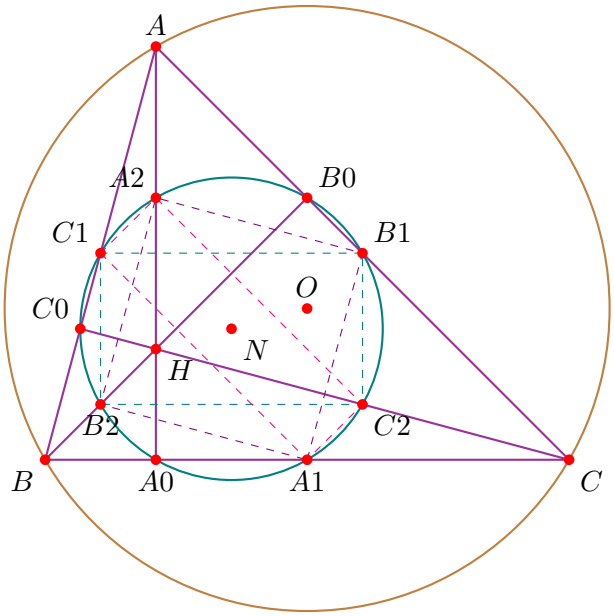
九点圆拥有许多有趣的几何性质，其中最重要的包括：

- **半径关系**：九点圆的半径恰好是三角形**外接圆半径的一半**。

- **圆心位置**：九点圆的圆心位于三角形的**欧拉线**上（即外心、重心、垂心三点所在的直线），并且是**垂心与外心连线的中点**。
- **费尔巴哈定理**：这是九点圆最著名的性质之一。它指出，九点圆与三角形的**内切圆**以及**三个旁切圆**都相切。

```cpp
\begin{tikzpicture}
  \tikzmath{
    % Define vertices of the triangle using polar coordinates
    \A{1} = 4 * cos(120); \A{2} = 4 * sin(120);
    \B{1} = 4 * cos(210); \B{2} = 4 * sin(210);
    \C{1} = 4 * cos(-30); \C{2} = 4 * sin(-30);
  }
  \tikzset{
    % --- Orthic Triangle Construction ---
    % Project vertices onto opposite sides to find altitude feet (Ai, Bi, Ci)
    plane/project={\B,\A,\C,\Ai},
    plane/project={\C,\B,\A,\Bi},
    plane/project={\A,\C,\B,\Ci},
    % --- Medial Triangle Construction ---
    % Find midpoints of the sides (Aj, Bj, Cj)
    plane/scale={\B,\C,0.5,\Aj},
    plane/scale={\C,\A,0.5,\Bj},
    plane/scale={\A,\B,0.5,\Cj},
    % --- Triangle Centers & Euler Line ---
    % Compute the Orthocenter (H) and Circumcenter (O)
    plane/orthocenter={\A,\B,\C,\H},
    plane/circumcenter={\A,\B,\C,\O},
    % The Nine-point center (N) is the midpoint of the Euler line segment OH
    plane/scale={\O,\H,0.5,\N},
    % --- Euler Points ---
    % Find midpoints between vertices and the orthocenter (Ak, Bk, Ck)
    plane/scale={\A,\H,0.5,\Ak},
    plane/scale={\B,\H,0.5,\Bk},
    plane/scale={\C,\H,0.5,\Ck},
    % --- Geometric Radii ---
    % Circumradius (Radius) and Nine-point radius (radius)
    plane/length={\O,\A,\Radius},
    plane/length={\N,\Ai,\radius},
  }

  % Convert calculated vectors/points to TikZ coordinates
  \coordinate (A) at (\A{1},\A{2});
  \coordinate (B) at (\B{1},\B{2});
  \coordinate (C) at (\C{1},\C{2});
```

```
  \coordinate (O) at (\O{1},\O{2});
  \coordinate (H) at (\H{1},\H{2});
  \coordinate (N) at (\N{1},\N{2});
  \coordinate (A0) at (\Ai{1},\Ai{2});
  \coordinate (B0) at (\Bi{1},\Bi{2});
  \coordinate (C0) at (\Ci{1},\Ci{2});
  \coordinate (A1) at (\Aj{1},\Aj{2});
  \coordinate (B1) at (\Bj{1},\Bj{2});
  \coordinate (C1) at (\Cj{1},\Cj{2});
  \coordinate (A2) at (\Ak{1},\Ak{2});
  \coordinate (B2) at (\Bk{1},\Bk{2});
  \coordinate (C2) at (\Ck{1},\Ck{2});

  % Draw the Circumcircle (brown) and the Nine-Point Circle (teal)
  \draw[thick,brown] (O) circle (\Radius);
  \draw[thick,teal] (N) circle (\radius);

  % Draw the reference triangle and its altitudes
  \draw[thick,violet!80] (A) -- (B) -- (C) -- cycle;
  \draw[thick,violet!80] (A) -- (A0) (B) -- (B0) (C) -- (C0);

  % Draw auxiliary quadrilaterals connecting midpoints and Euler points
  \draw[dashed,violet] (A1) -- (B1) -- (A2) -- (B2) -- cycle;
  \draw[dashed,teal] (B1) -- (C1) -- (B2) -- (C2) -- cycle;
  \draw[dashed,magenta] (C1) -- (A1) -- (C2) -- (A2) -- cycle;

  % Label all critical points
  \foreach \p/\placement in {A/above,B/below left,C/below right,
  A0/below,B0/above right,C0/above left,
  A1/below,B1/above right,C1/above left,
  A2/above left,B2/below,C2/below right,
  O/above,H/below right,N/below right}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {\small $\p$};
  }
\end{tikzpicture}
```


A
A2
B0
C1
B1
O
C0
N
H
B2
C2
B
A0
A1
C

第七章　圆锥曲线 Conics

7.1　齐次坐标与坐标变换

齐次坐标 (Homogeneous Coordinates) 是射影几何中的一个核心概念，它通过增加一个维度，将欧几里得空间中的几何问题转化为更具通用性的代数形式。

在计算机图形学、计算机视觉和几何计算中，齐次坐标是处理**平移变换**、**透视投影**以及**无穷远点**的利器。

7.1.1　齐次坐标

对于一个 n 维空间的点，齐次坐标用 $n+1$ 个分量来表示。

- **从笛卡尔到齐次：** 二维点 (x, y) 在齐次坐标下表示为 (u, v, w)，其中 $x = u/w, y = v/w$。
- **等价性：** 齐次坐标具有**缩放不变性**。对于任何非零常数 k，(u, v, w) 与 $(\lambda u, \lambda v, \lambda w)$ 表示欧几里得空间中的同一个点。

为什么要引入齐次坐标?

1. 统一变换矩阵 (Matrix Unification)

在欧几里得几何中，旋转和缩放可以用线性变换（矩阵乘法）表示，但**平移**必须通过向量加法实现。引入齐次坐标后，平移也可以写成矩阵乘法。以二维平移 (t_x, t_y) 为例：

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

这使得复杂的仿射变换可以串联成一个单一的矩阵乘积。

2. 表示无穷远点 (Points at Infinity)

在普通坐标系中，平行线永远不相交。但在射影几何中，平行线相交于 “无穷远点”。

- 当齐次分量 $w \to 0$ 时，点 $(u, v, 0)$ 就代表了方向为 (u, v) 的**无穷远点**。
- 这消除了几何运算中处理 “平行” 这种特殊情况的需要，使公式更加鲁棒。

在二次曲线与圆锥曲线中的应用，齐次坐标可以将其方程统一表示为二次型：

$$\boldsymbol{x}^T C \boldsymbol{x} = 0$$

其中 $\boldsymbol{x} = [x, y, w]^T$，$C$ 是一个 3×3 的对称矩阵。

- **切线计算**：若已知曲线上一点 $\boldsymbol{p}$，其切线方程直接为 $\boldsymbol{l} = C\boldsymbol{p}$。
- **退化情况**：通过检查矩阵 C 的行列式（$\det(C)$），可以轻松判断曲线是圆、椭圆、双曲线，还是退化成了直线对。

> 归一化：虽然 $(2, 4, 2)$ 是一个合法的齐次坐标，但在映射回二维空间前，通常需要将其除以 w 得到归一化形式 $(1, 2, 1)$。

7.1.2 向量叉乘

在计算机图形学和射影几何中，**齐次坐标（Homogeneous Coordinates）**下的叉乘（Cross Product）是一个非常强大的工具。它最核心的应用在于：**通过两个点的齐次坐标叉乘直接求出过这两点的直线方程，或者通过两条直线的方程叉乘求出它们的交点。**

7.1.2.1 点与直线的对偶性

在二维射影平面 P^2 中，点和直线都可以用一个三维向量来表示：

- **点 (Point)**： $\boldsymbol{p} = [x, y, w]^T$
- **直线 (Line)**： $\boldsymbol{l} = [a, b, c]^T$（对应方程 $ax + by + c = 0$）

点 $\boldsymbol{p}$ 在直线 $\boldsymbol{l}$ 上的充要条件是它们的点积为零：

$$\boldsymbol{p} \cdot \boldsymbol{l} = ax + by + cw = 0$$

1. 已知两点求直线

如果你有两个点 $\boldsymbol{p_1}$ 和 $\boldsymbol{p_2}$，过这两点的直线 $\boldsymbol{l}$ 必须同时满足 $\boldsymbol{p_1} \cdot \boldsymbol{l} = 0$ 和 $\boldsymbol{p_2} \cdot \boldsymbol{l} = 0$。根据向量空间性质，满足这个条件的向量 $\boldsymbol{l}$ 就是这两个点向量的**叉乘**：

$$\boldsymbol{l} = \boldsymbol{p_1} \times \boldsymbol{p_2}$$

2. 已知两线求交点

同理，如果你有两条直线 $\boldsymbol{l_1}$ 和 $\boldsymbol{l_2}$，它们的交点 $\boldsymbol{p}$ 必须同时在两条直线上。根据对偶性（Duality）：

$$\boldsymbol{p} = \boldsymbol{l_1} \times \boldsymbol{l_2}$$

7.1.2.2 叉乘计算步骤

假设有两个齐次向量 $\boldsymbol{a} = [a_1, a_2, a_3]$ 和 $\boldsymbol{b} = [b_1, b_2, b_3]$，它们的叉乘结果 $\boldsymbol{c}$ 计算如下：

$$\boldsymbol{c} = \boldsymbol{a} \times \boldsymbol{b} = \begin{vmatrix} \boldsymbol{i} & \boldsymbol{j} & \boldsymbol{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} = [a_2b_3 - a_3b_2,\ a_3b_1 - a_1b_3,\ a_1b_2 - a_2b_1]$$

齐次坐标统一处理无穷远点（平行线交点），这是齐次坐标最迷人的地方。

- **普通坐标**：两条平行线 $x = 1$ 和 $x = 2$ 没有交点。
- **齐次坐标**：
  - $\boldsymbol{l_1} = [1, 0, -1]$ （对应 $1x + 0y - 1 = 0$）
  - $\boldsymbol{l_2} = [1, 0, -2]$ （对应 $1x + 0y - 2 = 0$）
  - $\boldsymbol{p} = \boldsymbol{l_1} \times \boldsymbol{l_2} = [0, 1, 0]$ 结果 $[0, 1, 0]$ 表示这是一个**方向向量**（$w = 0$），即指向 y 轴正方向的**无穷远点**。这完美解释了平行线在无穷远处相交的概念。

7.1.2.3 截距点

在解析几何中，直线与坐标轴的交点统称为截距点。根据具体的坐标轴，可以细分为以下两种情况：

1. 与坐标轴交点的名称 x 轴截距点：直线与 x 轴的交点。该点的纵坐标始终为 0，坐标形式通常为：

$$[0,1,0]^T \times \boldsymbol{l}$$

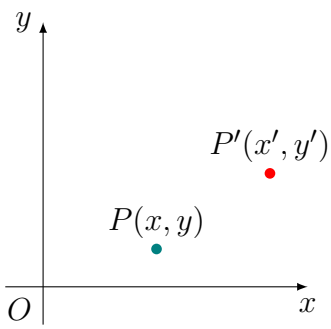
2. y 轴截距点：直线与 y 轴的交点。该点的横坐标始终为 0，坐标形式通常为：

$$[1,0,0]^T \times \boldsymbol{l}$$

> 在给定直线方程，任取直线上两点时，可以采用求截距点的方法。

7.1.3 坐标变换 Transformations of Coordinates

7.1.3.1 坐标平移 Translation of Coordinates

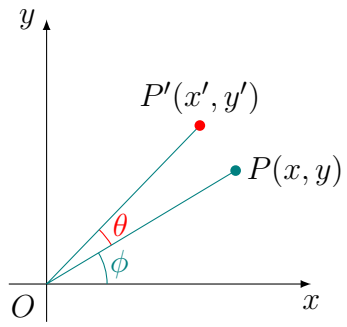


$$\begin{cases} x' = x + t_x \\ y' = y + t_y \end{cases}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

7.1.3.2 坐标旋转 Rotation of Coordinates



$$\begin{cases} x = r\cos\phi \\ y = r\sin\phi \end{cases}$$

$$\begin{cases} x' = r\cos(\phi + \theta) \\ y' = r\sin(\phi + \theta) \end{cases}$$

$$\begin{cases} x' = x\cos\theta - y\sin\theta \\ y' = y\cos\theta + x\sin\theta \end{cases}$$

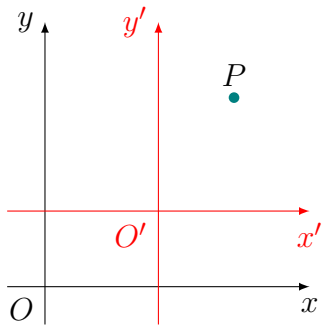
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

7.1.4 坐标轴变换 Transformations of Axes

下面讨论坐标轴平移和旋转后坐标的变化。

7.1.4.1 坐标轴平移 Translation of Axes

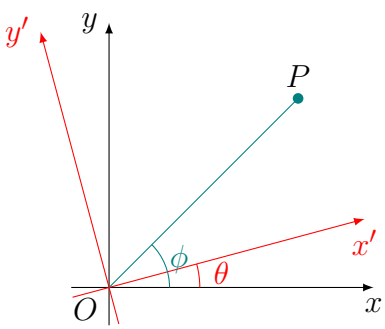


$$\begin{cases} x' = x - t_x \\ y' = y - t_y \end{cases}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

7.1.4.2 坐标轴旋转 Rotation of Axes



$$\begin{cases} x = r\cos\phi \\ y = r\sin\phi \end{cases}$$

$$\begin{cases} x' = r\cos(\phi - \theta) \\ y' = r\sin(\phi - \theta) \end{cases}$$

$$\begin{cases} x' = x\cos\theta + y\sin\theta \\ y' = y\cos\theta - x\sin\theta \end{cases}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

7.1.5 齐次坐标在 cartes 包中的表示

在 cartes 包中，使用矩阵来表示齐次坐标，如：

```
\tikzmath{
  % 定义点
  \P{1,1} = 1.5; \P{2,1} = 5.0; \P{3,1} = 1.0;
  \Q{1,1} = 0.5; \Q{2,1} = 3.0; \Q{3,1} = 1.0;
  % 定义直线 2 x - y + 3 = 0
  \l{1,1} = 2; \l{2,1} = -1; \l{3,1} = 3;
}
```

7.1.6 语法形式

Syntax

- conics/homogenize={\P,\Q}: 将二维笛卡尔坐标齐次化，\P 为笛卡尔坐标，\Q 为齐次坐标
- conics/dehomogenize={\P,\Q}: 将齐次坐标去齐次化，\P 为齐次坐标，\Q 为笛卡尔坐标
- conicss/distance={\P,\Q,\d}: 求齐次坐标下的两点 P, Q 的距离
- conics/cross={\U,\V,\W}: 向量叉乘，\U,\V,\W 为点或直线的齐次坐标

- conics/x-intercept={\l,\P}: 直线 l 在 x 轴上的截距点（齐次坐标）
- conics/y-intercept={\l,\P}: 直线 l 在 y 轴上的截距点（齐次坐标）
- conics/translation matrix={\xshift,\yshift,\T}: 构造平移矩阵
- conics/rotation matrix={\ang,\R}: 构造旋转矩阵，\ang 是旋转角度（deg）

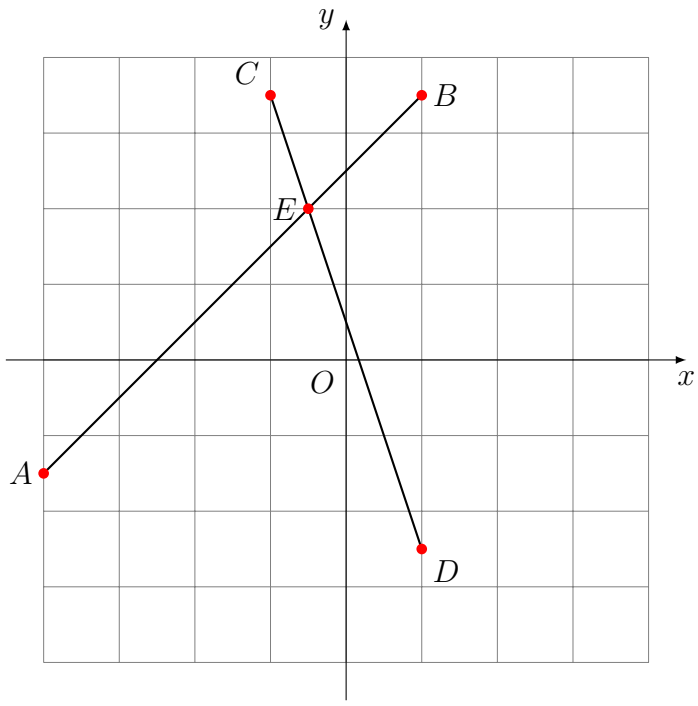
无穷远点无法去齐次化。

7.1.7 示例：齐次坐标下两直线的交点

参考 示例：笛卡尔坐标下两直线的交点[p. 65] 。

```
\begin{tikzpicture}
  \tikzmath{
    % 定义 A, B, C, D 的齐次坐标
    \A{1,1} = -4; \A{2,1} = -1.5; \A{3,1} = 1.0;
    \B{1,1} = 1; \B{2,1} = 3.5; \B{3,1} = 1.0;
    \C{1,1} = -1; \C{2,1} = 3.5; \C{3,1} = 1.0;
    \D{1,1} = 1; \D{2,1} = -2.5; \D{3,1} = 1.0;
  }
  \tikzset{
    % 计算过 A, B 的直线与过 C, D 的直线
    conics/cross={\A,\B,\Lab},
    conics/cross={\C,\D,\Lcd},
    % 计算 Lab, Lcd 的交点
    conics/cross={\Lab,\Lcd,\E},
    % 取齐次化
    conics/dehomogenize={\A,\AA},
    conics/dehomogenize={\B,\BB},
    conics/dehomogenize={\C,\CC},
    conics/dehomogenize={\D,\DD},
    conics/dehomogenize={\E,\EE},
  }
  \axes[grid] (-4:4,-4:4);
  \coordinate (A) at (\AA{1},\AA{2});
  \coordinate (B) at (\BB{1},\BB{2});
  \coordinate (C) at (\CC{1},\CC{2});
  \coordinate (D) at (\DD{1},\DD{2});
  \coordinate (E) at (\EE{1},\EE{2});
  \draw[thick] (A) -- (B) (C) -- (D);

```

```
  \foreach \p/\placement in {A/left,B/right,
  C/above left,D/below right,E/left}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.2 圆锥曲线方程的一般形式

7.2.1 圆锥曲线的系数矩阵

设二次曲线的方程为:

$$ax^2 + bxy + cy^2 + dx + ey + f = 0$$

记

$$C = \begin{bmatrix} a & b/2 & d/2 \\ b/2 & c & e/2 \\ d/2 & e/2 & f \end{bmatrix}$$

$$\boldsymbol{x} = [x, y, 1]^T$$

则:

$$\boldsymbol{x}^T C \boldsymbol{x} = 0$$

C 称为圆锥曲线的系数矩阵。

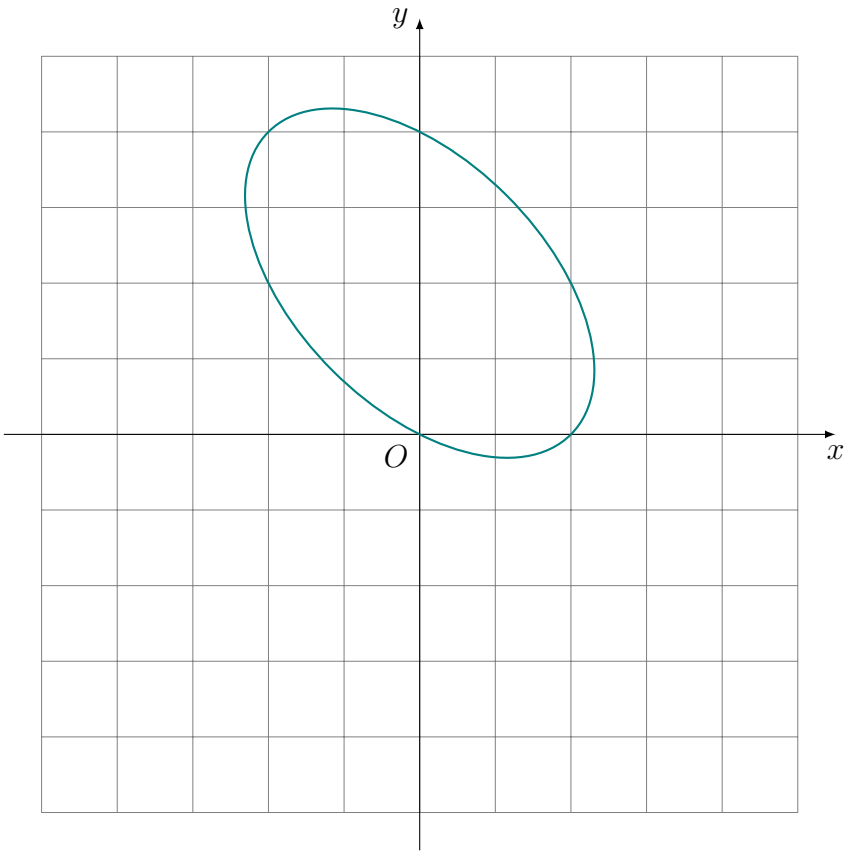
7.2.2 语法形式

Syntax

- `conics/define/quadratic={\coefs,\C}`: 由二次曲线方程系数计算系数矩阵
  - `\coefs`: 二次曲线方程系数 a, b, c, d, e, f（数组）
  - `\C`: 返回的圆锥曲线的系数矩阵 C

7.2.3 示例：由方程系数确定的二次曲线

```
\begin{tikzpicture}
  \tikzmath{
    \coefs{1} = 1; \coefs{2} = 1; \coefs{3} = 1;
    \coefs{4} = -2; \coefs{5} = -4; \coefs{6} = 0;
  }
  % \tikzset{
  %   conics/define/quadratic={\coefs,\A},
  % }

  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal,thick,conics/define/quadratic={\coefs,\A}] (\A);
\end{tikzpicture}
```



7.3 由焦点-准线定义推导圆锥曲线的系数矩阵

7.3.1 焦点-准线定义圆锥曲线

设焦点 F 在齐次坐标为 $\boldsymbol{f} = [f_x, f_y, 1]^T$。
设准线为一般直线 $l : n_x x + n_y y + n_z = 0$，在齐次坐标下可记为法向量 $\boldsymbol{n} = [n_x, n_y, n_z]^T$，满足对于直线上的点 $\boldsymbol{x}$ 有 $\boldsymbol{n}^T\boldsymbol{x} = 0$。

根据圆锥曲线的定义：

$$\frac{\mathrm{dist}(\boldsymbol{x}, F)}{\mathrm{dist}(\boldsymbol{x}, l)} = e, \quad 0 < e < 1$$

$\mathrm{dist}(\boldsymbol{x}, l)$ 是点到直线的**有向距离**（或绝对值距离），平方后可避免符号。

点到焦点的距离平方：

$$d_F^2 = (x - f_x)^2 + (y - f_y)^2 = \boldsymbol{x}^T P \boldsymbol{x}$$

其中

$$P = \begin{bmatrix} 1 & 0 & -f_x \\ 0 & 1 & -f_y \\ -f_x & -f_y & f_x^2 + f_y^2 \end{bmatrix}.$$

将 $\boldsymbol{x} = [x, y, 1]^T$ 代入后，展开即得。

点到直线 l 的距离平方(有向):直线方程 $n_x x + n_y y + n_z = 0$,法向量 $\boldsymbol{n} = [n_x, n_y, n_z]^T$。有向距离

$$d_l = \frac{\boldsymbol{n}^T \boldsymbol{x}}{\sqrt{n_x^2 + n_y^2}}.$$

平方得:

$$d_l^2 = \frac{(\boldsymbol{n}^T \boldsymbol{x})^2}{n_x^2 + n_y^2} = \boldsymbol{x}^T Q \boldsymbol{x}$$

其中

$$Q = \frac{1}{n_x^2 + n_y^2} \boldsymbol{n}\boldsymbol{n}^T.$$

注意，$\boldsymbol{n}\boldsymbol{n}^T$ 是 3×3 矩阵，注意它秩为 1，且 $\boldsymbol{x}^T(\boldsymbol{n}\boldsymbol{n}^T)\boldsymbol{x} = (\boldsymbol{n}^T \boldsymbol{x})^2$。

根据圆锥曲线的定义平方并移项:

$$d_F^2 = e^2 d_l^2$$

即

$$\boldsymbol{x}^T P \boldsymbol{x} = e^2 \cdot \boldsymbol{x}^T Q \boldsymbol{x}.$$

$$\boldsymbol{x}^T (P - e^2 Q) \boldsymbol{x} = 0.$$

于是系数矩阵

$$C = P - e^2 Q.$$

其中 P 仅依赖于焦点 F，Q 仅依赖于准线 l（且已归一化使分母 $n_x^2 + n_y^2$ 融入）。

重要说明

- C 是一个对称矩阵，它的二次型 $\boldsymbol{x}^T C \boldsymbol{x} = 0$ 给出了根据圆锥曲线的次方程。
- 此 C 是**直接**由几何参数（F、l、e）通过矩阵减法得到的，无需预先假设根据圆锥曲线的齐或中心在原点。
- 注意这里 C 不一定是对角矩阵，它包含了旋转信息（来自斜准线和焦点位置）。
- 得到的根据圆锥曲线的定是标准位置，可能是任意位置、任意方向的根据圆锥曲线的

7.3.2 语法形式

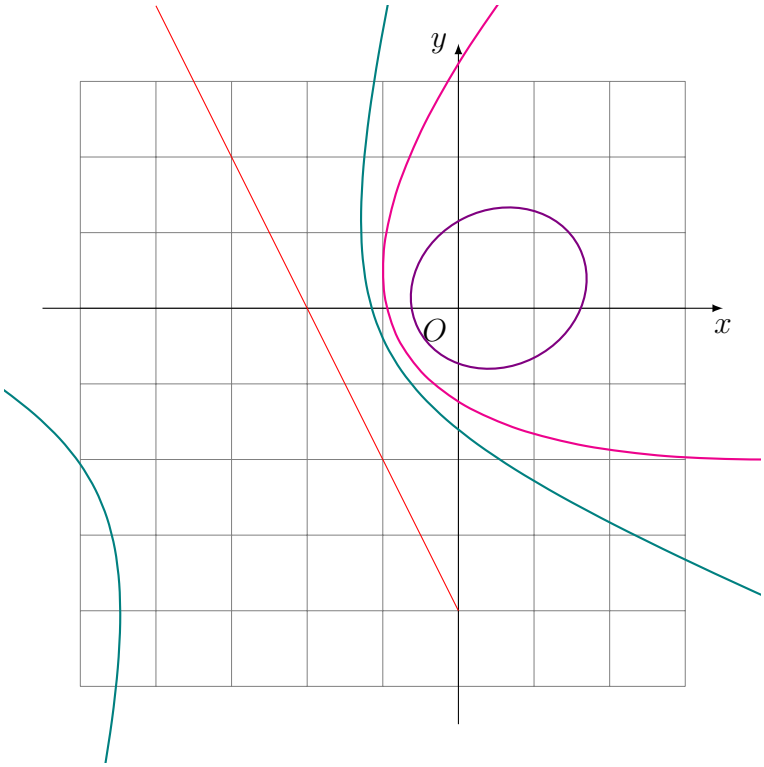
Syntax

- `conics/define/focus-directrix={\F,\D,\e,\C}`: 由焦点-准线定义推导圆锥曲线的系数矩阵
  - `\F`: 焦点的齐次坐标（列向量）
  - `\D`: 准线的齐次坐标（列向量）
  - `\e`: 离心率
  - `\C`: 返回的圆锥曲线的系数矩阵 C

7.3.3 示例：椭圆、抛物线和双曲线的焦点-准线定义

```
\begin{tikzpicture}
  \clip (-6,-6) rectangle (4,4); % 剪裁区域
  \tikzmath{
    % focus
    \F{1,1} = 0.0; \F{2,1} = 0.0; \F{3,1} = 1.0;
    % directrix: 2 x + y + 4 = 0
    \D{1,1} = 2.0; \D{2,1} = 1.0; \D{3,1} = 4.0;
  }
  \tikzset{
    conics/define/focus-directrix={\F,\D,0.5,\Ca},
    conics/define/focus-directrix={\F,\D,1.0,\Cb},
    conics/define/focus-directrix={\F,\D,1.5,\Cc},
  }

```

```
  \axes[grid] (-5:3,-5:3);
  \conic[draw=violet,thick] (\Ca);
  \conic[draw=magenta,thick] (\Cb);
  \conic[draw=teal,thick] (\Cc);
  \segment[draw=red,clip=(-4:4,-4:4)] (\D);

\end{tikzpicture}
```



7.4 由焦点-距离定义推导椭圆/双曲线的系数矩阵

7.4.1 焦点-距离定义圆锥曲线

先推导靠近 $F_1(x_1, y_1)$ 的准线方程

核心参数定义：

- 中心 O：两焦点的中点 $O = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2}\right)$。
- 焦距的一半 c：$c = \frac{1}{2}\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$。
- 离心率 e：$e = \frac{c}{a}$。

准线与长轴 F_1F_2 垂直，因此准线的法向量可以取为 $\overline{F_1F_2} = (x_2 - x_1, y_2 - y_1)$。

靠近 F_1 的准线与长轴的交点 P，将 F_2 绕中心 O 旋转 90° 得到 Q，将 PQ 按长轴方向平移 $-\dfrac{a^2}{c}$ 就得到准线，平移向量 $\boldsymbol{s}$ 为：

$$\boldsymbol{s} = -\frac{a^2}{c}\frac{\overline{F_1F_2}}{|\overline{F_1F_2}|} = -\frac{1}{2e^2}\overline{F_1F_2}$$

准线的方程可由 P, Q 两点的齐次坐标叉乘得到：

$$\boldsymbol{d} = \boldsymbol{P} \times \boldsymbol{Q}$$

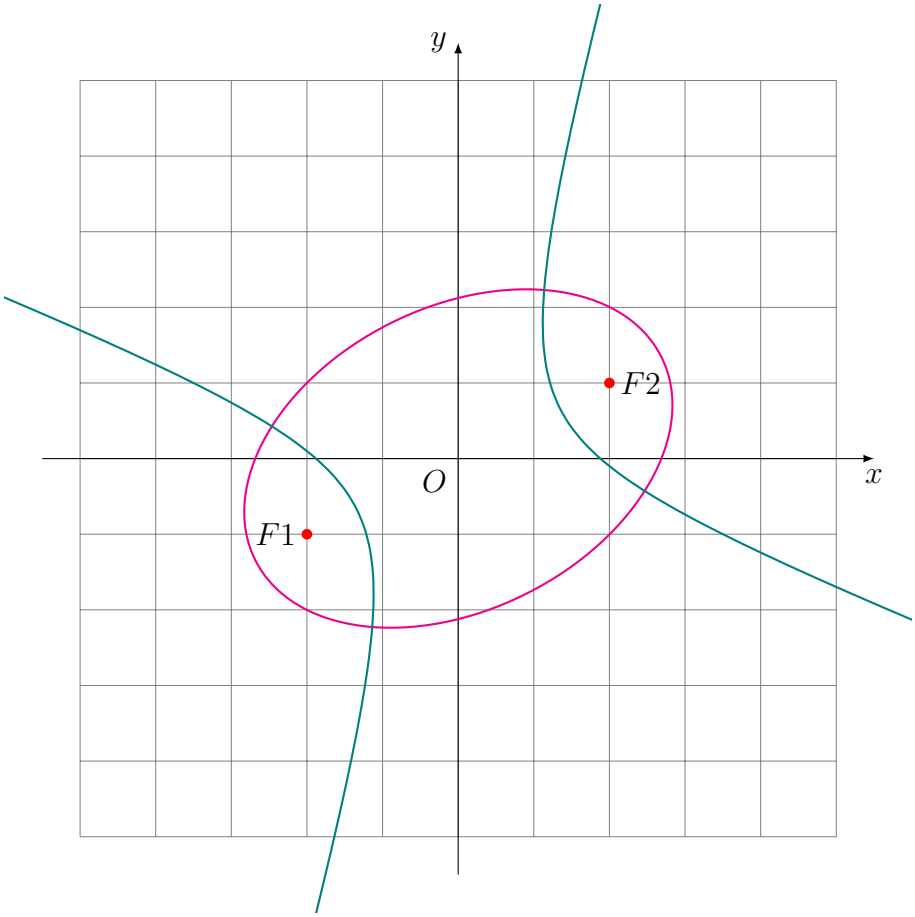
7.4.2 语法形式

Syntax

- `conics/define/foci={\Fa,\Fb,\a,\C}`: 由焦点-准线定义推导圆锥曲线的系数矩阵
  - `\Fa,\Fb`: 焦点的齐次坐标（列向量），如果 F_a, F_b 重合，返回圆的系数矩阵
  - `\a`: 半长轴长 Semi-major axis length，如果 $a > c$，返回椭圆的系数矩阵；如果 $a < c$，返回双曲线的系数矩阵；如果 $a = c$，报错（其中 $2c = |\overline{F_aF_b}|$）。
  - `\C`: 返回的圆锥曲线的系数矩阵
- `conics/define/ellipse/foci-point={\Fa,\Fb,\P,\C}`: 由焦点和曲线上一点确定一个椭圆
  - `\Fa,\Fb`: 焦点的齐次坐标（列向量），如果 F_a, F_b 重合，返回圆的系数矩阵
  - `\P`: 椭圆上的一点
  - `\C`: 返回的圆锥曲线的系数矩阵
- `conics/define/hyperbola/foci-point={\Fa,\Fb,\P,\C}`: 由焦点和曲线上一点确定一个双曲线
  - `\Fa,\Fb`: 焦点的齐次坐标（列向量）
  - `\P`: 双曲线上的一点
  - `\C`: 返回的圆锥曲线的系数矩阵

7.4.3 示例 1：椭圆和双曲线的双焦点定义

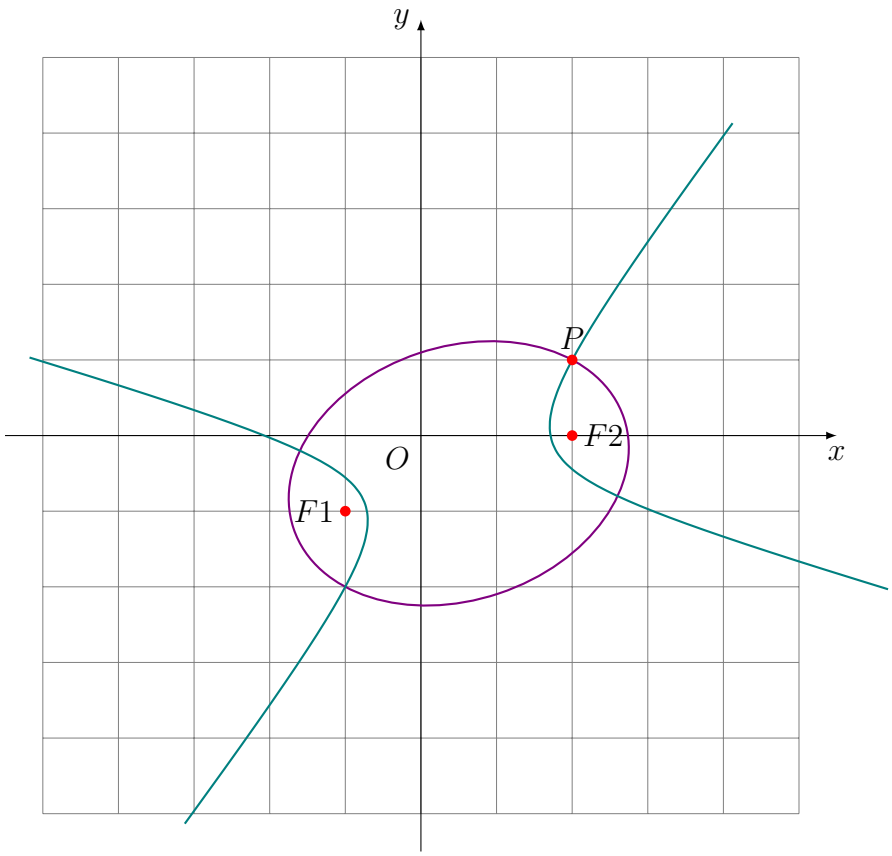
```latex
\begin{tikzpicture}
  \clip (-6,-6) rectangle (6,6); % 剪裁区域
  \tikzmath{
    % focus
    \Fa{1,1} = -2.0; \Fa{2,1} = -1.0; \Fa{3,1} = 1.0;
    \Fb{1,1} = 2.0; \Fb{2,1} = 1.0; \Fb{3,1} = 1.0;
  }
  \tikzset{
    conics/define/foci={\Fa,\Fb,3,\Ca},
    conics/define/foci={\Fa,\Fb,1.5,\Cb},
  }

  \axes[grid] (-5:5,-5:5);
  \coordinate (F1) at (\Fa{1,1},\Fa{2,1});
  \coordinate (F2) at (\Fb{1,1},\Fb{2,1});

  \conic[draw=magenta,thick] (\Ca);
  \conic[draw=teal,thick,domain=-2:2] (\Cb);

  \foreach \p/\placement in {F1/left,F2/right}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.4.4 示例 2：由焦点和曲线上一点确定椭圆或双曲线

```
\begin{tikzpicture}
  \tikzmath{
    \P{1,1} = 2; \P{2,1} = 1; \P{3,1} = 1;
    \A{1,1} = -1; \A{2,1} = -1; \A{3,1} = 1;
    \B{1,1} = 2; \B{2,1} = 0; \B{3,1} = 1;
  }
  \tikzset{
    conics/define/ellipse/foci-point={\A,\B,\P,\Ca},
    conics/define/hyperbola/foci-point={\A,\B,\P,\Cb},
  }

  \coordinate (F1) at (\A{1,1},\A{2,1});
  \coordinate (F2) at (\B{1,1},\B{2,1});
  \coordinate (P) at (\P{1,1},\P{2,1});

  \axes[grid] (-5:5,-5:5);
  \conic[draw=violet,thick] (\Ca);
  \conic[draw=teal,thick] (\Cb);
```

```tikz

  \foreach \p/\placement in {F1/left,F2/right,P/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.5 由圆心-半径或三点确定圆的方程

7.5.1 三点确定的圆

假设这三个点不共线（否则无法确定一个圆），则存在唯一一个圆通过这三点。

用齐次坐标直接表达（射影几何方法）

若希望保留齐次形式，可将圆的方程写成**齐次二次曲线**的形式：

$$x^2 + y^2 + axw + byw + cw^2 = 0,$$

其中 $(x:y:w)$ 是齐次坐标，且该方程在仿射部分（$w \neq 0$）对应于上述笛卡尔圆方程。

将三个齐次点 (x_i, y_i, w_i) 代入：

$$x_i^2 + y_i^2 + ax_iw_i + by_iw_i + cw_i^2 = 0, \quad i = 1, 2, 3.$$

同样得到关于 a, b, c 的线性方程组：

$$\begin{bmatrix} x_1w_1 & y_1w_1 & w_1^2 \\ x_2w_2 & y_2w_2 & w_2^2 \\ x_3w_3 & y_3w_3 & w_3^2 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = - \begin{bmatrix} x_1^2 + y_1^2 \\ x_2^2 + y_2^2 \\ x_3^2 + y_3^2 \end{bmatrix}$$

解出 a, b, c，或等价地用上述行列式形式直接写出。

7.5.2 语法形式

Syntax

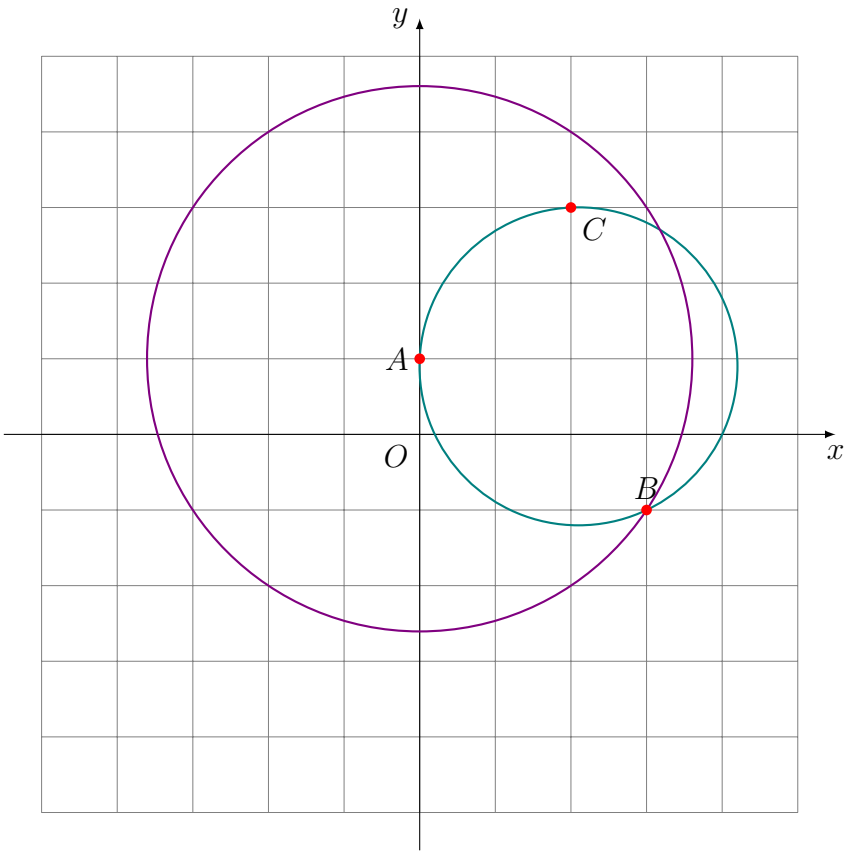
- `conics/define/circle={\A,\B,\C,\Circle}`: 求过 A, B, C 三点的圆的系数矩阵
  - `\A,\B,\C`: 三点的齐次坐标（列向量 3×1）
  - `\Circle`: 圆的系数矩阵
- `conics/define/circle={\O,\r,\Circle}`: 求以 O 为圆心，r 为半径的圆的系数矩阵
  - `\O`: 圆心的齐次坐标（列向量 3×1）
  - `\r`: 半径
  - `\Circle`: 圆的系数矩阵

> `conics/define/circle` 根据参数的个数选择不同的构造方法，在 LaTeX 或 TikZ 中，要实现根据参数类型重载是比较困难的。

7.5.3 示例：圆的定义

```
\tikzmath{
  \A{1,1} = 0; \A{2,1} = 1; \A{3,1} = 1;
```

```
  \B{1,1} = 3; \B{2,1} = -1; \B{3,1} = 1;
  \C{1,1} = 2; \C{2,1} = 3; \C{3,1} = 1;
}
\tikzset{
  % 过 A,B,C 三点的圆
  conics/define/circle={\A,\B,\C,\Ca},
  % 以 A 为圆心 AB 为半径的圆
  conics/distance={\A,\B,\d},
  conics/define/circle={\A,\d,\Cb},
}

\begin{tikzpicture}
  \coordinate (A) at (\A{1,1},\A{2,1});
  \coordinate (B) at (\B{1,1},\B{2,1});
  \coordinate (C) at (\C{1,1},\C{2,1});
  \axes[grid] (-5:5,-5:5);
  \conic[thick,draw=teal] (\Ca);
  \conic[thick,draw=violet] (\Cb);

  \foreach \p/\placement in {A/left,B/above,C/below right}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.6 化简一般圆锥曲线为标准方程

当我们将二次曲线（圆锥曲线）表示为矩阵形式时，旋转变换的操作非常优雅。

二次曲线的矩阵表示:

$$\boldsymbol{x}^T C \boldsymbol{x} = 0$$

7.6.1 圆锥曲线的旋转变换

假设我们将坐标系**逆时针旋转** θ **角**。如果原始点为 $\boldsymbol{x}$，旋转后的点为 $\boldsymbol{x}'$，因此，

$$\boldsymbol{x} = R\boldsymbol{x}'$$

于是，

$$\boldsymbol{x}' = R^{-1}\boldsymbol{x}$$

其中 R 是齐次坐标下的旋转矩阵：

$$R = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

将 $\boldsymbol{x} = R\boldsymbol{x}'$ 代入原方程 $\boldsymbol{x}^T C\boldsymbol{x} = 0$：

$$(R\boldsymbol{x}')^T C(R\boldsymbol{x}') = 0$$
$$(\boldsymbol{x}')^T (R^T CR)\boldsymbol{x}' = 0$$

因此，旋转后的新系数矩阵 C' 为：

$$C' = R^T CR$$

如果你展开后的具体系数，计算 $C' = R^T CR$ 后，新的系数 $A', B', C', \ldots$ 如下：

- $A' = A\cos^2\theta + C\sin^2\theta + B\sin\theta\cos\theta$
- $B' = B(\cos^2\theta - \sin^2\theta) - 2(A - C)\sin\theta\cos\theta$
- $C' = A\sin^2\theta + C\cos^2\theta - B\sin\theta\cos\theta$
- $D' = D\cos\theta + E\sin\theta$
- $E' = -D\sin\theta + E\cos\theta$
- $F' = F$

在旋转变换下，矩阵 C 的某些属性是不变的（即**不变量**），这常用于识别圆锥曲线的类型：

1. **行列式不变**：$\det(C')$ 恒等于 $\det(C)$。
2. **迹（Trace）不变**（仅针对左上角 2×2 子矩阵）：$A' + C' = A + C$。
3. **判别式不变**：$(B')^2 - 4A'C' = B^2 - 4AC$。

若已知系数矩阵 C：

- **旋转曲线**：新矩阵 $C' = (R^{-1})^T CR^{-1}$（如果你是将曲线本身旋转）。
- **旋转坐标轴**：新矩阵 $C' = R^T CR$（如果你是站在旋转后的视角看曲线）。

通常在几何变换中，$C' = R^T C R$ 是最常用的表达形式。

7.6.2 圆锥曲线的平移变换

对于平移变换，处理方式与旋转完全一致：通过**矩阵的合同变换**来更新系数矩阵。

假设平移向量为 $\mathbf{t} = [t_x, t_y]^T$，即新的坐标系（或点）满足：

$$\boldsymbol{x} = T\boldsymbol{x}'$$

其中 T 是齐次坐标下的平移矩阵：

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

将 $\boldsymbol{x} = T\boldsymbol{x}'$ 代入原方程 $\boldsymbol{x}^T C \boldsymbol{x} = 0$，得到新方程：

$$(T\boldsymbol{x}')^T C (T\boldsymbol{x}') = 0 \implies (\boldsymbol{x}')^T (T^T C T) \boldsymbol{x}' = 0$$

因此，平移后的新系数矩阵 C' 为：

$$C' = T^T C T$$

我们将上式展开后，可以得到平移后的新系数（以 $A', B', \ldots$ 表示）：

系数	变换后的值	说明
A'	A	**二次项系数不变**
B'	B	**交叉项系数不变**
C'	C	**二次项系数不变**
D'	$2At_x + Bt_y + D$	受平移影响
E'	$Bt_x + 2Ct_y + E$	受平移影响
F'	$At_x^2 + Bt_x t_y + Ct_y^2 + Dt_x + Et_y + F$	本质是原方程在 (t_x, t_y) 处的值

重要结论

1. **二次项的稳定性**：平移**不会改变**二次项系数 (A, B, C)。这意味着圆锥曲线的**形状**（长短轴、离心率等）和**开口方向**在平移过程中保持不变。只有它的**位置**（一次项 D, E）和相对于原点的**截距**（常数项 F）会改变。

2. **配方消去一次项**：在解析几何中，我们经常通过平移来消去 D 和 E，从而找到圆锥曲线的**中心**。令 $D' = 0$ 和 $E' = 0$，求解出的 (t_x, t_y) 即为中心坐标：

$$\begin{cases} 2At_x + Bt_y + D = 0 \\ Bt_x + 2Ct_y + E = 0 \end{cases}$$

   这实际上是在求解梯度为零的点。

3. **不变量**：由于 T 的行列式 $\det(T) = 1$，所以平移变换依然保持 $\det(C)$ 不变。

- **旋转**：$C' = R^T C R$（改变 A, B, C, D, E）。
- **平移**：$C' = T^T C T$（仅改变 D, E, F）。
- **复合（先旋再平）**：$C' = (TR)^T C (TR) = R^T T^T C T R$。

这种矩阵表示法的最大优势在于：无论变换多复杂，你只需要把所有的基本变换矩阵乘在一起，最后对 C 做一次合同变换即可。

7.6.3 圆锥曲线的中心

设 A 是矩阵 C 的代数余子式, 如果 $A_{33} = 0$, 则曲线是抛物线, 其中心在无穷远处; 否则曲线的中心是 $\left(\dfrac{A_{31}}{A_{33}}, \dfrac{A_{32}}{A_{33}}\right)$。

因为抛物线是没有中心的, 对于任意圆锥曲线, 可以先进行旋转, 消除交叉项 $\boldsymbol{x}y$, 然后再处理。

7.6.4 化简步骤

旋转坐标系

首先是旋转, 消除交叉项 xy, 旋转角度为 $\theta(-\pi/4 \le \theta \le \pi/4)$, 且:

$$\cot 2\theta = \frac{1 - \tan^2\theta}{2\tan\theta} = \frac{a - c}{b}$$

令 $\tau = \cot 2\theta$, 则:

$$\tan^2\theta + 2\tau\tan\theta - 1 = 0$$

求解 $t = \tan\theta$，我们需要解 $t^2 + 2\tau t - 1 = 0$。为了保证数值稳定性（避免相近数相减导致精度丢失），我们选择绝对值较小的那个根：

$$t = \frac{\operatorname{sgn}(\tau)}{|\tau| + \sqrt{1+\tau^2}}$$

或者写作更常见的形式（若 $\tau \geq 0$，取负根；若 $\tau < 0$，取正根，以确保 $|\theta| \leq \pi/4$）：

$$t = \begin{cases} \dfrac{1}{\tau + \sqrt{1+\tau^2}} & \text{if } \tau \geq 0 \\ \dfrac{-1}{-\tau + \sqrt{1+\tau^2}} & \text{if } \tau < 0 \end{cases}$$

这可以统一写为：

$$t = \frac{1}{\tau + \operatorname{sgn}(\tau)\sqrt{1+\tau^2}}$$

> 这里选择小根是为了让旋转角度 θ 落在 $[-\pi/4, \pi/4]$ 之间，保证 $c \geq s$，从而提高稳定性。

$$\begin{aligned} t &= \tan\theta \\ c = \cos\theta &= \frac{1}{\sqrt{t^2+1}} \\ s = \sin\theta &= \tan\theta\cos\theta = tc \end{aligned}$$

这样无需进行三角函数运算。

构造旋转矩阵：

$$R = \begin{bmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

计算 $C' = R^T C R$ 就得到新的系数矩阵。

平移坐标系

然后根据系数矩阵 C' 判断是否为抛物线, 即 $C'_{11} = 0$ 或 $C'_{22} = 0$。如果是抛物线, 则将坐标系平移至顶点; 否则将坐标系平移至中心。

若 $C'_{11} = 0$ 且 $C'_{22} \neq 0$, 则抛物线简化为:

$$cy^2 + d\boldsymbol{x} + ey + f = 0$$

其中:

$$\begin{aligned} c &= C'_{22} \\ d &= 2C'_{13} \\ e &= 2C'_{23} \\ f &= C'_{33} \end{aligned}$$

则抛物线的顶点为: $\left(-\dfrac{e}{2c}, \dfrac{e^2 - 4cf}{4cd}\right)$。

对于椭圆和双曲线, 计算其代数余子式 A, 中心为: $\left(\dfrac{A_{31}}{A_{33}}, \dfrac{A_{32}}{A_{33}}\right)$。

构造平移矩阵:

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

其中

$$\begin{aligned} t_x &= \frac{A_{31}}{A_{33}} \\ t_y &= \frac{A_{32}}{A_{33}} \end{aligned}$$

计算 $C'' = T^T C' T$ 就得到化为标准形式的系数矩阵。

7.6.5 语法形式

Syntax

- `conics/rotate={\CO,\CR,\R,\ang}`: 将含 xy 交叉项的二次曲线系数矩阵通过旋转变换消除交叉项
  - `\CO`: 含 xy 交叉项的二次曲线系数矩阵
  - `\CR`: 消除交叉项之后的二次曲线系数矩阵
  - `\R`: 旋转矩阵
  - `\ang`: 旋转角度
- `conics/translate={\CR,\CT,\T,\shift}`: 将已消除 xy 交叉项的二次曲线系数矩阵通过平移变换成标准形
  - `\CR`: 消除交叉项之后的二次曲线系数矩阵
  - `\CT`: 二次曲线标准形的系数矩阵
  - `\T`: 平移矩阵
  - `\shif`: `\shif{1}` 是 x 方向平移量，`\shif{2}` 是 y 方向平移量

7.7 五点确定的圆锥曲线

通过给定的 5 个点确定一条圆锥曲线（Conic Section）是射影几何中的一个经典问题。

由于方程两边可以同时除以一个非零常数，这个方程实际上只有 5 个独立的自由度。因此，**一般情况下，平面上任意 5 个点可以唯一确定一条圆锥曲线。**

7.7.1 代数解法：齐次线性方程组

设二次曲线的方程为:

$$ax^2 + bxy + cy^2 + dxw + eyw + fw^2 = 0$$

将 $(x_i, y_i, w_i)(i = 1, 2, \cdots, 5)$ 代入上式, 并组成线性方程组. 由于 $(a, b, c, d, e, f) \neq \mathbf{0}$, 该方程组的系数矩阵的行列式为 0, 即:

$$
\begin{vmatrix}
x^2 & xy & y^2 & xw & yw & w^2 \\
x_1^2 & x_1y_1 & y_1^2 & x_1w_1 & y_1w_1 & w_1^2 \\
x_2^2 & x_2y_2 & y_2^2 & x_2w_2 & y_2w_2 & w_2^2 \\
x_3^2 & x_3y_3 & y_3^2 & x_3w_3 & y_3w_3 & w_3^2 \\
x_4^2 & x_4y_4 & y_4^2 & x_4w_4 & y_4w_4 & w_4^2 \\
x_5^2 & x_5y_5 & y_5^2 & x_5w_5 & y_5w_5 & w_5^2
\end{vmatrix} = 0
$$

化简该行列式即得到二次曲线的方程。

7.7.2 几何构造法：线性束（Pencil of Conics）

根据射影几何理论（例如 Richter-Gebert 的著作），利用**圆锥曲线束**的概念会更直观：

1. **取前 4 个点**：假设为 A, B, C, D。
2. **构造两条退化圆锥曲线**（即直线对）：
   - L_1：由直线 (AB) 和 (CD) 组成。
   - L_2：由直线 (AC) 和 (BD) 组成。
3. **构造曲线束**：过这 4 个点的所有圆锥曲线可以表示为 $L_1 + \lambda L_2 = 0$。
4. **代入第 5 个点**：将点 E 的坐标代入方程，解出唯一的 λ，从而得到过这 5 点的特定曲线。

特殊情况与退化

虽然通常解是唯一的，但点的位置关系会导致不同的结果：

- **唯一非退化曲线**：若任意 3 点不共线，且 5 点不在同一条二次曲线上，则得到标准的椭圆、双曲线或抛物线。
- **退化曲线（直线对）**：如果有 3 点共线（例如 A, B, C 在直线上），那么唯一的圆锥曲线将退化为两条直线：一条是过这 3 点的直线，另一条是连接剩余两点的直线。
- **无唯一解**：如果有 4 点共线，则过这 5 点的圆锥曲线不唯一（包含该直线与过第 5 点的任意直线的组合）。

Richter-Gebert 的著作中仅用 $\mathbf{l}_1\mathbf{l}_2^T$ 构造一个基，仅用 $\mathbf{m}_1\mathbf{m}_2^T$ 构造另一个基，求 $M = C_1 + \lambda C_2$，以 $M + M^T$ 为结果。实际上是在利用线性代数中 **“对称化算子”** 的性质，把非对称的矩阵运算转化为了合法的圆锥曲线矩阵。

简单来说，$(\mathbf{l}_1\mathbf{l}_2^T + \mathbf{l}_2\mathbf{l}_1^T)$ 和 $(\mathbf{l}_1\mathbf{l}_2^T + \mathbf{l}_2\mathbf{l}_1^T)$ 的线性组合，完全等同于先组合再对称化。

为什么 C_1 和 C_2 的线性组合一定包含了所求曲线?

因为圆锥曲线的方程本质上是线性的。

虽然圆锥曲线在几何上看起来是弯曲的（x^2, xy 等），但在**系数空间**（即 A, B, C, D, E, F 构成的空间）里，它们遵循线性叠加原理。

以下是详细的逻辑推导，解释为什么 C_1 和 C_2 的线性组合一定能 "抓住" 那条唯一的曲线。

1. 核心原理：解空间的维度

圆锥曲线的一般方程是：

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0$$

我们可以把系数看作一个 6 维向量 $\mathbf{c} = [A, B, C, D, E, F]^T$。每给定一个点 (x_i, y_i)，就相当于给这个向量加了一个线性约束（一个方程）：

$$Ax_i^2 + Bx_iy_i + \cdots + F = 0$$

- **总自由度**：6 个系数。
- **约束数量**：4 个点给出 4 个方程。
- **剩余自由度**：$6 - 4 = 2$。

这意味着，所有经过这 4 个点的圆锥曲线，其系数向量构成了一个 **2 维的线性子空间**。

线性代数的基本定理告诉我们：在一个 2 维线性空间中，任意两个**线性无关**的向量（基向量），都可以通过线性组合生成该空间内的**所有**向量。

- 我们的目标曲线 Q_{target} 肯定在这个空间里（因为它经过这 4 个点）。
- 我们构造的 C_1 和 C_2 也在这个空间里（因为它们也经过这 4 个点）。
- 只要 C_1 和 C_2 不是同一条曲线（线性无关），那么 Q_{target} **一定**可以表示为 $C_1 + \lambda C_2$。

2. 几何直观：为什么选 C_1 和 C_2 做基?

我们构造的 C_1 和 C_2 是**退化圆锥曲线**（两直线之积）。

- C_1 (**直线对** L_1L_2)：经过 P_1, P_2, P_3, P_4。
- C_2 (**直线对** M_1M_2)：经过 P_1, P_2, P_3, P_4。

这就好比我们在地图上找位置：- 所有经过这 4 点的曲线构成了一个 "平面"。- C_1

是这个平面上的一条 “经线”。- C_2 是这个平面上的一条 “纬线”。- 我们要找的目标曲线 Q_{target} 是这个平面上的某一个点。

通过调整 λ（相当于沿着某条路径走），我们一定能穿过 Q_{target} 所在的位置。

3. 为什么第五点能确定 λ?

线性组合 $Q(\lambda) = C_1 + \lambda C_2$ 代表了**所有**经过前 4 点的曲线族。

当我们把第五点 P_5 代入时：

$$\text{Value}(P_5) = C_1(P_5) + \lambda C_2(P_5) = 0$$

这是一个关于 λ 的**一次方程**。只要 $C_2(P_5) \neq 0$（即 P_5 不在 C_2 上），这个方程就有**唯一解**。

这个唯一的 λ 就像一把钥匙，从无穷多条经过前 4 点的曲线中，精准地挑出了经过第 5 点的那一条。

4. 特殊情况（什么时候会失败?）

虽然理论上 “一定包含”，但在以下特殊情况下，这种构造会失效（或者说基选得不好）：

1. C_1 **和** C_2 **线性相关**：如果 C_1 和 C_2 代表的是同一条曲线（例如点的位置非常特殊，导致两种分组方式得到的直线对完全一样），那么它们张成的空间只有 1 维，无法覆盖整个 2 维解空间。
   - 现实情况：只要 4 点处于一般位置（无三点共线），不同的配对方式（12-34 和 13-24）通常都会产生线性无关的 C_1, C_2。
2. P_5 **在基曲线上**：如果 P_5 恰好在 C_2 上，那么 $C_2(P_5) = 0$。
   - 如果 $C_1(P_5) \neq 0$，则方程无解（$\lambda \to \infty$），意味着目标曲线其实就是 C_2（或者需要交换 C_1, C_2 的角色）。
   - 如果 $C_1(P_5) = 0$ 且 $C_2(P_5) = 0$，说明 P_5 是前 4 点确定的交点之一（即点重合），此时无法确定唯一曲线。

C_1 和 C_2 的线性组合之所以一定包含所求曲线，是因为：**所有经过 4 个定点的圆锥曲线构成了一个 2 维线性空间，而 C_1 和 C_2 是这个空间的一组基。**

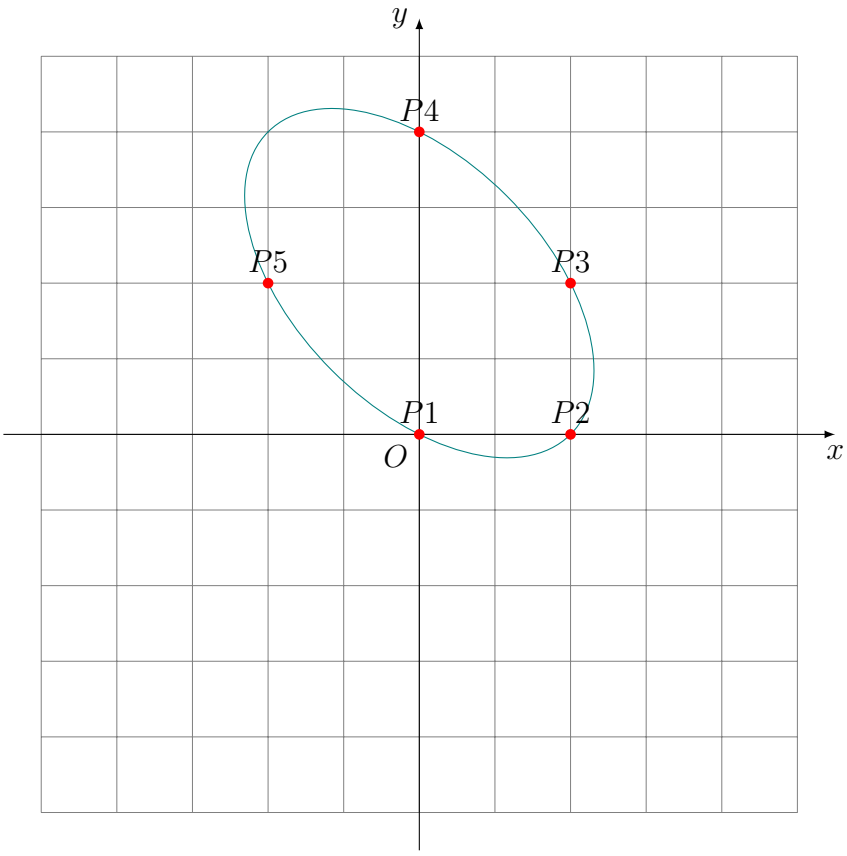
7.7.3 语法形式

Syntax

- conics/define/five points={\Pa,\Pb,\Pc,\Pd,\Pe,\C}: 由五点确定的圆锥曲线的系数矩阵
  - \Pa,\Pb,\Pc,\Pd,\Pe: 五点的齐次坐标，列向量
  - \C: 返回的圆锥曲线的系数矩阵

7.7.4 示例 1：五点确定的椭圆

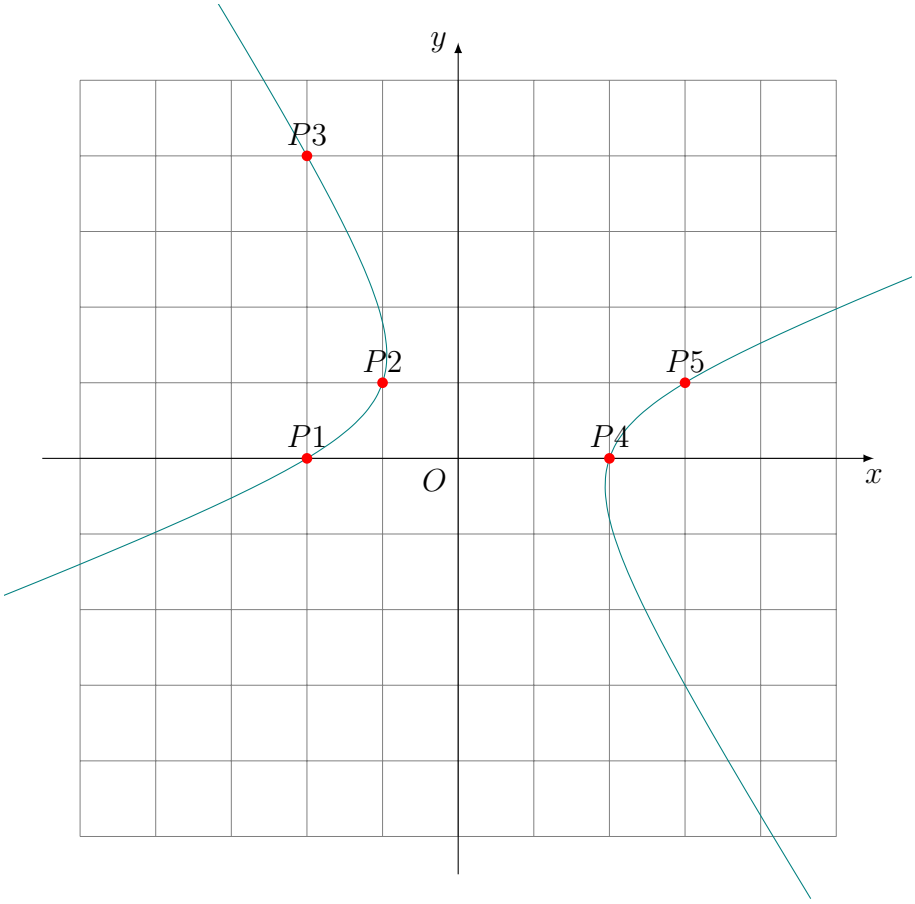
```
\begin{tikzpicture}
  \tikzmath{
    % 5 点的齐次坐标
    \Pa{1,1} = 0; \Pa{2,1} = 0; \Pa{3,1} = 1;
    \Pb{1,1} = 2; \Pb{2,1} = 0; \Pb{3,1} = 1;
    \Pc{1,1} = 2; \Pc{2,1} = 2; \Pc{3,1} = 1;
    \Pd{1,1} = 0; \Pd{2,1} = 4; \Pd{3,1} = 1;
    \Pe{1,1} = -2; \Pe{2,1} = 2; \Pe{3,1} = 1;
  }
  \tikzset{
    conics/define/five points={\Pa,\Pb,\Pc,\Pd,\Pe,\A},
    mat/stdout={\A,3,3},
  }

  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal] (\A);

  \coordinate (P1) at (\Pa{1,1},\Pa{2,1});
  \coordinate (P2) at (\Pb{1,1},\Pb{2,1});
  \coordinate (P3) at (\Pc{1,1},\Pc{2,1});
  \coordinate (P4) at (\Pd{1,1},\Pd{2,1});
  \coordinate (P5) at (\Pe{1,1},\Pe{2,1});

  \foreach \p/\placement in {P1/above, P2/above, P3/above, P4/above,
    P5/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.7.5 示例 2：五点确定的双曲线

```
\begin{tikzpicture}
  \tikzmath{
    % 5 点的齐次坐标
    \Pa{1,1} = -2; \Pa{2,1} = 0; \Pa{3,1} = 1;
    \Pb{1,1} = -1; \Pb{2,1} = 1; \Pb{3,1} = 1;
    \Pc{1,1} = -2; \Pc{2,1} = 4; \Pc{3,1} = 1;
    \Pd{1,1} = 2; \Pd{2,1} = 0; \Pd{3,1} = 1;
    \Pe{1,1} = 3; \Pe{2,1} = 1; \Pe{3,1} = 1;
  }
  \tikzset{
    conics/define/five points={\Pa,\Pb,\Pc,\Pd,\Pe,\A},
    mat/stdout={\A,3,3},
  }

  \clip (-6,-6) rectangle (6,6);
  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal] (\A);

  \coordinate (P1) at (\Pa{1,1},\Pa{2,1});
  \coordinate (P2) at (\Pb{1,1},\Pb{2,1});
```

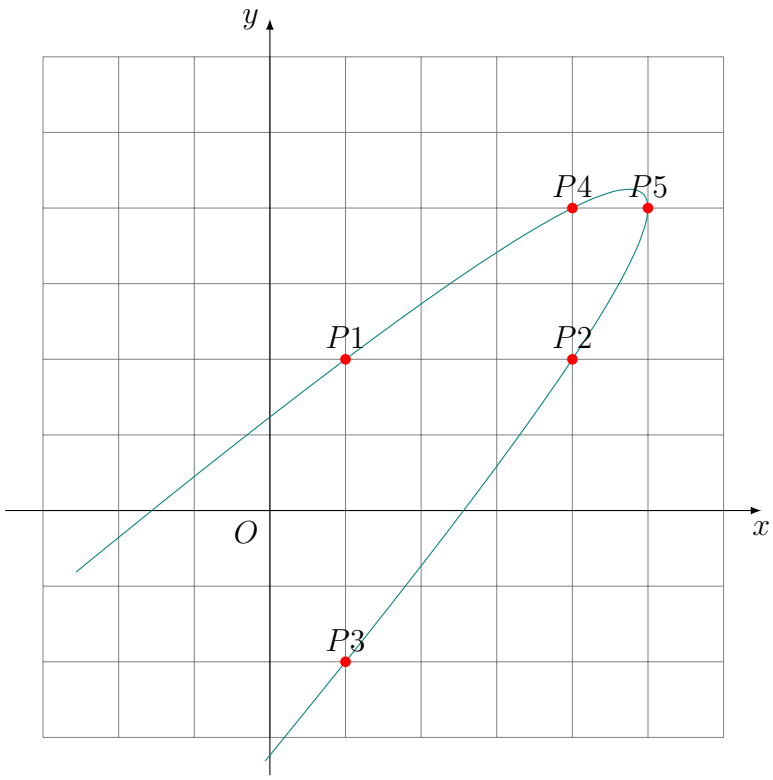
```
  \coordinate (P3) at (\Pc{1,1},\Pc{2,1});
  \coordinate (P4) at (\Pd{1,1},\Pd{2,1});
  \coordinate (P5) at (\Pe{1,1},\Pe{2,1});

  \foreach \p/\placement in {P1/above, P2/above, P3/above, P4/above, P5/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.7.6 示例 3：五点确定的抛物线

```
\begin{tikzpicture}
  \tikzmath{
    % 5 点的齐次坐标
    \Pa{1,1} = 1; \Pa{2,1} = 2; \Pa{3,1} = 1;
    \Pb{1,1} = 4; \Pb{2,1} = 2; \Pb{3,1} = 1;
    \Pc{1,1} = 1; \Pc{2,1} = -2; \Pc{3,1} = 1;
    \Pd{1,1} = 4; \Pd{2,1} = 4; \Pd{3,1} = 1;
```

```
    \Pe{1,1} = 5; \Pe{2,1} = 4; \Pe{3,1} = 1;
  }
  \tikzset{
    conics/define/five points={\Pa,\Pb,\Pc,\Pd,\Pe,\A},
    mat/stdout={\A,3,3},
  }

  \axes[grid] (-3:6,-3:6);
  \conic[draw=teal,domain=-5:5] (\A);

  \coordinate (P1) at (\Pa{1,1},\Pa{2,1});
  \coordinate (P2) at (\Pb{1,1},\Pb{2,1});
  \coordinate (P3) at (\Pc{1,1},\Pc{2,1});
  \coordinate (P4) at (\Pd{1,1},\Pd{2,1});
  \coordinate (P5) at (\Pe{1,1},\Pe{2,1});

  \foreach \p/\placement in {P1/above, P2/above, P3/above, P4/above,
    P5/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.8 五条切线确定的圆锥曲线

从射影几何的对偶性（Duality）来看，**“与 5 条直线相切的圆锥曲线”** 实际上是 “过 5 个点的圆锥曲线” 的完全对偶问题。

在点几何中，我们讨论点落在曲线上；在**线几何（Line Geometry）**中，我们讨论直线与曲线相切。

7.8.1 对偶原理：从点到线

根据帕斯卡定理（Pascal's Theorem）的对偶版本——**布利安桑定理（Brianchon's Theorem）**：如果一个六边形的外切圆锥曲线存在，那么其三条主对角线（连接相对顶点的直线）必交于一点。

对于给定的 5 条直线 L_1, L_2, L_3, L_4, L_5，它们唯一确定一条圆锥曲线。这与 5 点定一线的逻辑完全一致：

- **点约束**：每个点提供 1 个自由度约束。
- **线约束**：每条切线同样提供 1 个自由度约束。

7.8.2 齐次坐标下的对偶表示

如果你正在处理矩阵算法，通常会使用**对偶圆锥曲线矩阵（Dual Conic Matrix）**，记为 $\boldsymbol{C}^*$：

1. **原始矩阵 $\boldsymbol{C}$**：描述点 $\boldsymbol{x}$ 是否在曲线上，方程为 $\boldsymbol{x}^T\boldsymbol{C}\boldsymbol{x} = 0$。
2. **对偶矩阵 $\boldsymbol{C}^*$**：描述直线 $\boldsymbol{l} = [a, b, c]^T$ 是否与曲线相切，方程为：

$$\boldsymbol{l}^T\boldsymbol{C}^*\boldsymbol{l} = 0$$

其中 $\boldsymbol{C}^*$ 是 $\boldsymbol{C}$ 的伴随矩阵（Adjoint Matrix）。在非退化情况下，$\boldsymbol{C}^* = \boldsymbol{C}^{-1}$。

通过已知的 5 条直线 $\boldsymbol{l}_1, \ldots, \boldsymbol{l}_5$，你可以建立类似于点问题的齐次线性方程组，求解出 $\boldsymbol{C}^*$ 矩阵的 6 个独立参数。

> 在求解与 5 条直线相切的抛物线时出现 “较大差异”，问题通常不出在数学原理上，而是出在**数值稳定性（Numerical Stability）**或**伴随矩阵求逆**的细节上。将向量或矩阵归一化能够改善计算的稳定性。

特殊情况与退化

这是一个完美的对偶关系：

- **点构型**：5 个点，若有 3 点共线，则曲线退化。
- **切线构型**：5 条切线，若有 3 线共点，则曲线退化。

当给定的 5 条切线中有**三条或更多交于同一点**时，所确定的圆锥曲线将是**退化的**，并且这条圆锥曲线**可能不唯一**。

详细解释：

1. **退化情况的发生**在一般情况下，平面上任意五条切线（其中任意三条不共点）可以唯一确定一条非退化的圆锥曲线。然而，如果这五条切线中有三条或更多相交于同一个点，情况就会发生变化。这个公共交点会成为退化圆锥曲线的一个特殊点（如两条相交直线的交点，或一个孤立点），导致曲线不再是光滑的椭圆、双曲线或抛物线，而是分解为更简单的几何图形，例如两条直线或一个点。

2. **对偶曲线束方法的适用性**对偶问题的求解思路与点问题完全对应。我们可以构造一个 “对偶曲线束”，它由通过其中四条切线的圆锥曲线族构成。

    - 当第五条切线不与前四条中的任意三条共点时，代入其条件可以解出唯一的参数，得到唯一的非退化圆锥曲线。
    - 当第五条切线恰好通过前四条切线中某两条的交点时，就满足了 “三线共点” 的退化条件。此时，对偶曲线束中会包含退化的圆锥曲线。

3. **唯一性问题与退化形态**这是问题的关键。与五点问题类似，当出现退化时，解可能不唯一。

    - **唯一性丧失**：例如，如果五条切线恰好是两条直线（一条上有 3 条切线，另一条上有 2 条），那么这两条直线本身就是一条满足条件的退化圆锥曲线（两条相交直线）。但在某些构型下，可能存在多种方式将这五条线分组，从而导致解不唯一。
    - **退化形态**：退化的圆锥曲线可以是以下几种形式：
        - **两条相交直线**：最常见的退化形式，对应于三条切线交于一点的情况。
        - **两条平行直线**：可视为交点在无穷远处的特殊情况。
        - **一个孤立点**：例如，当所有切线都 “切” 于同一个点时。
        - **一条直线**：更为退化的情形。

总而言之，对偶的曲线束方法在处理 5 条切线时是普适的。但当遇到三条或更多切线共点的情况时，你需要意识到最终得到的将是一条退化的圆锥曲线，并且这个结果可能不是唯一的。

在计算抛物线时这个矩阵可能属于 “病态” 矩阵，导致误差较大。为什么它是病态的?

1. 几何原因：切点过于密集，病态的本质是**信息冗余**或**区分度低**。当我们将几乎共线的向量进行外积运算构造矩阵时，矩阵的行/列向量之间就会呈现**近似线性相关**，导致行列式趋近于 0，条件数趋向无穷大。

2. 数值原因：巨大的条件数，让我们计算最终得到的线圆锥曲线矩阵 S 的条件数。在数值线性代数中，条件数超过 10^3 通常被认为是有问题的，超过 10^6 则是极度病态。对于 3×3 矩阵来说，**1000 左右的条件数意味着我们在计算逆矩阵时，可能会损失约 3-4 位有效数字的精度。**

病态带来的后果

如果在计算机中使用单精度浮点数(float32)进行计算:1. **求逆误差大**:计算 $Q = S^{-1}$ 时，微小的舍入误差会被放大 1000 倍。2. **系数漂移**：原本应该是整数的系数（如 2, -4, -8)，可能会变成 $1.9998, -4.0005, -7.992$。3. **类型误判**：- 抛物线的判别式应该是严格的 **0**。- 由于误差，计算出的判别式可能是 10^{-5} 或 -10^{-5}。- 这会导致计算机误判该曲线为**双曲线**或**椭圆**，而不是抛物线。

7.8.3 语法形式

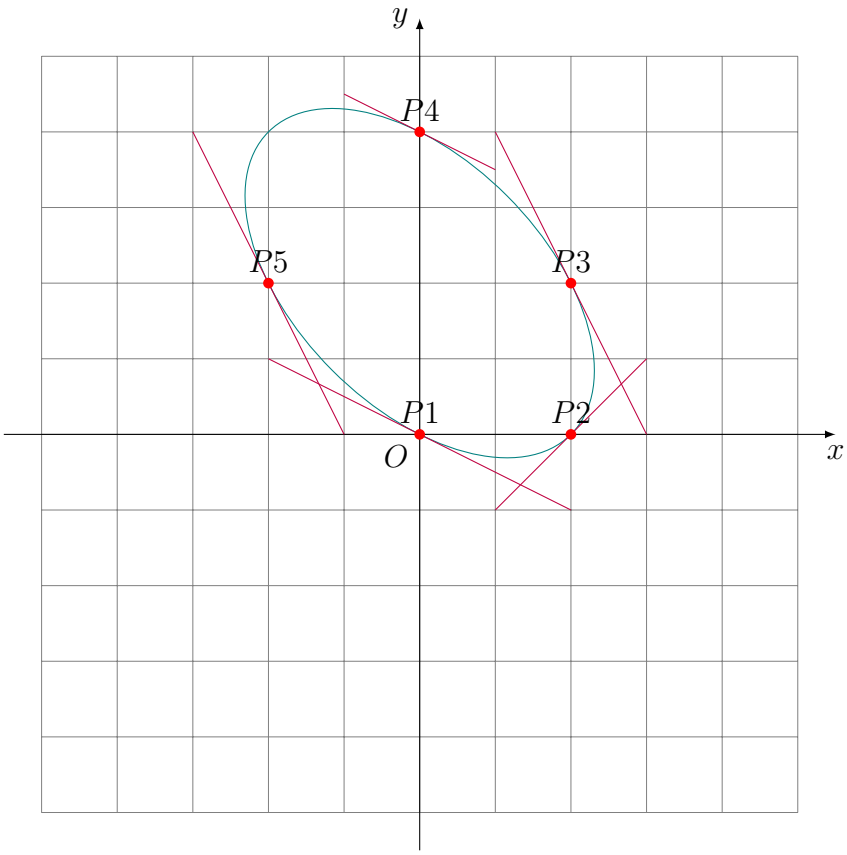
Syntax

- `conics/define/five tangents={\La,\Lb,\Lc,\Ld,\Le,\C}`: 由五条切线确定的圆锥曲线的系数矩阵
  - `\La,\Lb,\Lc,\Ld,\Le`: 五条切线的齐次坐标，列向量
  - `\C`: 返回的圆锥曲线的系数矩阵

7.8.4 示例 1：五条切线确定的椭圆

```
\begin{tikzpicture}
  \tikzmath{
    % 5 点的齐次坐标
    \Pa{1,1} = 0; \Pa{2,1} = 0; \Pa{3,1} = 1;
    \Pb{1,1} = 2; \Pb{2,1} = 0; \Pb{3,1} = 1;
    \Pc{1,1} = 2; \Pc{2,1} = 2; \Pc{3,1} = 1;
    \Pd{1,1} = 0; \Pd{2,1} = 4; \Pd{3,1} = 1;
    \Pe{1,1} = -2; \Pe{2,1} = 2; \Pe{3,1} = 1;
  }
  \tikzset{
```

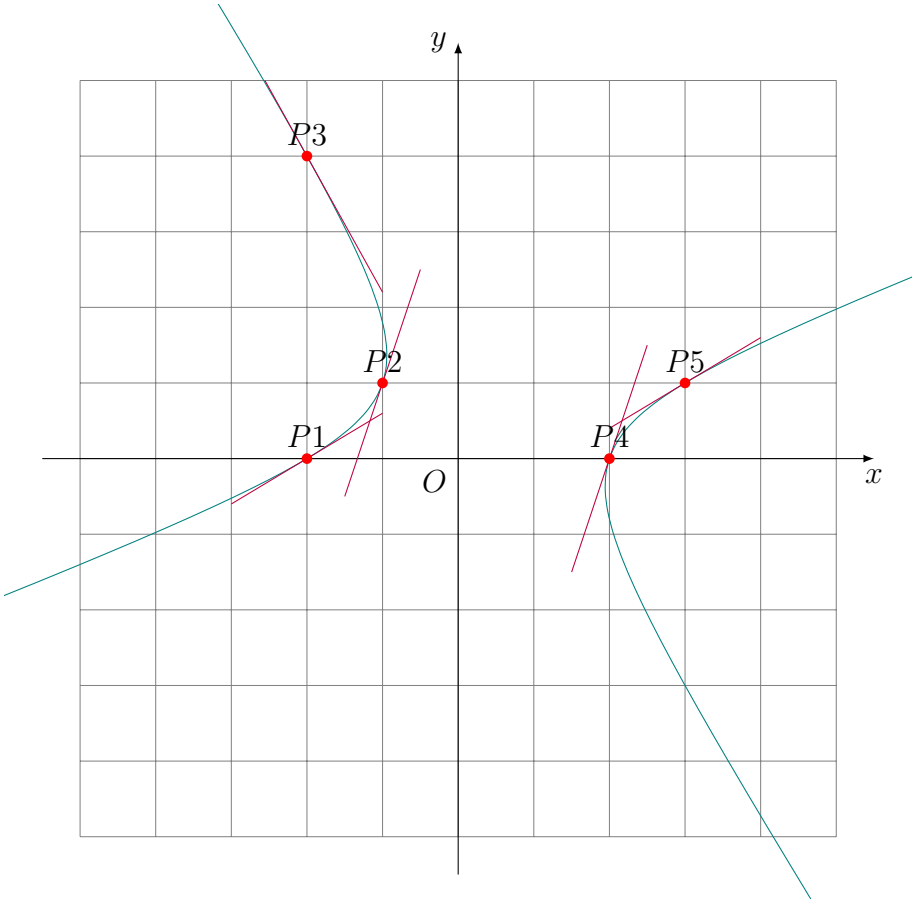
```
    % 通过 5 点确定曲线
    conics/define/five points={\Pa,\Pb,\Pc,\Pd,\Pe,\A},
    % 计算 5 条切线
    mat/product={\A,\Pa,3,3,1,\La},
    mat/product={\A,\Pb,3,3,1,\Lb},
    mat/product={\A,\Pc,3,3,1,\Lc},
    mat/product={\A,\Pd,3,3,1,\Ld},
    mat/product={\A,\Pe,3,3,1,\Le},
    % 计算与该 5 条切线相切的曲线
    conics/define/five tangents={\La,\Lb,\Lc,\Ld,\Le,\B},
    mat/stdout={\B,3,3},
  }

  \coordinate (P1) at (\Pa{1,1},\Pa{2,1});
  \coordinate (P2) at (\Pb{1,1},\Pb{2,1});
  \coordinate (P3) at (\Pc{1,1},\Pc{2,1});
  \coordinate (P4) at (\Pd{1,1},\Pd{2,1});
  \coordinate (P5) at (\Pe{1,1},\Pe{2,1});

  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal] (\B);
  \segment[draw=purple,clip=(-2:2,-4:4)] (\La);
  \segment[draw=purple,clip=(1:3,-4:4)] (\Lb);
  \segment[draw=purple,clip=(1:3,-4:4)] (\Lc);
  \segment[draw=purple,clip=(-1:1,-4:5)] (\Ld);
  \segment[draw=purple,clip=(-3:-1,-4:4)] (\Le);

  \foreach \p/\placement in {P1/above, P2/above, P3/above, P4/above,
    P5/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.8.5 示例 2：五条切线确定的双曲线

```
\begin{tikzpicture}
  \tikzmath{
    % 5 点的齐次坐标
    \Pa{1,1} = -2; \Pa{2,1} = 0; \Pa{3,1} = 1;
    \Pb{1,1} = -1; \Pb{2,1} = 1; \Pb{3,1} = 1;
    \Pc{1,1} = -2; \Pc{2,1} = 4; \Pc{3,1} = 1;
    \Pd{1,1} = 2; \Pd{2,1} = 0; \Pd{3,1} = 1;
    \Pe{1,1} = 3; \Pe{2,1} = 1; \Pe{3,1} = 1;
  }
  \tikzset{
    % 通过 5 点确定曲线
    conics/define/five points={\Pa,\Pb,\Pc,\Pd,\Pe,\A},
    % 计算 5 条切线
    mat/product={\A,\Pa,3,3,1,\La},
    mat/product={\A,\Pb,3,3,1,\Lb},
    mat/product={\A,\Pc,3,3,1,\Lc},
    mat/product={\A,\Pd,3,3,1,\Ld},
    mat/product={\A,\Pe,3,3,1,\Le},
    % 计算与该 5 条切线相切的曲线
    conics/define/five tangents={\La,\Lb,\Lc,\Ld,\Le,\B},
```

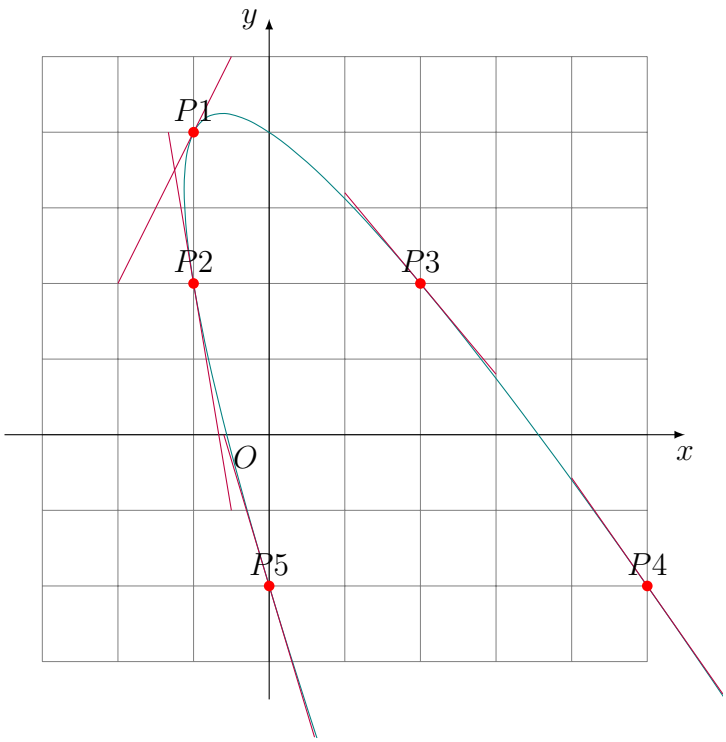
```
    mat/stdout={\B,3,3},
  }

  \coordinate (P1) at (\Pa{1,1},\Pa{2,1});
  \coordinate (P2) at (\Pb{1,1},\Pb{2,1});
  \coordinate (P3) at (\Pc{1,1},\Pc{2,1});
  \coordinate (P4) at (\Pd{1,1},\Pd{2,1});
  \coordinate (P5) at (\Pe{1,1},\Pe{2,1});

  \clip (-6,-6) rectangle (6,6);
  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal] (\B);
  \segment[draw=purple,clip=(-3:-1,-4:4)] (\La);
  \segment[draw=purple,clip=(-1.5:-0.5,-4:4)] (\Lb);
  \segment[draw=purple,clip=(-3:-1,-4:5)] (\Lc);
  \segment[draw=purple,clip=(1.5:2.5,-4:5)] (\Ld);
  \segment[draw=purple,clip=(2:4,-4:4)] (\Le);

  \foreach \p/\placement in {P1/above, P2/above, P3/above, P4/above,
    P5/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.8.6 示例 3：五条切线确定的抛物线

```
\begin{tikzpicture}
  \tikzmath{
    % % 5 点的齐次坐标
    \Pa{1,1} = -1; \Pa{2,1} = 4; \Pa{3,1} = 1;
    \Pb{1,1} = -1; \Pb{2,1} = 2; \Pb{3,1} = 1;
    \Pc{1,1} = 2; \Pc{2,1} = 2; \Pc{3,1} = 1;
    \Pd{1,1} = 5; \Pd{2,1} = -2; \Pd{3,1} = 1;
    \Pe{1,1} = 0; \Pe{2,1} = -2; \Pe{3,1} = 1;
  }
  \tikzset{
    % 通过 5 点确定曲线
    conics/define/five points={\Pa,\Pb,\Pc,\Pd,\Pe,\A},
    % 计算 5 条切线
    mat/product={\A,\Pa,3,3,1,\La},
    mat/product={\A,\Pb,3,3,1,\Lb},
    mat/product={\A,\Pc,3,3,1,\Lc},
    mat/product={\A,\Pd,3,3,1,\Ld},
    mat/product={\A,\Pe,3,3,1,\Le},
```

```
    % 计算与该 5 条切线相切的曲线
    conics/define/five tangents={\La,\Lb,\Lc,\Ld,\Le,\B},
    mat/stdout={\B,3,3},
  }

  \coordinate (P1) at (\Pa{1,1},\Pa{2,1});
  \coordinate (P2) at (\Pb{1,1},\Pb{2,1});
  \coordinate (P3) at (\Pc{1,1},\Pc{2,1});
  \coordinate (P4) at (\Pd{1,1},\Pd{2,1});
  \coordinate (P5) at (\Pe{1,1},\Pe{2,1});

  \clip (-4,-4) rectangle (6,6);
  \axes[grid] (-3:5,-3:5);
  \conic[draw=teal,domain=-4:4] (\B);
  \segment[draw=purple,clip=(-2:0,-4:5)] (\La);
  \segment[draw=purple,clip=(-1.5:-0.5,-4:4)] (\Lb);
  \segment[draw=purple,clip=(1:3,-4:5)] (\Lc);
  \segment[draw=purple,clip=(4:6,-4:0)] (\Ld);
  \segment[draw=purple,clip=(-2:4,-4:0)] (\Le);

  \foreach \p/\placement in {P1/above, P2/above, P3/above, P4/above,
    P5/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.9 极点与极线（Pole and Polar Line）

7.9.1 极点与极线

极点（Pole）与极线（Polar Line）是射影几何中一对非常优美的对偶概念。它们建立了点与直线之间的一一对应关系，这种关系通常由一个**二次曲线**（如圆、椭圆、双曲线或抛物线）作为媒介。

一般地，对于任意二次曲线，极点与极线的定义可以基于 “调和点列”：过一点 P 作一条直线交二次曲线于 M、N 两点，如果直线上存在另一点 Q，使得 P、Q、M、N 四点构成调和点列，那么当这条直线绕 P 点旋转时，所有 Q 点的轨迹就形成一条直线，这条直线就是点 P 的极线，而 P 就是这条直线的极点。一个非常重要的性质是配极原则：如果点 P 的极线经过点 Q，那么点 Q 的极线也必然经过点 P。这体现了极点与极线之间完美的对称性。

设有圆（或圆锥曲线）的方程：

$$ax^2 + bxy + cy^2 + dx + ey + f = 0$$

对于平面上一点 $P(x_0, y_0)$（无论它在曲线内外），将其坐标代入方程，通过一种 “替换法则” 得到的直线方程，就是 P 的极线。

替换法则（以一般二次曲线为例）：

- $x^2 \to xx_0$
- $y^2 \to yy_0$
- $xy \to \dfrac{xy_0 + x_0y}{2}$
- $x \to \dfrac{x + x_0}{2}$
- $y \to \dfrac{y + y_0}{2}$
- 常数项不变

核心性质

极点与极线最迷人的地方在于它们的**互惠性（Reciprocity）**：

1. **对偶性**：如果点 P 的极线是直线 l，那么直线 l 的极点就是点 P。这种 “点” 与 “线” 的对应关系是互逆的。

2. **配极原则**：这是最核心的几何性质。**如果点 P 在点 Q 的极线上，那么点 Q 也一定在点 P 的极线上**。这被称为 “共轭” 关系。在射影几何中，这建立了一种对称的关联结构。

3. **调和共轭**：设一条直线穿过点 P，与圆锥曲线交于两点 M N，并与 P 的极线交于点 Q。那么点对 (P, Q) 调和分割点对 (M, N)。

$$(M, N; P, Q) = -1$$

这是一个射影不变的性质。

矩阵形式

在射影几何与线性代数中，利用**矩阵**来表示极点与极线不仅简洁，而且能推广到所有的二次曲线（圆、椭圆、双曲线、抛物线）。

1. 二次曲线的矩阵表示

任何二次曲线都可以表示为齐次坐标下的对称矩阵形式。设点 X 的齐次坐标为 $\boldsymbol{x} = [x, y, 1]^T$，二次曲线方程可以写为：

$$\boldsymbol{x}^T C \boldsymbol{x} = 0$$

其中 C 是一个 3×3 的**实对称矩阵**。

例如，对于单位圆 $x^2 + y^2 = 1$，其矩阵 C 为：

$$C = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

2. 极点与极线的映射关系

在线性代数视角下，极点与极线的转换本质上是一个**线性映射**。

- **极点 $\to$ 极线**：设极点 P 的齐次坐标为 $\boldsymbol{p}$，则其对应的极线 L 的坐标（系数向量 $\boldsymbol{l} = [a, b, c]^T$）为：

  $$\boldsymbol{l} = C\boldsymbol{p}$$

  极线方程即为：$\boldsymbol{l}^T \boldsymbol{x} = 0$ 或 $(C\boldsymbol{p})^T \boldsymbol{x} = 0$。

- **极线 $\to$ 极点**：如果已知极线 $\boldsymbol{l}$，则极点 $\boldsymbol{p}$ 为：

  $$\boldsymbol{p} = C^{-1}\boldsymbol{l}$$

  （前提是二次曲线是非退化的，即 $\det(C) \neq 0$）

3. 矩阵形式下的几何性质

- **点在曲线上**：点 $\boldsymbol{p}$ 在曲线上的充要条件是 $\boldsymbol{p}^T C\boldsymbol{p} = 0$。这也意味着点 $\boldsymbol{p}$ 落在它自己的极线上（即极线是切线）。
- **共轭点**：若点 $\boldsymbol{p}_1$ 在点 $\boldsymbol{p}_2$ 的极线上，则满足：

$$\boldsymbol{p}_1^T C\boldsymbol{p}_2 = 0$$

  由于 C 是对称矩阵，这个关系是对称的，这从代数上完美解释了**互惠性**。
- **切线交点**：如果两条极线 $\boldsymbol{l}_1$ 和 $\boldsymbol{l}_2$ 相交于点 $\boldsymbol{q}$，那么点 $\boldsymbol{q}$ 就是连接这两条极线对应极点的直线的极点。

7.9.2 语法形式

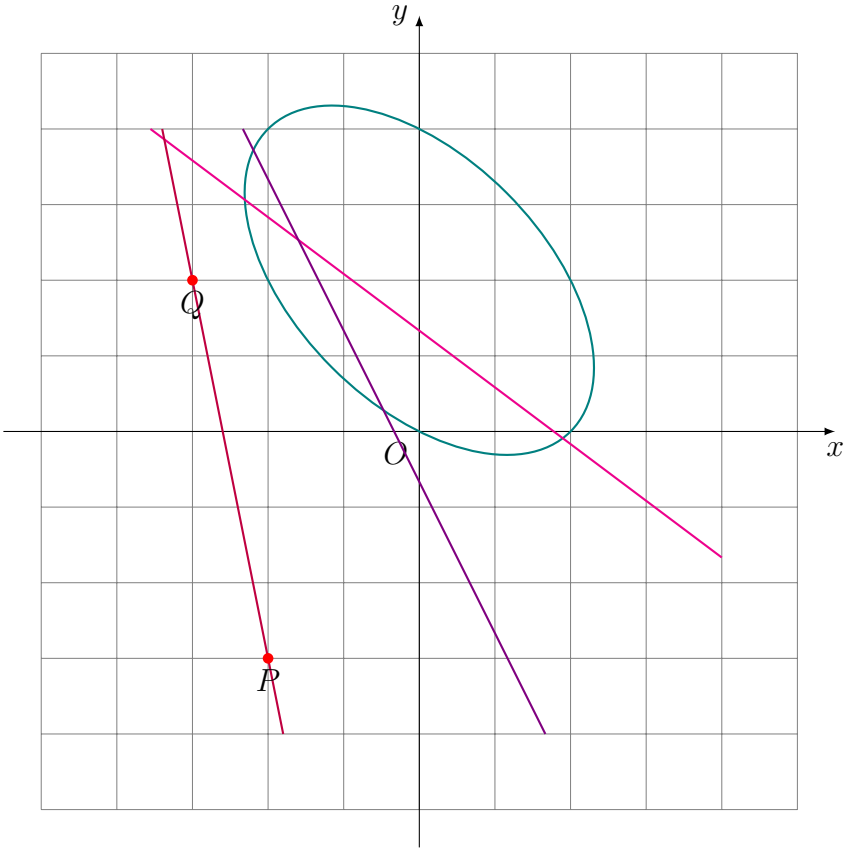
Syntax

- `conics/polar={\C,\P,\L}`: 计算极线的齐次坐标
  - `\C`: 二次曲线的系数矩阵
  - `\P`: 点的齐次坐标
  - `\L`: 直线的齐次坐标
- `conics/pole={\C,\L,\P}`: 计算极点的齐次坐标
  - `\C`: 二次曲线的系数矩阵
  - `\L`: 直线的齐次坐标
  - `\P`: 点的齐次坐标

7.9.3 示例：极点与极线

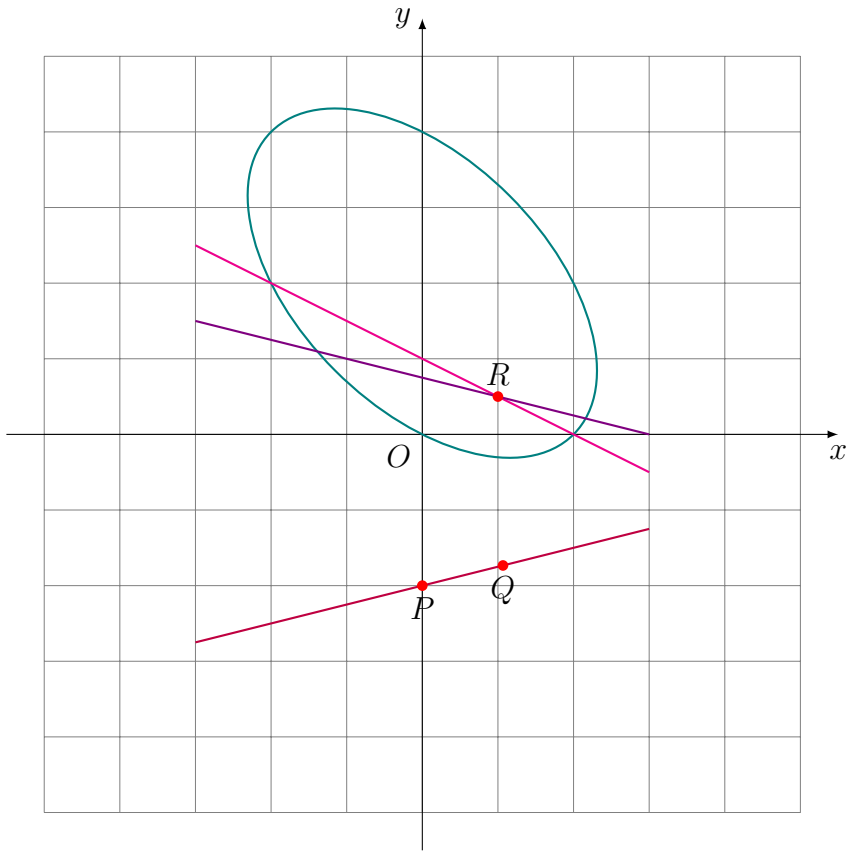
```
\begin{tikzpicture}
  \tikzmath{
    % 圆锥曲线系数
    \coefs{1} = 1; \coefs{2} = 1; \coefs{3} = 1;
    \coefs{4} = -2; \coefs{5} = -4; \coefs{6} = 0;
    % 点的齐次坐标
    \P{1,1} = -2; \P{2,1} = -3; \P{3,1} = 1;
    \Q{1,1} = -3; \Q{2,1} = 2; \Q{3,1} = 1;
  }
  \tikzset{
    conics/define/quadratic={\coefs,\A},
    conics/polar={\A,\P,\l},% P 点关于曲线 A 的极线 l
    conics/polar={\A,\Q,\m},% Q 点关于曲线 A 的极线 m
```

```
    conics/cross={\l,\m,\R},% l 和 m 的交点 R
    conics/polar={\A,\R,\n},% R 点关于曲线 A 的极线 n, 应通过 点 P, Q
  }

  \coordinate (P) at (\P{1,1},\P{2,1});
  \coordinate (Q) at (\Q{1,1},\Q{2,1});

  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal,thick] (\A);
  \segment[draw=magenta,thick,clip=(-4:4,-4:4)] (\l);
  \segment[draw=violet,thick,clip=(-4:4,-4:4)] (\m);
  \segment[draw=purple,thick,clip=(-4:4,-4:4)] (\n);

  \foreach \p/\placement in {P/below,Q/below}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```




```
\begin{tikzpicture}
  \tikzmath{
    % 圆锥曲线系数
```

```
    \coefs{1} = 1; \coefs{2} = 1; \coefs{3} = 1;
    \coefs{4} = -2; \coefs{5} = -4; \coefs{6} = 0;
    % 线的齐次坐标
    \l{1,1} = 1; \l{2,1} = 2; \l{3,1} = -2;
    \m{1,1} = 1; \m{2,1} = 4; \m{3,1} = -3;
  }
  \tikzset{
    conics/define/quadratic={\coefs,\A},
    conics/pole={\A,\l,\P},% 直线 l 关于曲线 A 的极点 P
    conics/pole={\A,\m,\Q},% 直线 m 关于曲线 A 的极点 Q
    conics/cross={\P,\Q,\n},% 过点 P Q 的直线 n
    conics/pole={\A,\n,\R},% 直线 n 关于曲线 A 的极点 R, 应为 l 和 m 的交点
    conics/dehomogenize={\P,\PP},
    conics/dehomogenize={\Q,\QQ},
    conics/dehomogenize={\R,\RR},
  }

  \coordinate (P) at (\PP{1},\PP{2});
  \coordinate (Q) at (\QQ{1},\QQ{2});
  \coordinate (R) at (\RR{1},\RR{2});

  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal,thick] (\A);
  \segment[draw=magenta,thick,clip=(-3:3,-4:4)] (\l);
  \segment[draw=violet,thick,clip=(-3:3,-4:4)] (\m);
  \segment[draw=purple,thick,clip=(-3:3,-4:4)] (\n);

  \foreach \p/\placement in {P/below,Q/below,R/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.10 直线与圆锥曲线的交点

在高等几何与计算机图形学中，使用**齐次坐标 (Homogeneous Coordinates)** 处理圆锥曲线与直线的交点问题，可以将复杂的代数展开转化为简洁的线性代数运算。

7.10.1 求解交点的参数化方法

设齐次坐标点与直线：

- **点**：平面上的点 (x, y) 表示为向量 $\boldsymbol{x} = [x, y, 1]^T$。
- **直线**：直线 $ax + by + c = 0$ 表示为向量 $\boldsymbol{l} = [a, b, c]^T$。

点在直线上的充要条件是：$\boldsymbol{l}^T\boldsymbol{x} = 0$。

点 $\boldsymbol{x}$ 在曲线上的充要条件是：$\boldsymbol{x}^T\boldsymbol{C}\boldsymbol{x} = 0$。

求直线 $\boldsymbol{l}$ 与曲线 $\boldsymbol{C}$ 的交点，本质上是在寻找满足 $\boldsymbol{l}^T\boldsymbol{x} = 0$ 且 $\boldsymbol{x}^T\boldsymbol{C}\boldsymbol{x} = 0$ 的向量 $\boldsymbol{x}$。

第一步：参数化直线

找到直线上任意两个不同的点 $\boldsymbol{u}$ 和 $\boldsymbol{v}$（即满足 $\boldsymbol{l}^T\boldsymbol{u}=0$ 和 $\boldsymbol{l}^T\boldsymbol{v}=0$）。

直线上任何一点都可以表示为它们的线性组合：

$$\boldsymbol{x}=\lambda\boldsymbol{u}+\mu\boldsymbol{v}$$

第二步：代入二次型

将参数化的 $\boldsymbol{x}$ 代入曲线方程：

$$(\lambda\boldsymbol{u}+\mu\boldsymbol{v})^T\boldsymbol{C}(\lambda\boldsymbol{u}+\mu\boldsymbol{v})=0$$

展开得到关于 λ 和 μ 的齐次二次方程：

$$\lambda^2(\boldsymbol{u}^T\boldsymbol{C}\boldsymbol{u})+2\lambda\mu(\boldsymbol{u}^T\boldsymbol{C}\boldsymbol{v})+\mu^2(\boldsymbol{v}^T\boldsymbol{C}\boldsymbol{v})=0$$

第三步：求解比例 λ/μ

令 $\rho=\lambda/\mu$，上述方程是一个典型的一元二次方程。* 通过求根公式解出 ρ_1,ρ_2。* 将解代回 $\boldsymbol{x}=\rho\boldsymbol{u}+\boldsymbol{v}$，即可得到两个交点的齐次坐标。

矩阵视角的几何优势：

1. **统一性**：无论曲线是椭圆、双曲线还是抛物线，计算过程完全一致。
2. **处理无穷远点**：齐次坐标允许我们直接计算直线与曲线在“无穷远处”的交点（例如直线平行于渐近线的情况），此时解出的坐标最后一位为 0。
3. **对偶性 (Duality)**：
   - 如果 $\boldsymbol{x}$ 是曲线上的点，则该点处的**切线**向量为 $\boldsymbol{l}=\boldsymbol{C}\boldsymbol{x}$。
   - 如果 $\boldsymbol{x}$ 是曲线外的一点，则 $\boldsymbol{l}=\boldsymbol{C}\boldsymbol{x}$ 代表该点对应的**极线 (Polar Line)**。
4. 判定相切的简洁条件

   在矩阵视角下，直线 $\boldsymbol{l}$ 与曲线 $\boldsymbol{C}$ 相切的充要条件可以表示为：

   $$\boldsymbol{l}^T\boldsymbol{C}^*\boldsymbol{l}=0$$

   其中 $\boldsymbol{C}^*$ 是 $\boldsymbol{C}$ 的伴随矩阵（在非退化情况下与逆矩阵 $\boldsymbol{C}^{-1}$ 成比例）。这在已知直线斜率求切线时非常高效。

7.10.2 Richter-Gebert 算法

Jürgen Richter-Gebert 算法 (Richter-Gebert, 2011, 页 194) 的核心是构造一个特定的**秩 2 矩阵**，并将其分解为两个**秩 1 矩阵**（代表两个交点）。这个方法规直接利用齐次坐标的代数性质。

7.10.3 语法形式

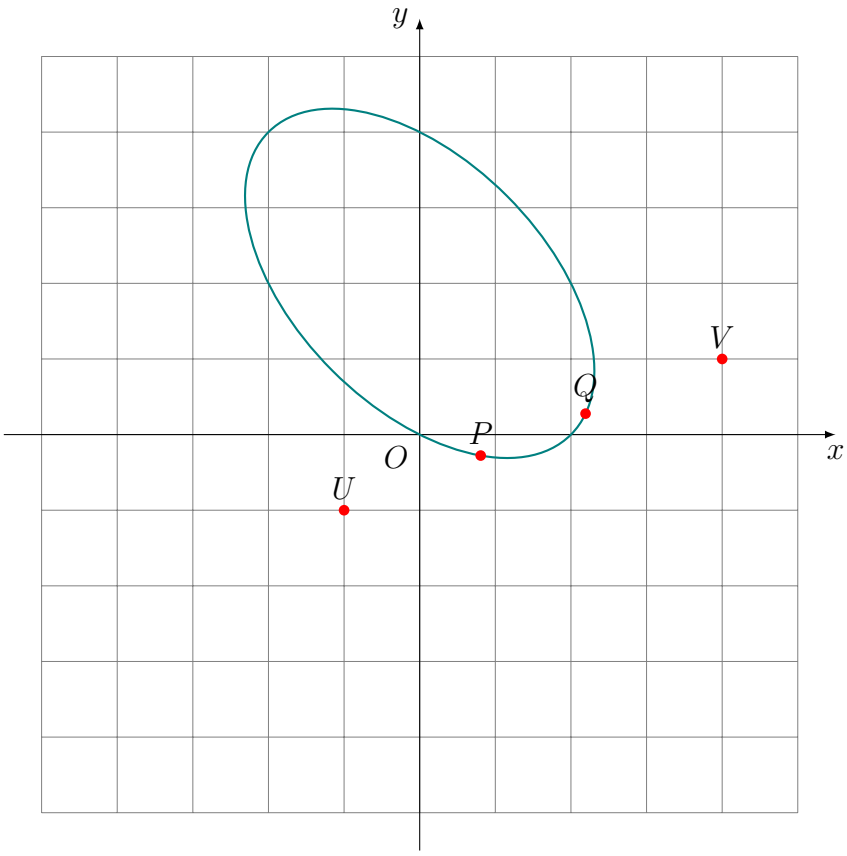
Syntax

- `conics/intersect cl={\C,\U,\V,\P,\Q}`: 采用参数化方法求圆锥曲线与直线的交点
  - `\C`: 圆锥曲线的系数矩阵
  - `\U,\V`: 直线上两点的齐次坐标，列向量
  - `\P,\Q`: 交点的齐次坐标，列向量，顺序是按从 U 到 V
- `conics/intersect cn={\C,\l,\P,\Q}`: 采用 Richter-Gebert 算法求圆锥曲线与直线的交点
  - `\C`: 圆锥曲线的系数矩阵
  - `\l`: 直线的齐次坐标，列向量
  - `\P,\Q`: 交点的齐次坐标，列向量，顺序是直线的正方向

7.10.4 示例 1：直线（两点）与圆锥曲线的交点

```
\begin{tikzpicture}
  \tikzmath{
    \coefs{1} = 1; \coefs{2} = 1; \coefs{3} = 1;
    \coefs{4} = -2; \coefs{5} = -4; \coefs{6} = 0;
    % 两点的齐次坐标
    \U{1,1} = -1; \U{2,1} = -1; \U{3,1} = 1;
    \V{1,1} = 4; \V{2,1} = 1; \V{3,1} = 1;
  }
  \tikzset{
    conics/define/quadratic={\coefs,\A},
    conics/intersect cl={\A,\U,\V,\P,\Q},
    conics/dehomogenize={\U,\UU},
    conics/dehomogenize={\V,\VV},
    conics/dehomogenize={\P,\PP},
    conics/dehomogenize={\Q,\QQ},
  }

```

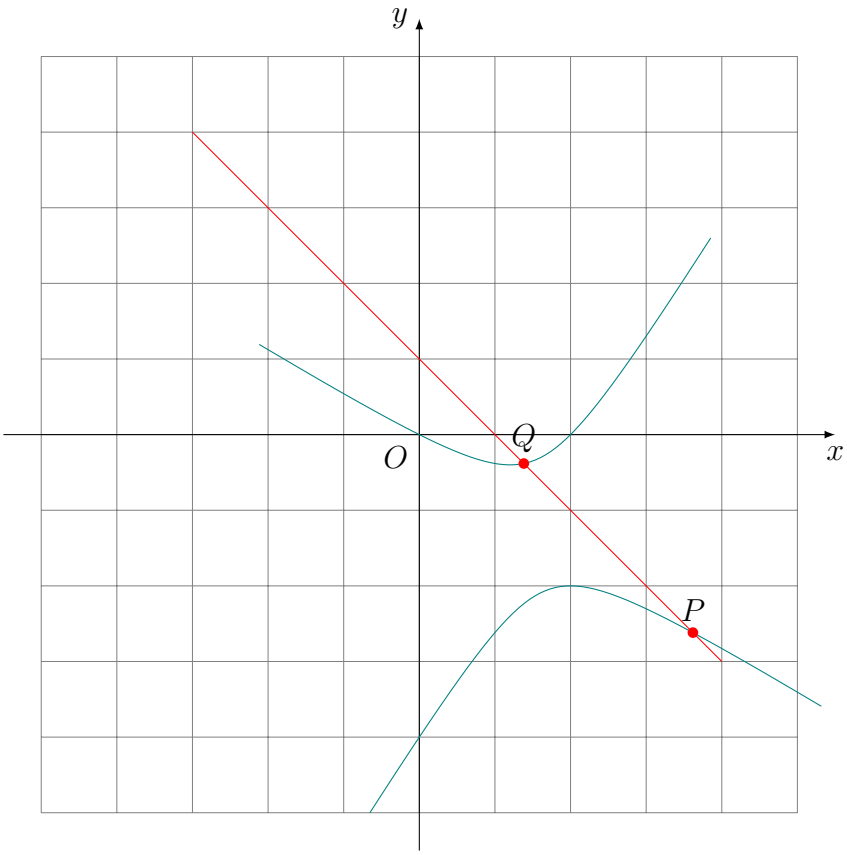
```
  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal,thick] (\A);

  \coordinate (U) at (\UU{1},\UU{2});
  \coordinate (V) at (\VV{1},\VV{2});
  \coordinate (P) at (\PP{1},\PP{2});
  \coordinate (Q) at (\QQ{1},\QQ{2});

  \foreach \p/\placement in {P/above, Q/above, U/above, V/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.10.5 示例 2：直线（系数）与圆锥曲线的交点

```
\begin{tikzpicture}
  \tikzmath{
    \coefs{1} = 1; \coefs{2} = 1; \coefs{3} = -1;
    \coefs{4} = -2; \coefs{5} = -4; \coefs{6} = 0;
    \l{1,1} = 1; \l{2,1} = 1; \l{3,1} = -1;
```

```
  }
  \tikzset{
    conics/define/quadratic={\coefs,\A},
    conics/intersect cn={\A,\l,\P,\Q},
    conics/dehomogenize={\P,\PP},
    conics/dehomogenize={\Q,\QQ},
  }

  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal] (\A);
  \segment[draw=red,clip=(-4:4,-4:4)] (\l);

  \coordinate (P) at (\PP{1},\PP{2});
  \coordinate (Q) at (\QQ{1},\QQ{2});

  \foreach \p/\placement in {P/above, Q/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.11 圆锥曲线的切线

利用**极线**和**对偶二次曲线**求解切线，本质上是利用**射影几何**的对偶原理。这种方法在处理复杂二次曲线（如包含 xy 交叉项）时，比传统的代数代入法更具系统性和几何美感。

这里的核心逻辑是：

1. **极线**：确定了两个切点的连线（切点弦）。
2. **对偶二次曲线**：提供了判断一条直线是否为切线的代数条件。

以下是详细的推导和计算步骤。

7.11.1 利用对偶曲线求切线

二次曲线的矩阵形式为：

$$\boldsymbol{x}^T C \boldsymbol{x} = 0$$

在射影几何中，点构成曲线 $\boldsymbol{x}^T C \boldsymbol{x} = 0$，而切线构成对偶曲线。

若 C 可逆（即非退化二次曲线，如椭圆、双曲线），则切线 $\boldsymbol{l}$ 满足：

$$\boldsymbol{l}^T C^{-1} \boldsymbol{l} = 0$$

这里 $\boldsymbol{l} = [u, v, w]^T$ 是直线 $ux + vy + w = 0$ 的系数向量。

假设已知点 $P(x_0, y_0)$ 在曲线外，求过 P 的切线。

第一步：计算对偶矩阵 C^{-1}

> 如果 C 不可逆（例如曲线是两条相交直线），则不能直接求逆，需用伴随矩阵代替，此处假设曲线是非退化的。

第二步：建立切线的通式

设所求切线为 $\boldsymbol{l} = [u, v, w]^T$，即直线方程 $ux + vy + w = 0$。

由于切线必须经过点 $P(x_0, y_0)$，代入直线方程得：

$$ux_0 + vy_0 + w = 0 \implies w = -(ux_0 + vy_0)$$

此时，直线向量可以写成只含 u, v 的形式：

$$\boldsymbol{l} = \begin{bmatrix} u \\ v \\ -(ux_0 + vy_0) \end{bmatrix}$$

第三步：代入对偶条件求解

将上述 $\boldsymbol{l}$ 代入对偶二次曲线方程 $\boldsymbol{l}^T C^{-1} \boldsymbol{l} = 0$：

$$\begin{bmatrix} u & v & -(ux_0 + vy_0) \end{bmatrix} C^{-1} \begin{bmatrix} u \\ v \\ -(ux_0 + vy_0) \end{bmatrix} = 0$$

展开这个式子，你会得到一个关于 u 和 v 的**齐次二次方程**：

$$K_1 u^2 + K_2 uv + K_3 v^2 = 0$$

其中系数 K_1, K_2, K_3 由 B 的元素和点 P 的坐标决定。

第四步：解出斜率或直线系数

解上述关于 u, v 的方程。通常有两种情况：

1. **若** $v \neq 0$：令 $k = -\dfrac{u}{v}$（即斜率），或者直接解出比值 $\lambda = u/v$。方程变为：$K_1\lambda^2 + K_2\lambda + K_3 = 0$。解出两个根，对应两条切线。
2. **若** $v = 0$：说明有一条切线是垂直于 x 轴的（$x = \text{const}$）。

得到 u, v 的比例后，回代 $w = -(ux_0 + vy_0)$ 得到完整的直线系数 $[u, v, w]$。

7.11.2 利用极线求切点

虽然上面的步骤求出了切线，但**极线**在这里有一个非常巧妙的用途：**直接求出切点坐标**，从而避免解二次方程的繁琐系数展开。

点 P 的极线 L_{polar} 是连接两个切点 T_1, T_2 的直线（切点弦）。切线 PT_1 与曲线相切于 T_1。根据极点极线性质，切点 T_1 既在曲线 C 上，也在极线 L_{polar} 上。

利用极线求切线的替代步骤：

第一步：求极线：

计算向量 $\boldsymbol{s} = C\boldsymbol{p}$，其中 $\boldsymbol{p} = [x_0, y_0, 1]^T$。极线方程为 $\boldsymbol{s}^T\boldsymbol{x} = 0$，即 $s_1x + s_2y + s_3 = 0$。

第二步：求极线与曲线的交点：

联立极线方程和原曲线方程：

$$\begin{cases} \boldsymbol{x}^T C \boldsymbol{x} = 0 \\ \boldsymbol{s}^T \boldsymbol{x} = 0 \end{cases}$$

解这个方程组得到的两个解即为切点 T_1, T_2。

切线就是直线 PT_1 和 PT_2。

7.11.3 语法形式

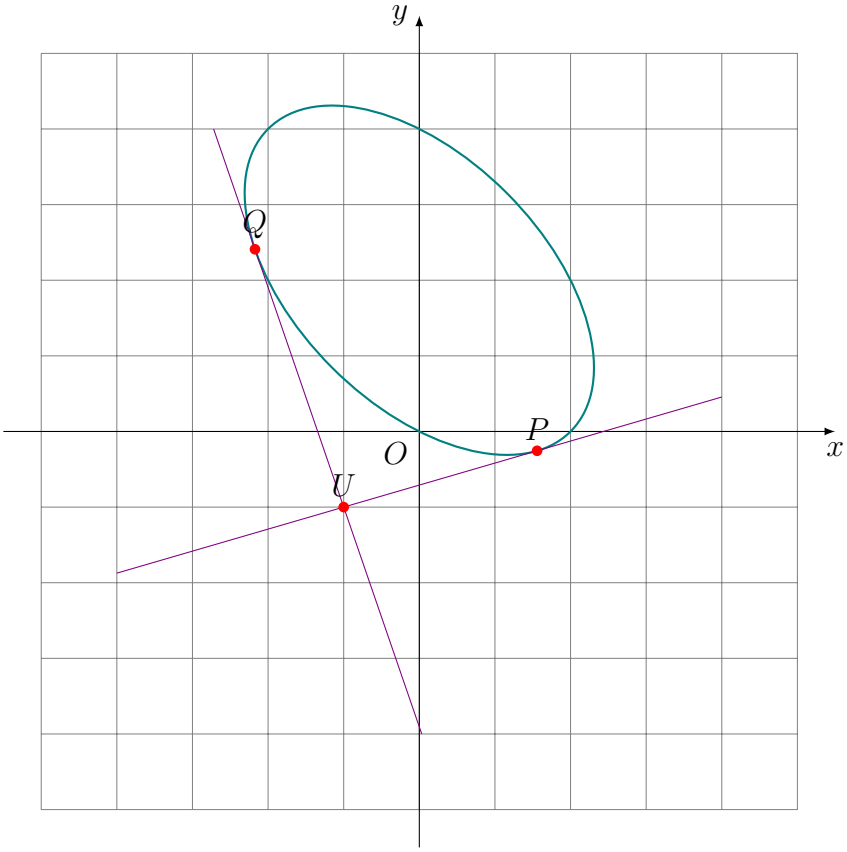
Syntax

- `conics/tangents={\C,\P,\Ta,\Tb}`：求圆锥曲线与直线的交点
  - `\C`：圆锥曲线的系数矩阵
  - `\P`：点的齐次坐标，列向量
  - `\Ta,\Tb`：切点的齐次坐标，列向量，顺序是过这两点直线的正方向

7.11.4 示例：椭圆的切线

```
\begin{tikzpicture}
  \tikzmath{
    \coefs{1} = 1; \coefs{2} = 1; \coefs{3} = 1;
    \coefs{4} = -2; \coefs{5} = -4; \coefs{6} = 0;
    % 点的齐次坐标
    \U{1,1} = -1; \U{2,1} = -1; \U{3,1} = 1;
  }
  \tikzset{
    conics/define/quadratic={\coefs,\A},
    conics/tangents={\A,\U,\P,\Q},
```

```
    conics/dehomogenize={\U,\UU},
    conics/dehomogenize={\P,\PP},
    conics/dehomogenize={\Q,\QQ},
  }

  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal,thick] (\A);
  \segment[draw=violet,clip=(-4:4,-4:4)] (\U,\P);
  \segment[draw=violet,clip=(-4:4,-4:4)] (\U,\Q);

  \coordinate (U) at (\UU{1},\UU{2});
  \coordinate (P) at (\PP{1},\PP{2});
  \coordinate (Q) at (\QQ{1},\QQ{2});

  \foreach \p/\placement in {P/above, Q/above, U/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```



7.12 分解退化的二次曲线

7.12.1 退化的二次曲线

在齐次坐标下，二次曲线（Conics）通常由一个 3×3 的对称矩阵 C 表示。若该二次曲线是**退化（Degenerate）**的，其核心代数特征是该矩阵的行列式为零。

二次曲线的齐次方程为：

$$x^T C x = 0$$

退化的判定条件

一条二次曲线被定义为退化，当且仅当矩阵 C 是**奇异矩阵**：

$$\det(C) = 0$$

根据矩阵 C 的**秩（Rank）**，退化情况可以进一步细分为以下几种：

表 7.2: 退化二次曲线的分类

矩阵的秩 $\text{rank}(C)$	几何解释	具体描述
秩 = 2	**两条相交直线或平行直线**	矩阵可以分解为两个线性因子的乘积：$C = \mathbf{l}_1\mathbf{l}_2^T + \mathbf{l}_2\mathbf{l}_1^T$。在复数域下，这也可能表示两条虚交直线（如一个点）。
秩 = 1	**一条重合直线**	矩阵可以写成一个列向量与其转置的乘积：$C = \mathbf{l}\mathbf{l}^T$。方程变为 $(ax + by + cw)^2 = 0$。
秩 = 0	**全平面**	所有系数均为 0，这种情形在几何讨论中通常被排除。

在对偶空间（Line Conics）中，如果表示直线的对称矩阵 C^* 退化，则意味着这组直线束通过同一个点或重合成一个点。其判定方式完全相同，即分析对偶矩阵的秩。

将其分解为两条直线的原理主要基于**射影几何中的矩阵表示和伴随矩阵（Cofactor Matrix）的性质**。

算法的步骤 (Richter-Gebert, 2011, 页 190)：

1. 计算退化二次曲线矩阵 A 的伴随矩阵 $B = \mathrm{adj}(C)$。
2. 定位 i：找到 $i \in \{1, 2, 3\}$ 使得 $|B_{ii}|$ 最大。
3. 计算向量 $\boldsymbol{p}$：$\beta = \sqrt{B_{ii}}$，$\boldsymbol{p} = B(:, i)/\beta$。
4. 计算 $C = A + [p]_\times$。
5. 定位核心坐标 (r, c)：在整个矩阵 C 中找到绝对值最大的元素 C_{rc}。
6. 提取结果（C 第 c 列和第 r 行）：直线 $\boldsymbol{g} = C(:, c)$，直线 $\boldsymbol{h} = C(r, :)$

相较于原算法，这里采用了选主元来改善数值稳定性。

7.12.2 语法形式

Syntax

- `conics/decompose={\C,\l,\m}`: 分解退化二次曲线为两条直线，$C = \boldsymbol{l} \cdot \boldsymbol{m}^T$

7.13 两圆锥曲线的交点

假设我们有两个二次曲线，并希望找到它们的交点。如果这两个曲线是圆，那么问题很简单，可以直接求出闭式解。然而，如果我们面对的是一对双曲线，情况就会变得复杂。例如，一对双曲线可能产生四个交点，而不像圆通常只有两个交点。

不过，仍然有一种非常精妙的算法可以计算任意两个二次曲线之间的交点。本文将讨论这一算法，其详细描述可见于 Jürgen Richter-Gebert 的著作 (Richter-Gebert, 2011, 页 196)。

7.13.1 二次曲线束

将二次曲线写成矩阵形式允许我们进行代数运算：例如，我们可以通过将两个二次曲线的矩阵相加来得到一个新的二次曲线。这种代数操作是下文描述的交点计算算法的关键。

假设我们有两个由对称矩阵 $C_1, C_2 \in \mathbb{R}^{3\times 3}$ 表示的二次曲线，其交点为 $p_1, \ldots, p_4$。这意味着对于每个 p_i，都有 $p_i^T C_1 p_i = p_i^T C_2 p_i = 0$。

根据线性性质，这意味着对于任何系数 $\lambda, \mu \in \mathbb{R}$，同样有：

$$p_i^T(\lambda C_1 + \mu C_2)p_i = 0$$

这定义了一个**二次曲线束**（pencil of conic sections），它们全部经过这四个交点。如果我们绘制由上述两条双曲线生成的其他二次曲线，我们会发现其中包含双曲线、椭圆，甚至还有一对经过这四个交点的**交叉直线**。

这对交叉直线是定位交点 p_i 的关键。毕竟，求解直线与二次曲线的交点非常简单——只需要解一个二次方程。

形式上，这对交叉直线构成了一个**退化二次曲线**（degenerate conic）。如果一个矩阵的行列式为零，则它表示一个退化二次曲线。因此，我们可以通过求解以下方程来找到这个退化曲线：

$$\det(\lambda C_1 + \mu C_2) = 0$$

这是一个三次方程（因为 C_1 和 C_2 是 3×3 矩阵）。随后，我们只需识别出构成该退化曲线的两条直线，并将它们与原始二次曲线之一求交即可。

算法总结如下：

1. 找到不全为零的 λ, μ，使得 $\det(\lambda C_1 + \mu C_2) = 0$。
2. 将退化二次曲线 $\lambda C_1 + \mu C_2$ 分解为一对直线 g 和 h。
3. 将直线 g 和 h 分别与 C_1 求交，确定所有四个交点。

7.13.2 步骤详解

1. 寻找退化二次曲线

虽然 Richter-Gebert 给出了在齐次坐标下求解三次方程的详细算法，但如果你可以使用线性代数库，会有更简单的方法。

- 如果 C_2 本身是退化的（即 $\det(C_2) = 0$），那么取 $\lambda = 0, \mu = 1$ 即可。
- 如果 C_2 是非退化的（可逆），我们可以令 $\lambda = 1$，两边乘以 C_2^{-1}，转而求解：

$$\det(\mu I - (-C_1 C_2^{-1})) = 0$$

  这实际上是在求矩阵 $-C_1C_2^{-1}$ 的**特征值** μ！标准线性代数库可以轻松计算它。(或者采用求**广义特征值**的 QZ 算法)

2. 将退化曲线分解为直线

假设我们有一个退化二次曲线矩阵 $A \in \mathbb{R}^{3\times 3}$。它实际上是一对直线 $g, h \in \mathbb{R}^3$。如果点 x 在第一条直线上，则 $g^T x = 0$。A 矩阵（在缩放意义下）必须等于对称化矩阵 $gh^T + hg^T$。

从 A 中提取 g 和 h 的具体步骤：1. 计算 A 的伴随矩阵（cofactor matrix）$B \leftarrow A^{\triangle}$。2. 找到 B 对角线上一个非零元素的索引 i。3. 计算 $\beta \leftarrow \sqrt{B(i,i)}$。4. 令 $p \leftarrow B(:,i)/\beta$。5. 令 $C \leftarrow A + [p]_\times$（其中 $[p]_\times$ 是 p 的叉积矩阵）。6. 找到 C 中一个非零元素的索引 (i,j)，则 $g = C(i,:)$，$h = C(:,j)$。

3. 直线与二次曲线求交

见 直线与圆锥曲线的交点[p. 150]。

7.13.3 语法形式

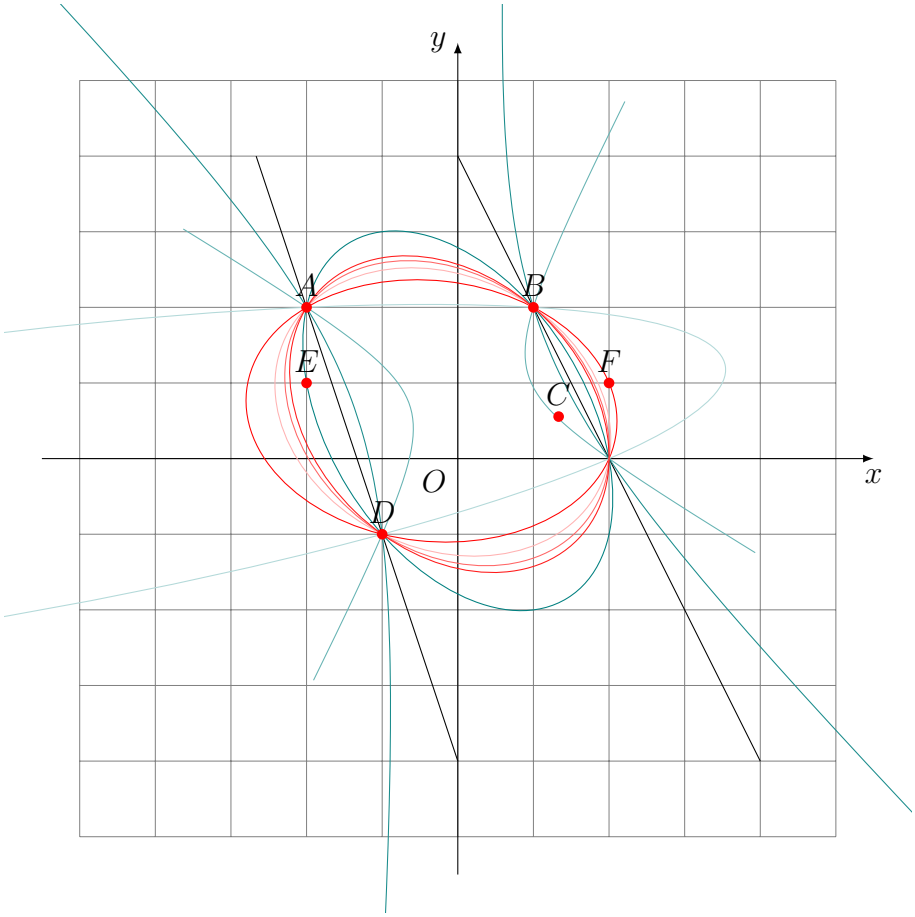
Syntax

- `conics/intersect cc={\Ca,\Cb,\C,\l,\m}`: 采用 Richter-Gebert 算法求过两圆锥曲线的交点的退化二次曲线（两条直线）
  - `\Ca,\Cb`: 两圆锥曲线的系数矩阵
  - `\l,\m`: 两直线的齐次坐标，列向量

7.13.4 示例：圆锥曲线束

```
\begin{tikzpicture}
  \tikzmath{
    % 5 点的齐次坐标
    \A{1,1} = -2; \A{2,1} = 2; \A{3,1} = 1;
    \B{1,1} = 1; \B{2,1} = 2; \B{3,1} = 1;
    \C{1,1} = 2; \C{2,1} = 0; \C{3,1} = 1;
    \D{1,1} = -1; \D{2,1} = -1; \D{3,1} = 1;
    \E{1,1} = -2; \E{2,1} = 1; \E{3,1} = 1;
    \F{1,1} = 2; \F{2,1} = 1; \F{3,1} = 1;
  }
  \tikzset{
    conics/define/five points={\A,\B,\C,\D,\E,\Ca},
    conics/define/five points={\A,\B,\C,\D,\F,\Cb},
    conics/intersect cc={\Ca,\Cb,\C,\l,\m},
  }

```

```
  \clip (-6,-6) rectangle (6,6);
  \axes[grid] (-5:5,-5:5);
  \conic[draw=teal] (\Ca);
  \conic[draw=red] (\Cb);
  \segment[draw,clip=(-4:4,-4:4)] (\l);
  \segment[draw,clip=(-4:4,-4:4)] (\m);

  \foreach \x in {30,60,...,90}{
    \tikzset{%color
      mat/combine={\Ca,\Cb,3,3,\x/100,-0.75,\C},
    }
    \conic[draw=teal!\x] (\C);
  }

  \foreach \x in {30,60,...,90}{
    \tikzset{%color
      mat/combine={\Ca,\Cb,3,3,\x/100,0.75,\C},
    }
    \conic[draw=red!\x] (\C);
  }

  \coordinate (A) at (\A{1,1},\A{2,1});
  \coordinate (B) at (\B{1,1},\B{2,1});
  \coordinate (C) at (\C{1,1},\C{2,1});
  \coordinate (D) at (\D{1,1},\D{2,1});
  \coordinate (E) at (\E{1,1},\E{2,1});
  \coordinate (F) at (\F{1,1},\F{2,1});

  \foreach \p/\placement in {A/above, B/above, C/above, D/above, E/
   above, F/above}{
    \fill[red] (\p) circle (2pt);
    \draw (\p) node[\placement] {$\p$};
  }
\end{tikzpicture}
```


y
x
A
B
E
F
C
O
D

参考文献

Richter-Gebert, J. (2011). *Perspectives on Projective Geometry: A Guided Tour Through Real and Complex Geometry*. Springer.